

Projeto e Análise de Algoritmos

Universidade Federal da Fronteira Sul
Programa de Pós-Graduação Profissional em
Computação Aplicada (PPGPCA)

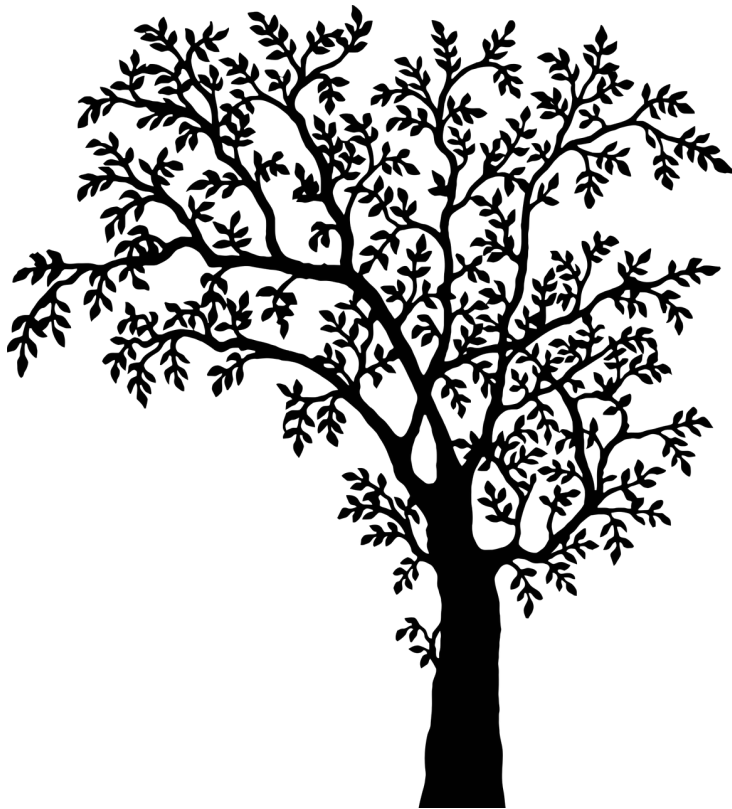
Denio Duarte
duarte@uffs.edu.br
Samuel Feitosa
samuel.feitosa@uffs.edu.br

Árvores

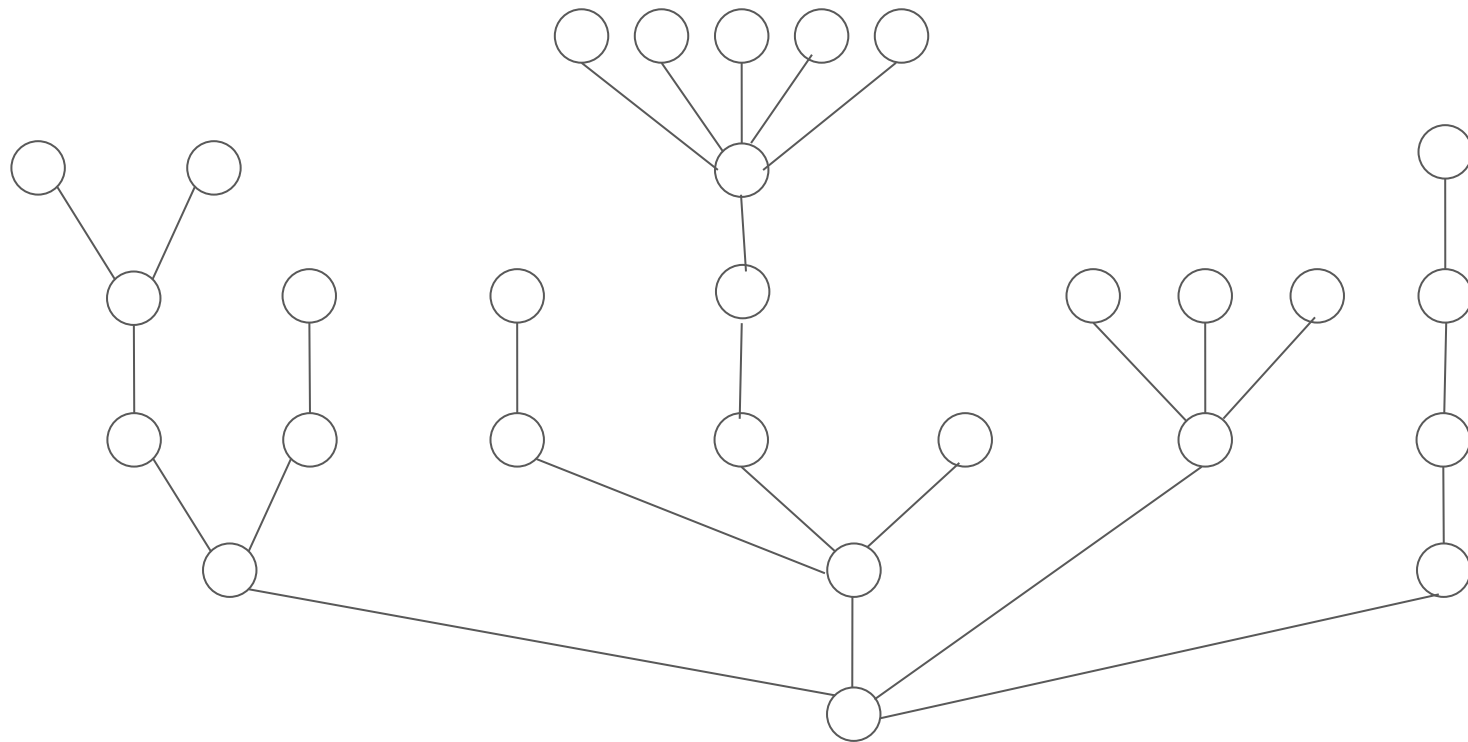
Árvore



Abstraindo a estrutura de uma árvore

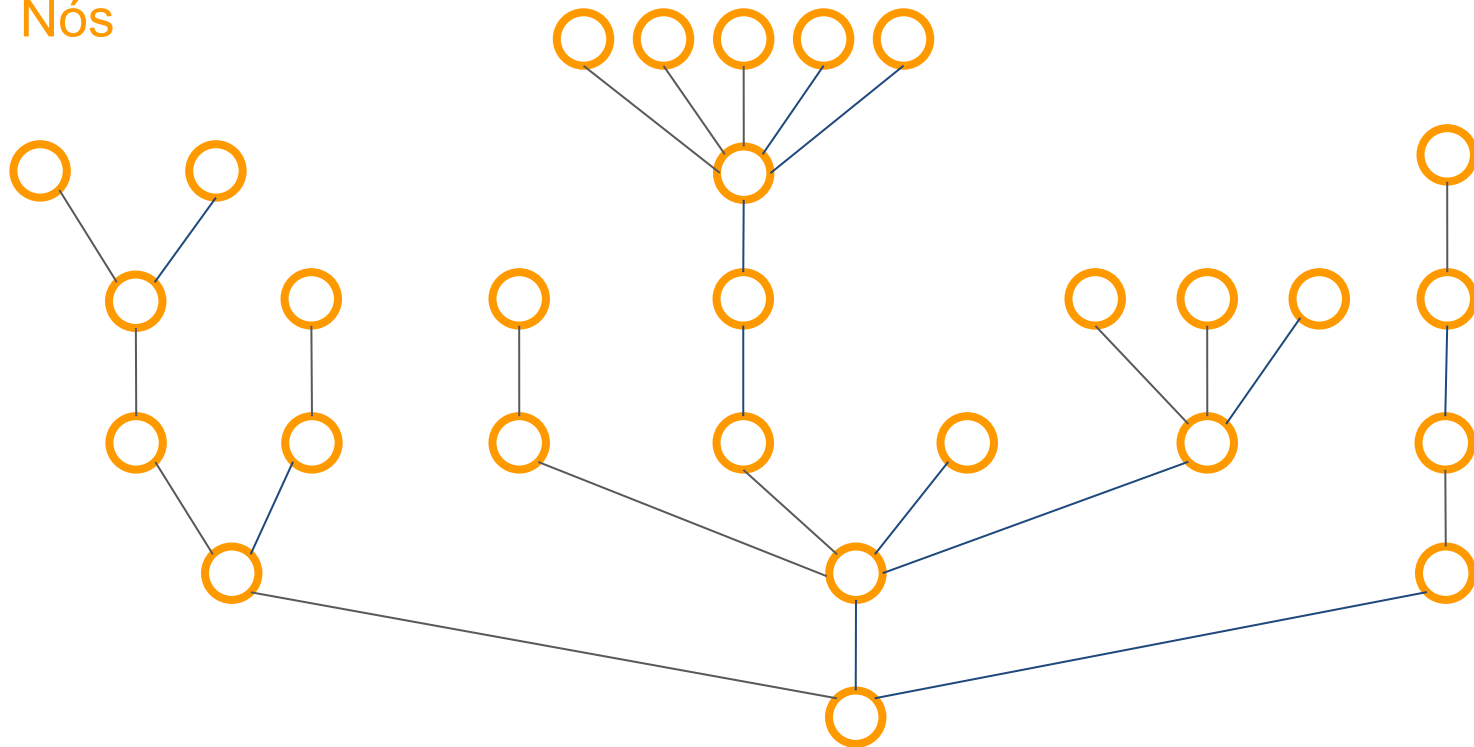


Abstraindo a estrutura de uma árvore



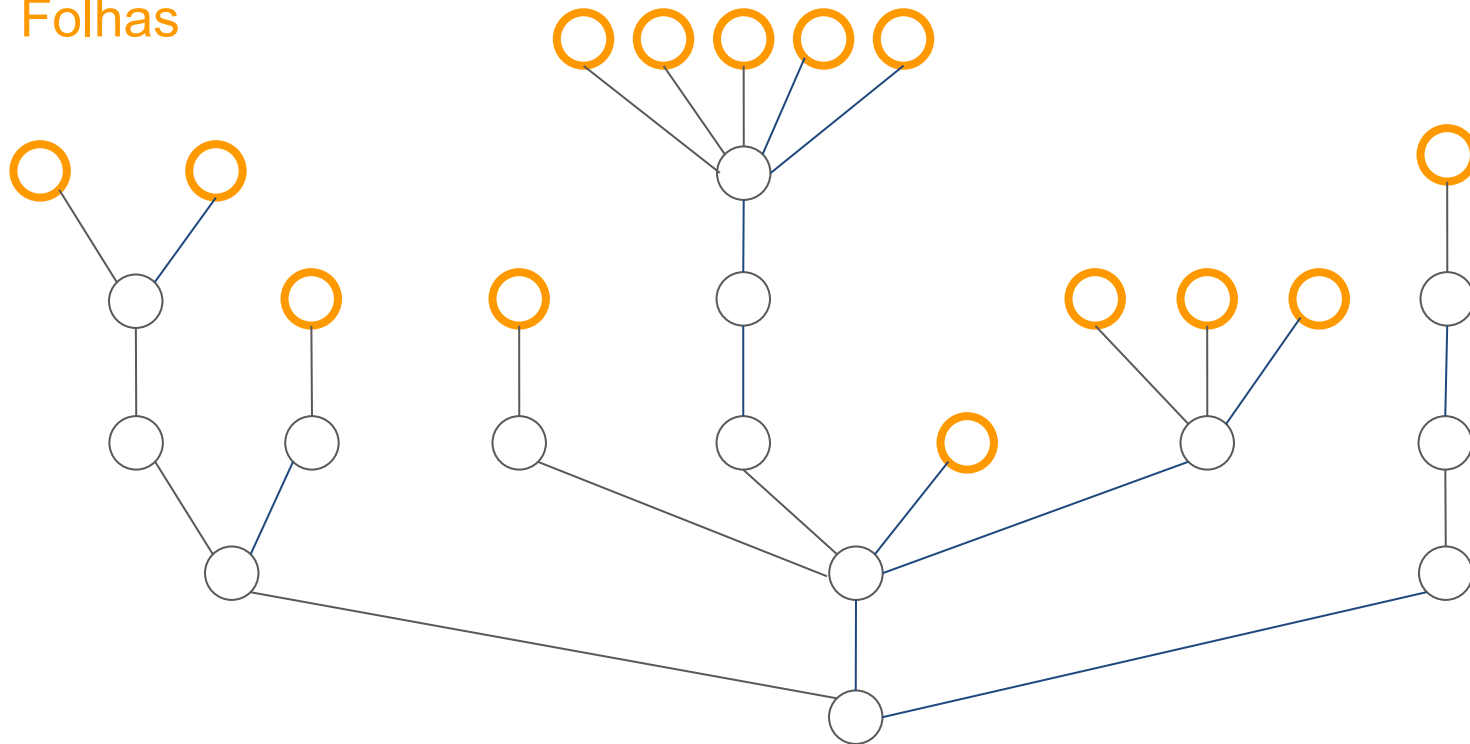
Abstraindo a estrutura de uma árvore

Nós

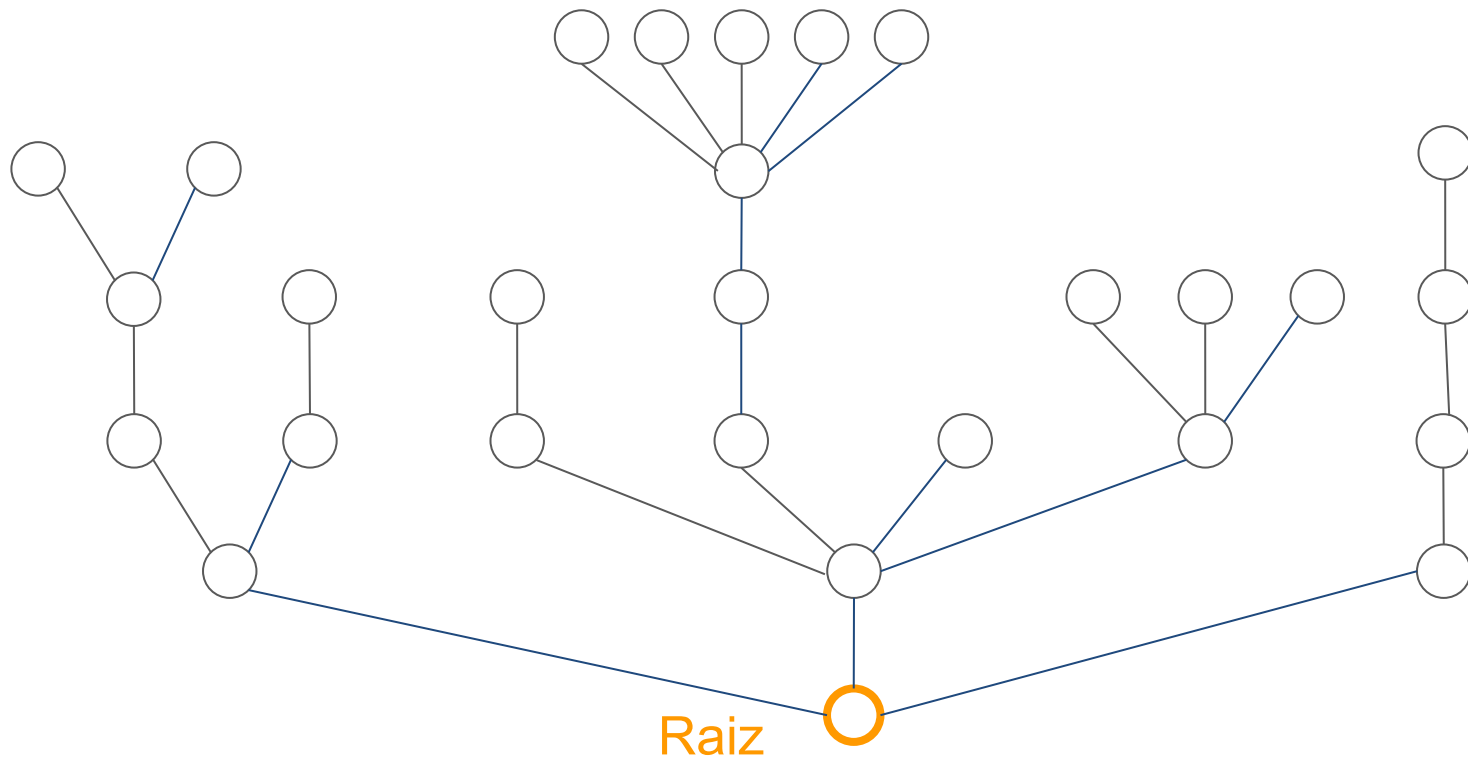


Abstraindo a estrutura de uma árvore

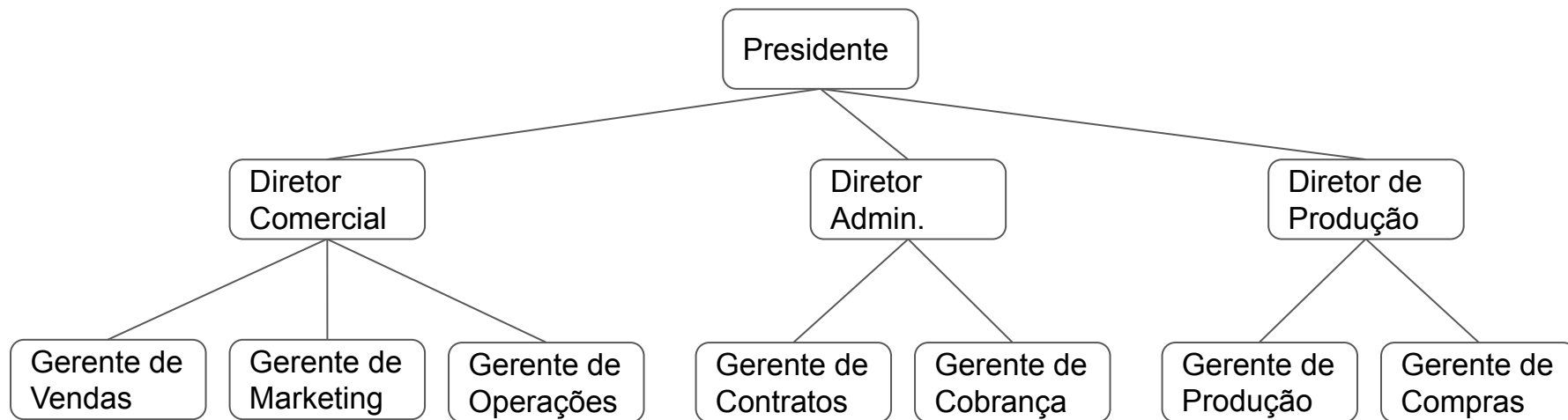
Folhas



Abstraindo a estrutura de uma árvore

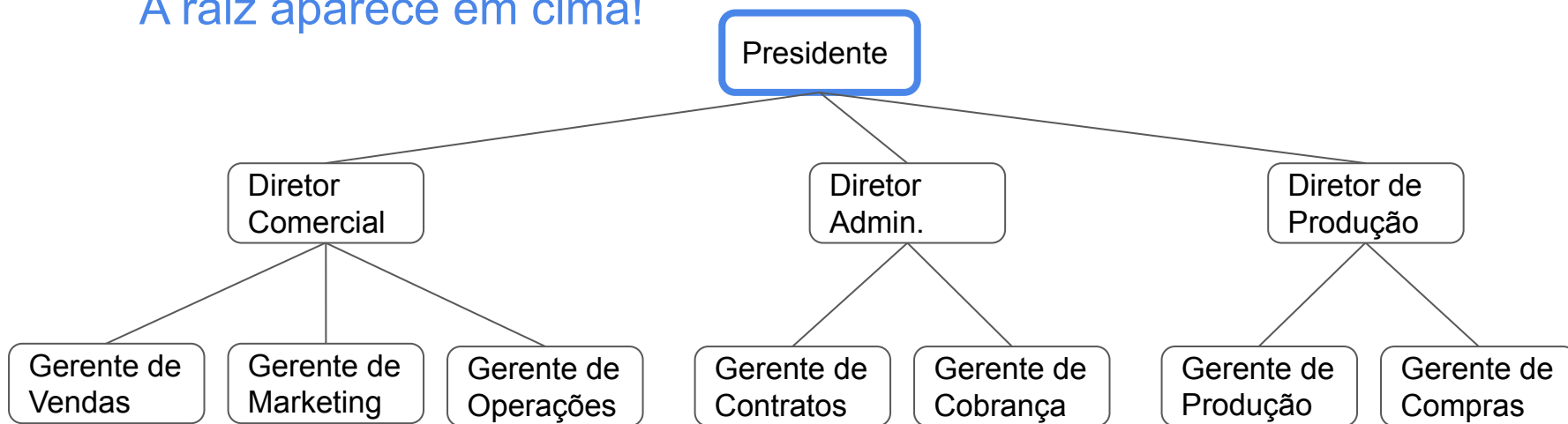


Exemplo de uso: organograma de uma empresa

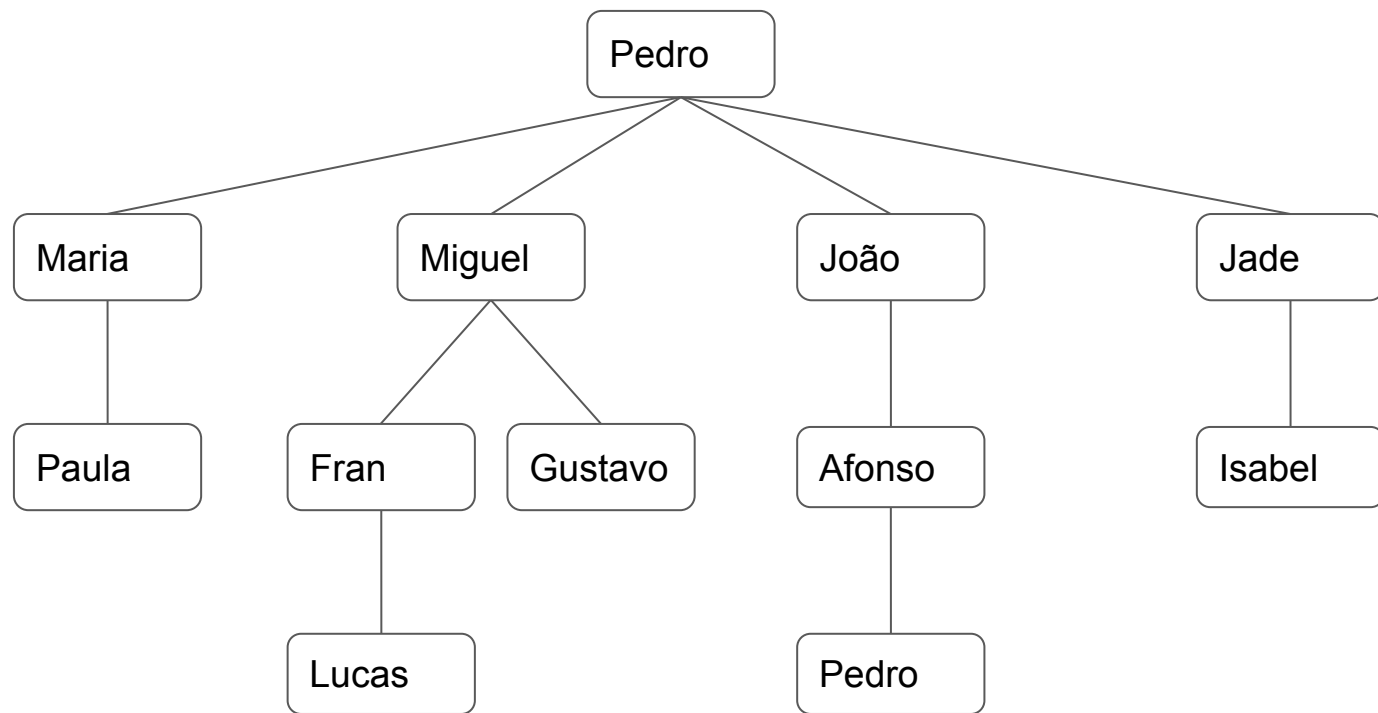


Exemplo de uso: organograma de uma empresa

A raiz aparece em cima!



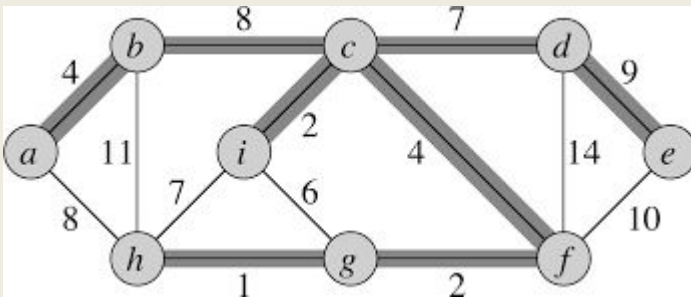
Exemplo de uso: descendência de uma pessoa



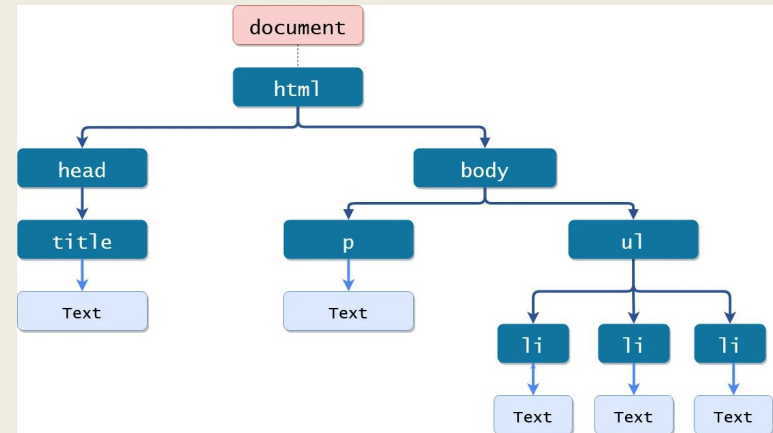
Árvores - uso na Computação

- Árvores são amplamente utilizadas na Computação
 - como uma ferramenta para descrever propriedades de algoritmos e
 - como uma estrutura de dados de fato

Árvore Geradora Mínima

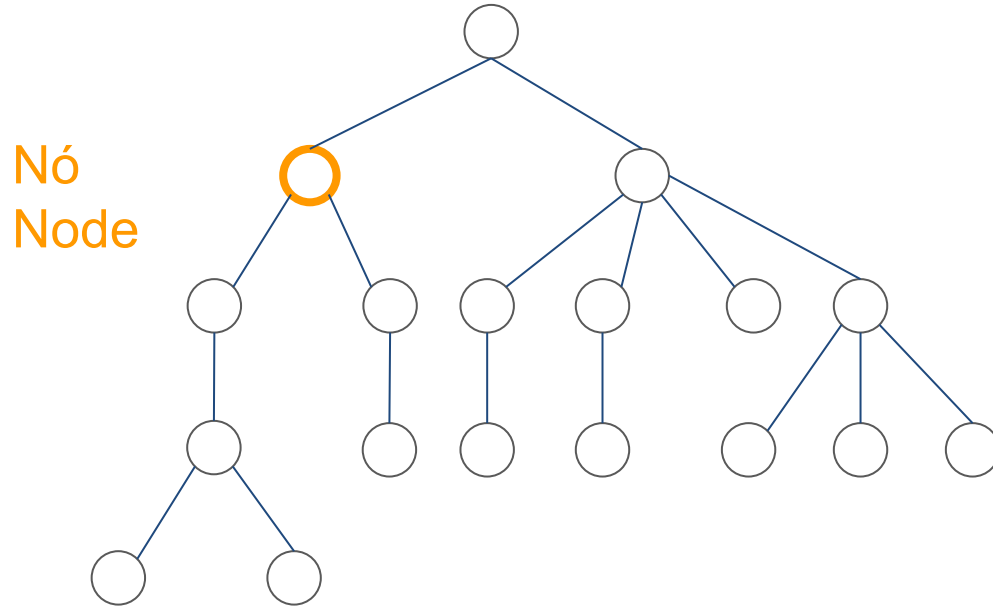


Document Object Model (DOM) HTML



Árvore - definição

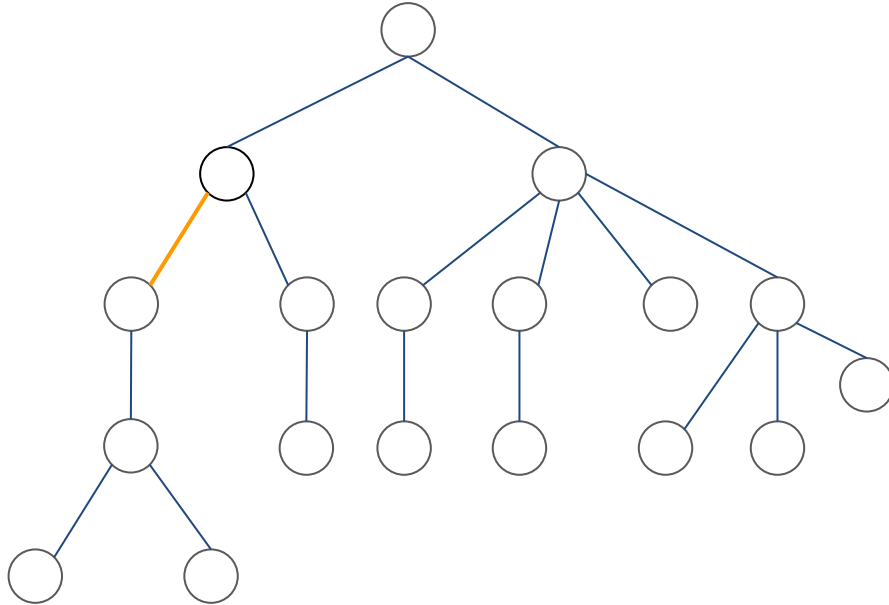
- Um **nó** (ou vértice) contém uma informação útil



Árvore - definição

- Um **nó** (ou vértice) contém uma informação útil
- Uma **aresta** é uma conexão entre dois nós

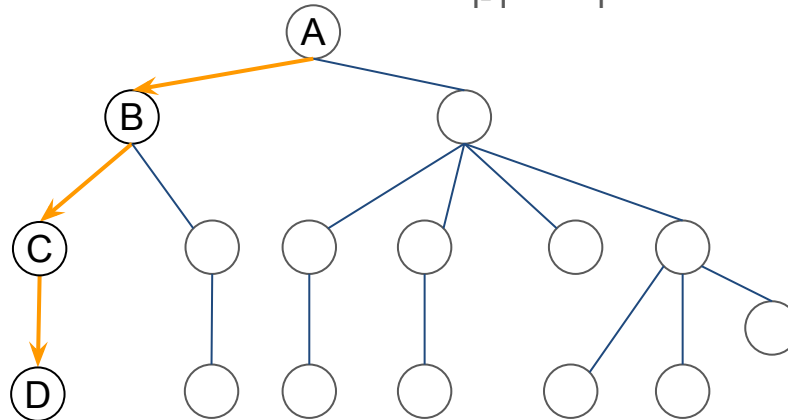
Aresta
Edge



Árvore - definição

- Um **nó** (ou vértice) contém uma informação útil
- Uma **aresta** é uma conexão entre dois nós
- Um **caminho** é uma sequência de nós $\langle v_0, v_1, v_2, \dots, v_k \rangle$ tal que
 - todos os nós da sequência são distintos
 - existe uma aresta conectando v_{i-1} e v_i para $i = 1, 2, \dots, k$

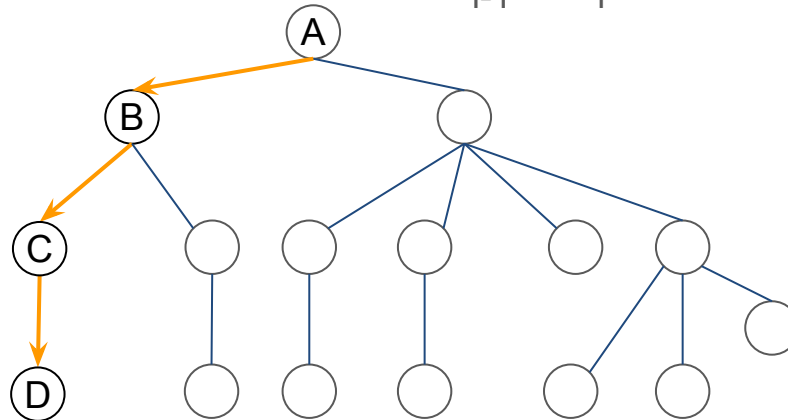
Caminho
Path
 $\langle A, B, C, D \rangle$



Árvore - definição

- Um **nó** (ou vértice) contém uma informação útil
- Uma **aresta** é uma conexão entre dois nós
- Um **caminho** é uma sequência de nós $\langle v_0, v_1, v_2, \dots, v_k \rangle$ tal que
 - todos os nós da sequência são distintos
 - existe uma aresta conectando v_{i-1} e v_i para $i = 1, 2, \dots, k$

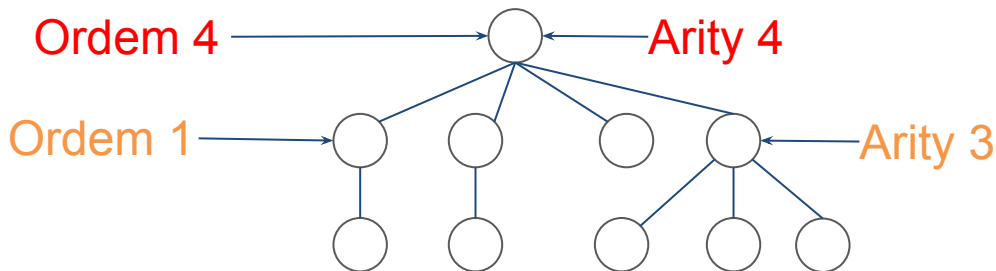
Caminho
Path
 $\langle A, B, C, D \rangle$



existe **exatamente** um **caminho**
conectando cada par de vértices

Árvore - definição

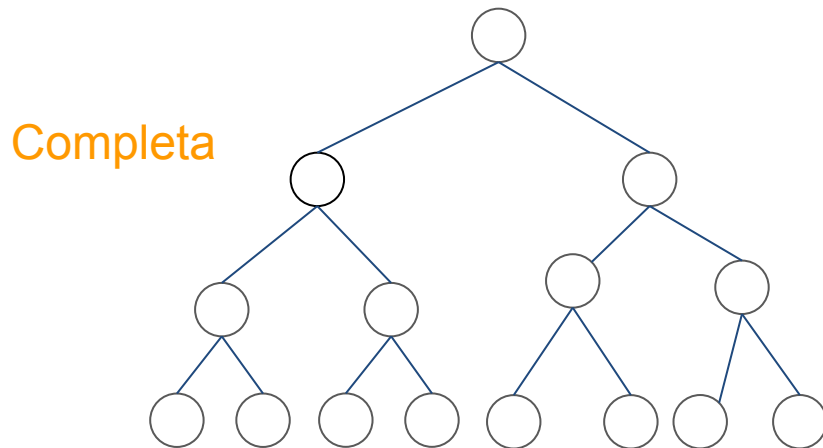
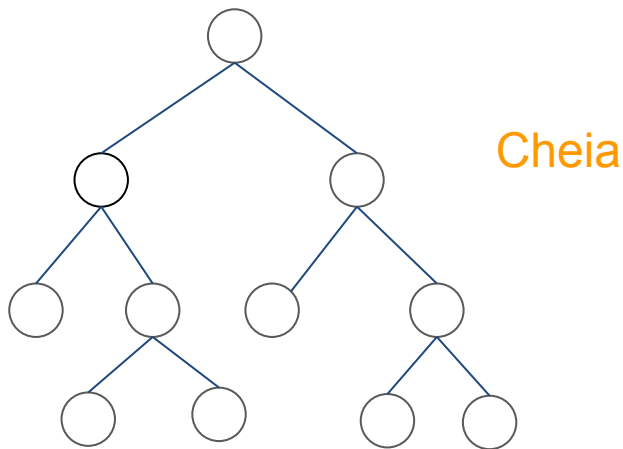
- A **ordem** de uma árvore é dada pelo nó com maior número de arestas



- Ordem: binária, ternária, n-ária
- Uma árvore é **n-ária completa** se todos os nós internos tem exatamente **n filhos**

Árvore - definição

- Uma árvore é **n-ária** é **cheia** se todos os nós internos tem exatamente **n filhos**
- Uma **árvore cheia** é **completa** se todas as folhas estiverem no mesmo nível (altura)

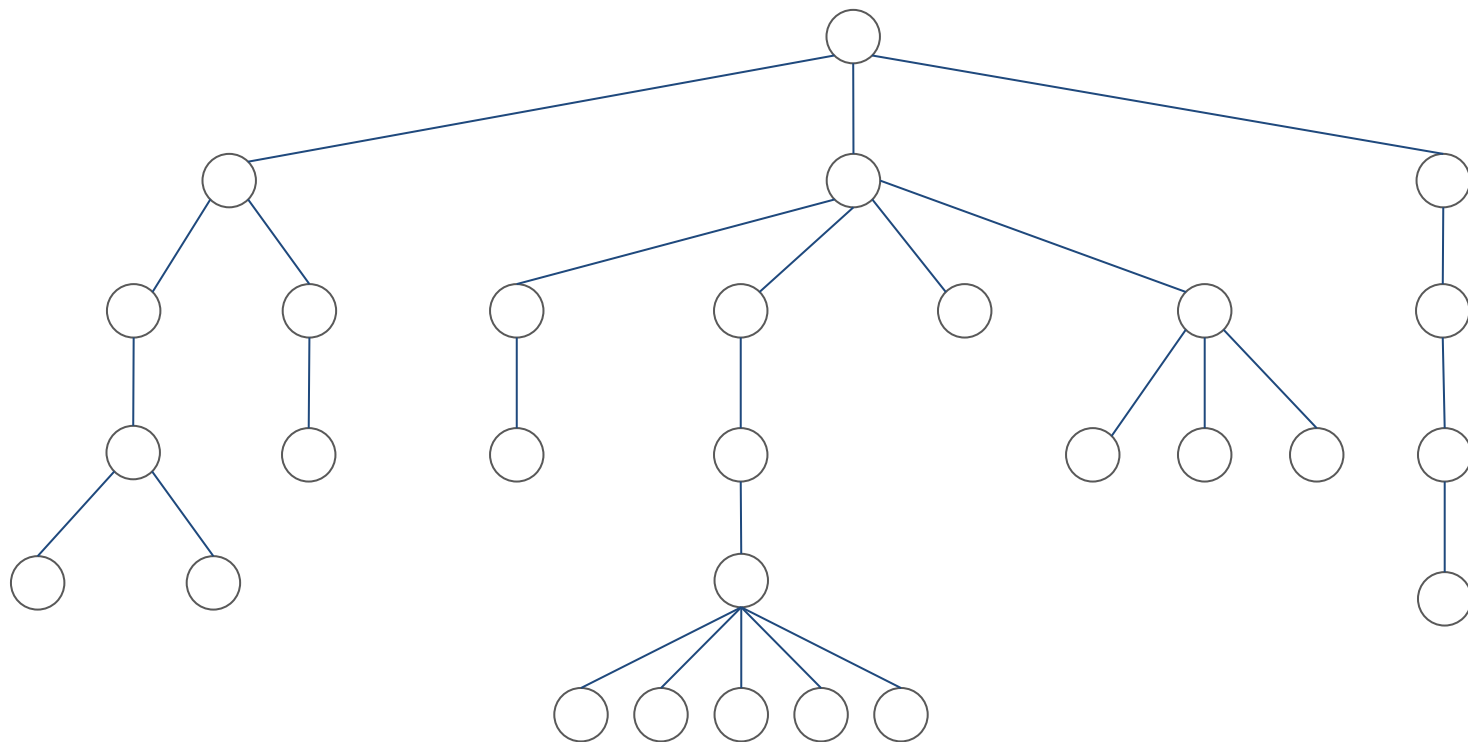


Árvore - definição

- **Profundidade** de um nó (**Depth** of a node)
 - Distância da raiz (tamanho do caminho até a raiz)
 - $\text{depth}(\text{root})=0$
- **Altura (Level)**
 - Todos nós na mesma profundidade formam a altura
 - $\text{level}(\text{root})=0$
 - $\text{level}(\text{root's children})=1$
 - Altura da árvore (**height**)
 - Altura da folha mais profunda

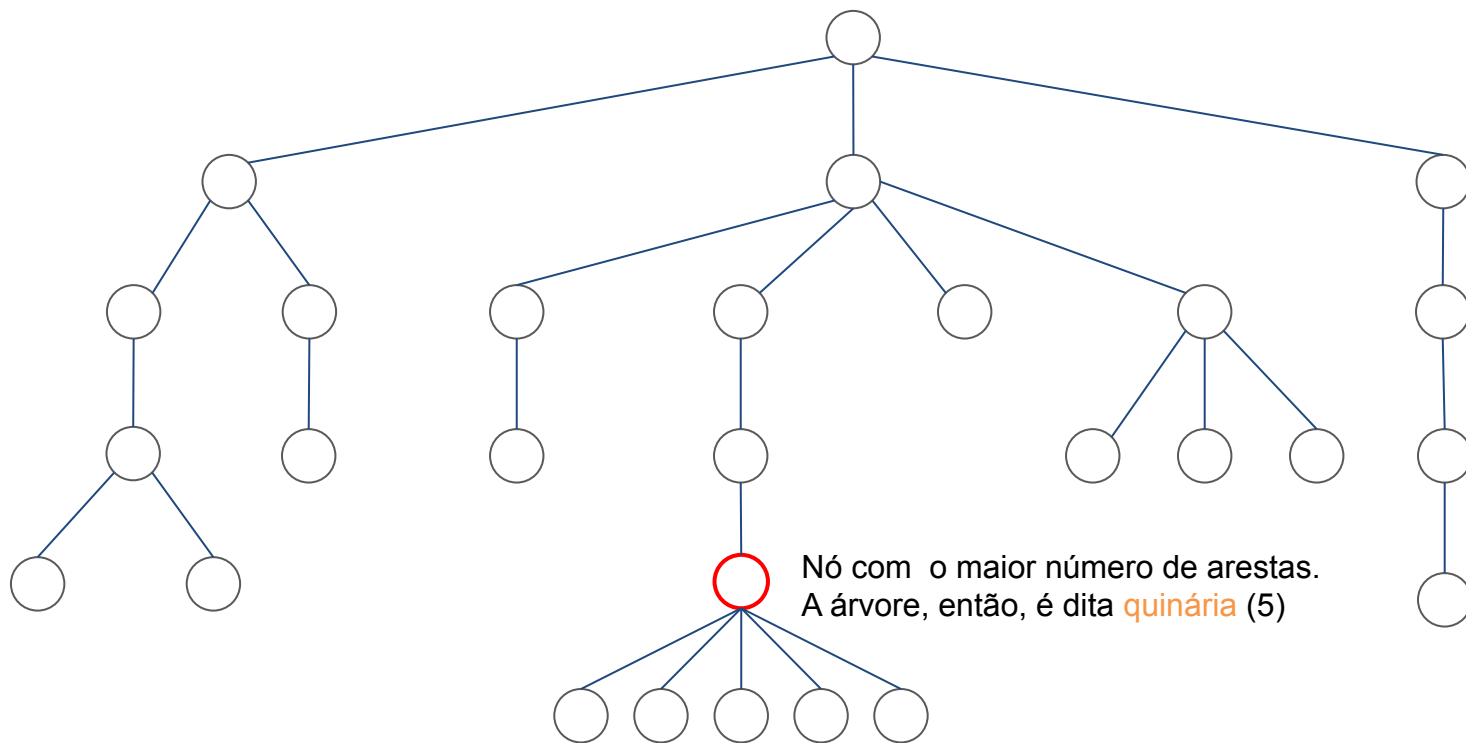
Árvore - definição

É uma árvore!



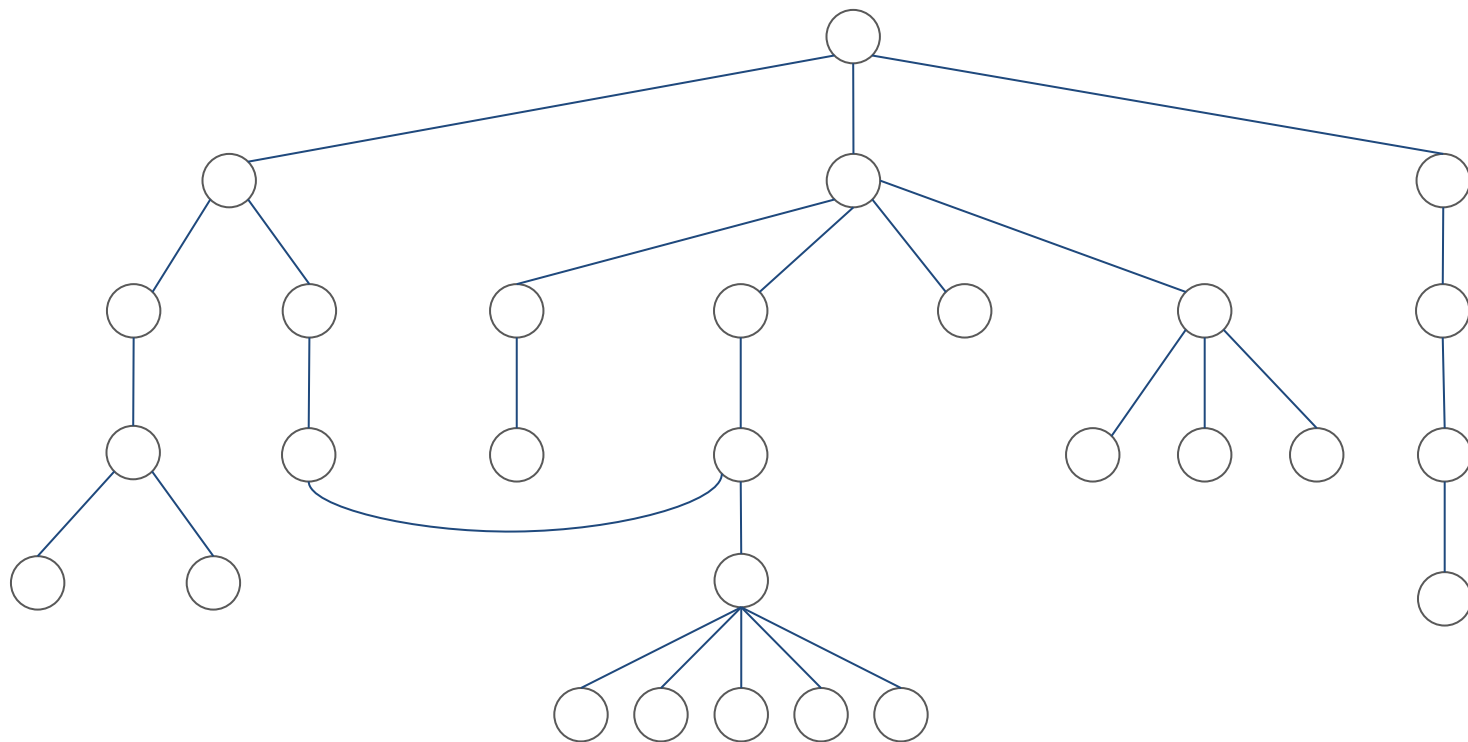
Árvore - definição

É uma árvore!



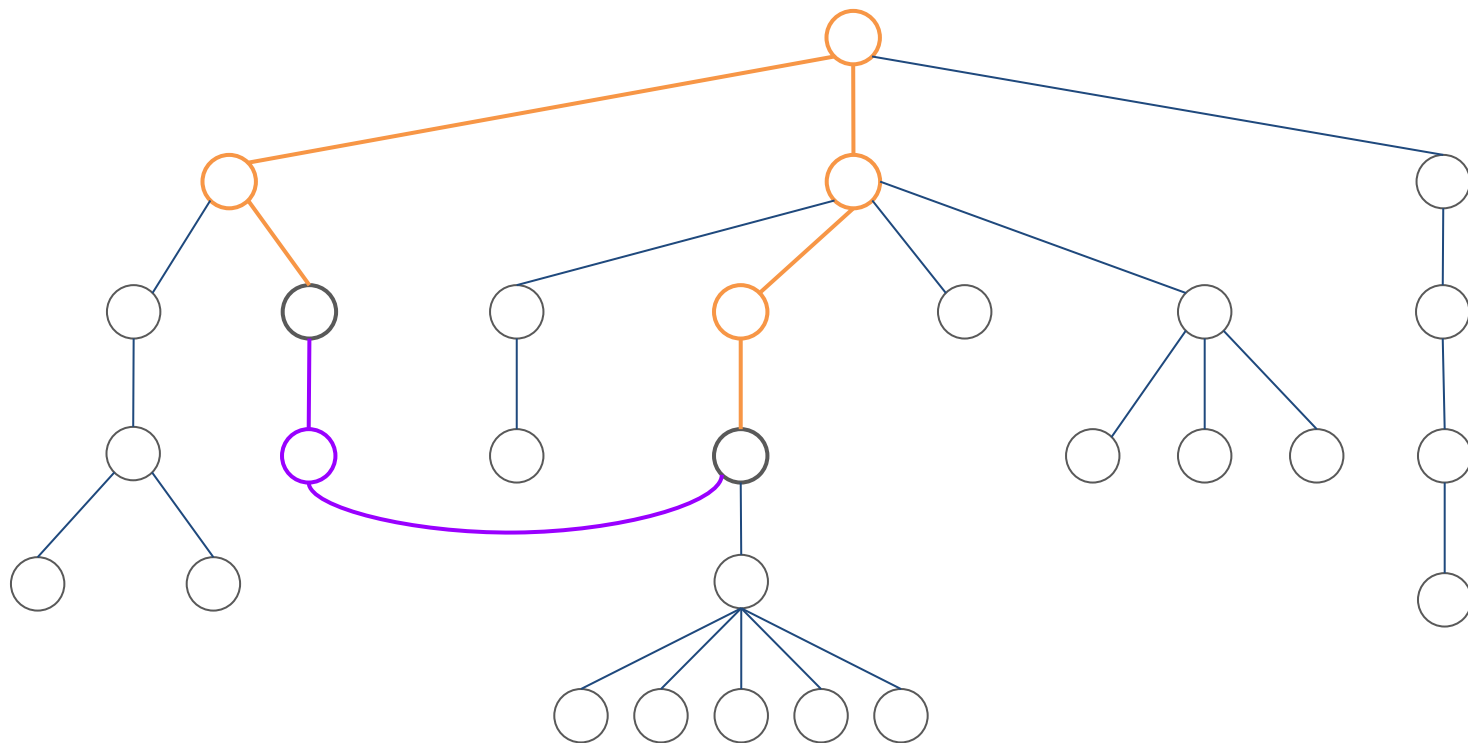
Árvore - definição

Não é uma árvore!



Árvore - definição

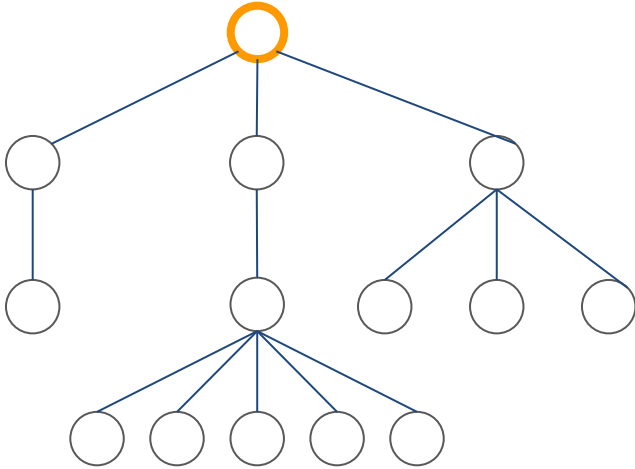
Não é uma árvore!



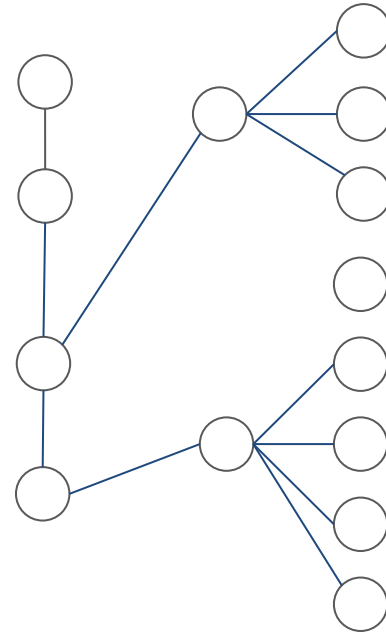
Árvore enraizada - terminologia

Árvore enraizada

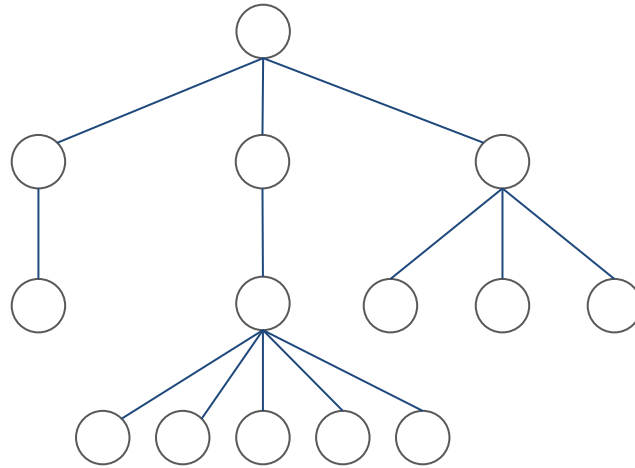
Raiz



Árvore não enraizada

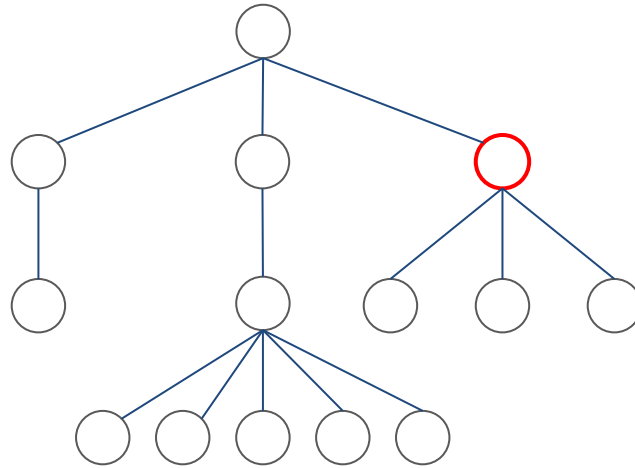


Árvore enraizada - terminologia



Árvore enraizada - terminologia

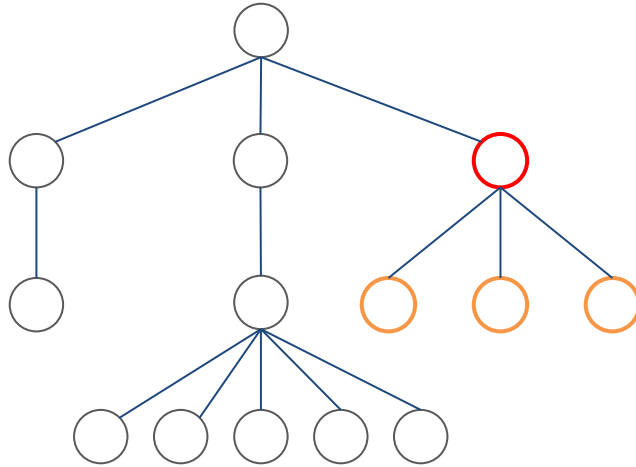
Nó (Node)



Árvore enraizada - terminologia

Nó (Node)

Filhos (Children)

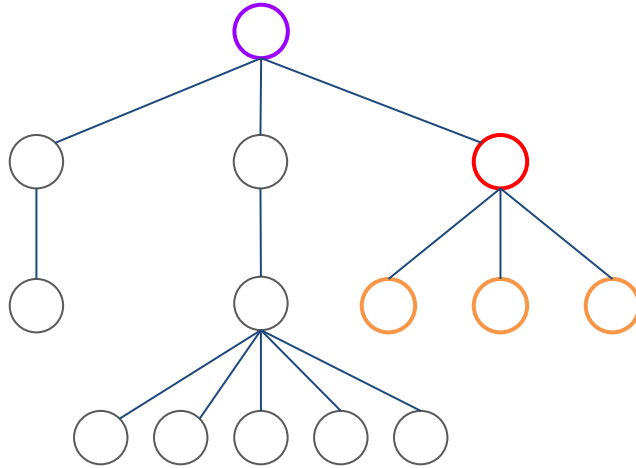


Árvore enraizada - terminologia

Pai (Parent)

Nó (Node)

Filhos (Children)

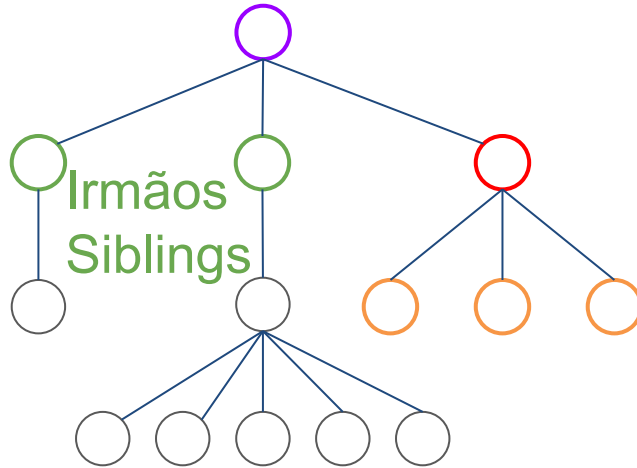


Árvore enraizada - terminologia

Pai (Parent)

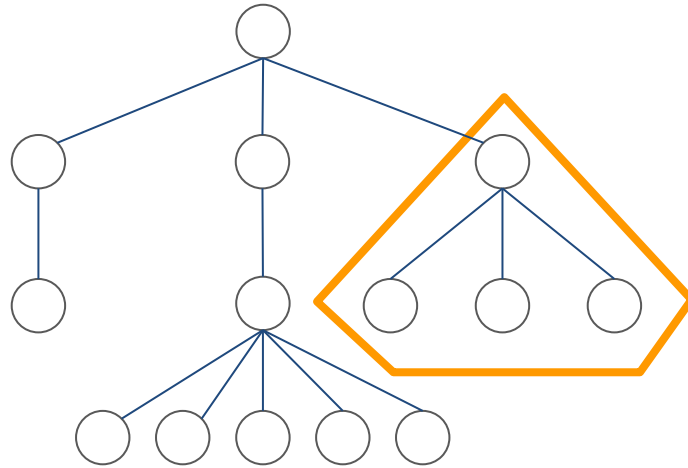
Nó (Node)

Filhos (Children)



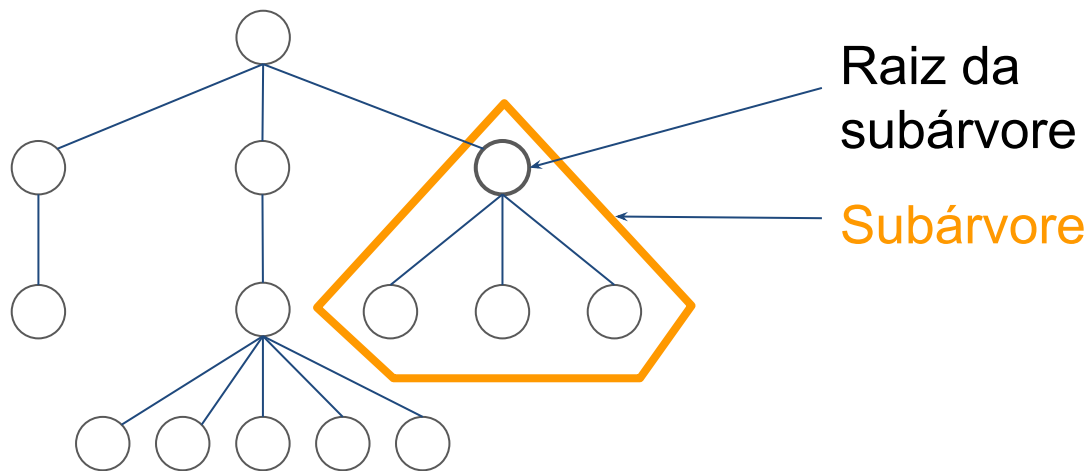
Root
Node
Leaf

Árvore enraizada - terminologia



Subárvore

Árvore enraizada - terminologia



Árvore binária

- (Definição recursiva.) Uma **árvore binária**
 - ou não contém nenhum nó
 - ou é composta de um nó conectado
 - a uma árvore binária chamada de **subárvore esquerda** do nó e
 - a uma árvore binária chamada de **subárvore direita** do nó

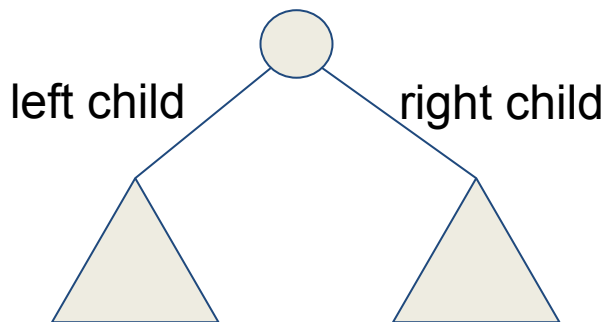
Árvore binária

- (Definição recursiva.) Uma **árvore binária**
 - ou não contém nenhum nó

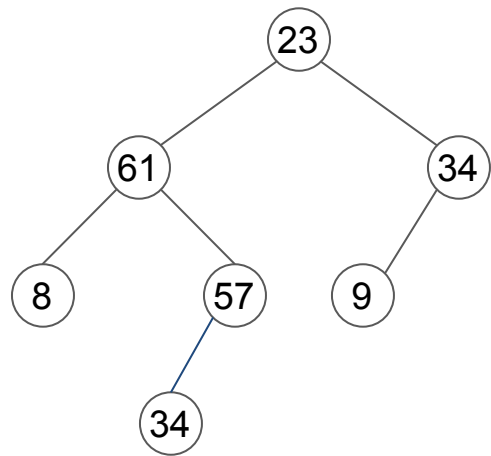
NULL

Árvore binária

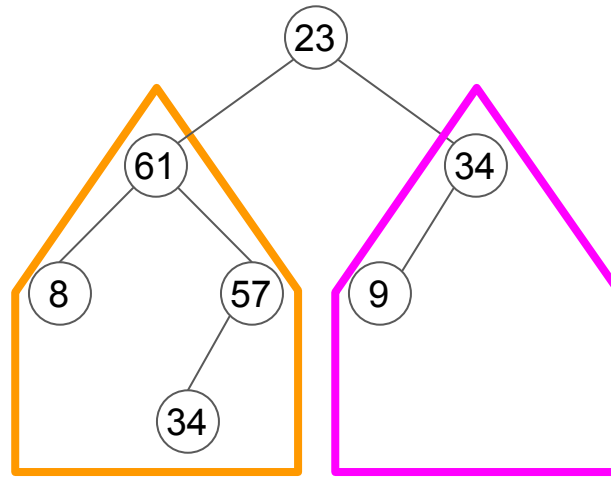
- (Definição recursiva.) Uma **árvore binária**
 - ou não contém nenhum nó
 - ou é composta de um nó conectado
 - a uma árvore binária chamada de **subárvore esquerda** do nó e
 - a uma árvore binária chamada de **subárvore direita** do nó



Árvore binária



Árvore binária

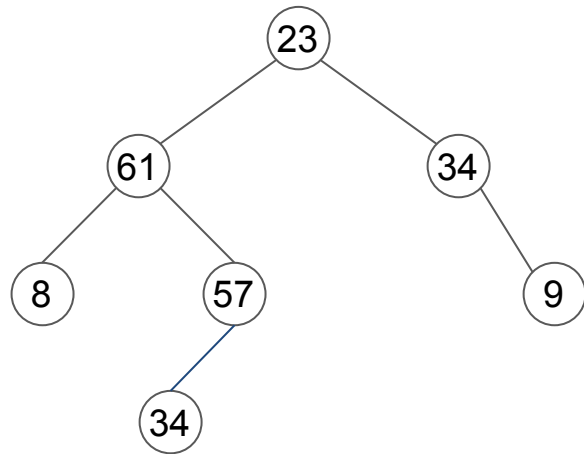
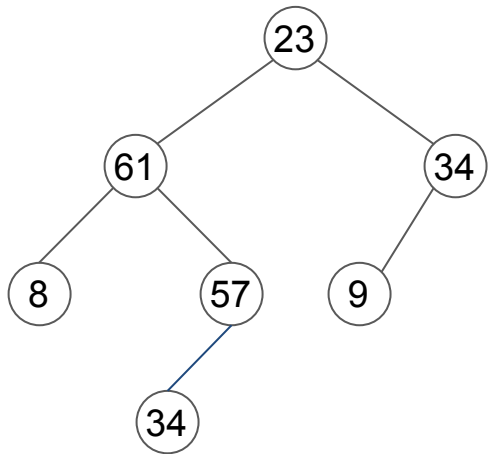


Subárvore
esquerda

Subárvore
direita

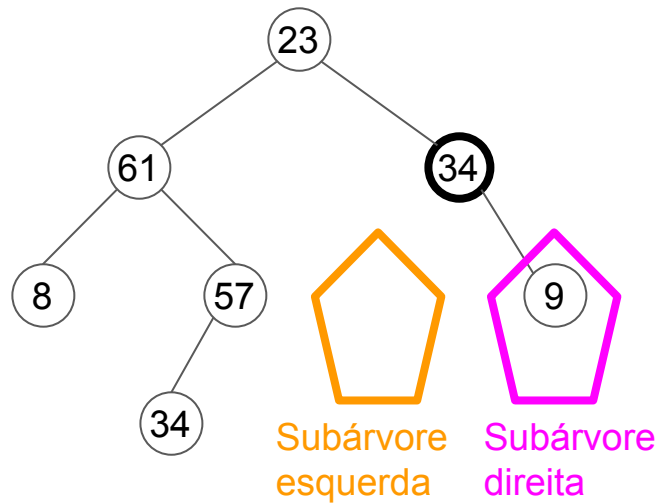
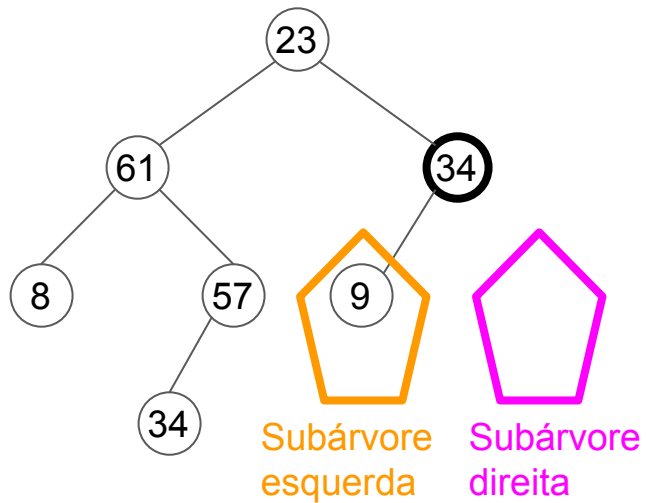
Árvore binária

Estas árvores binárias são iguais?



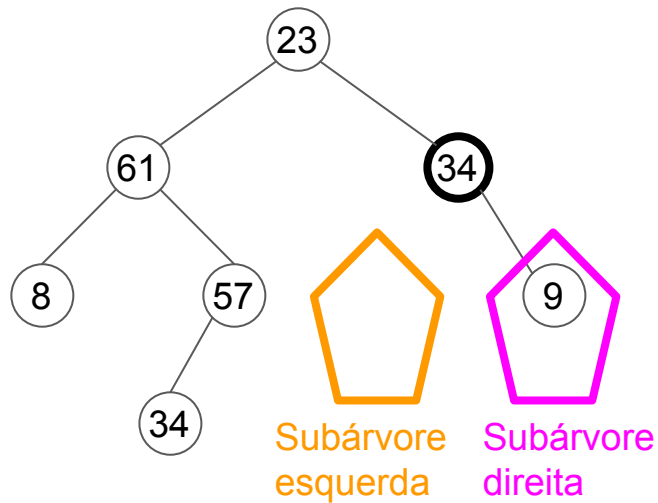
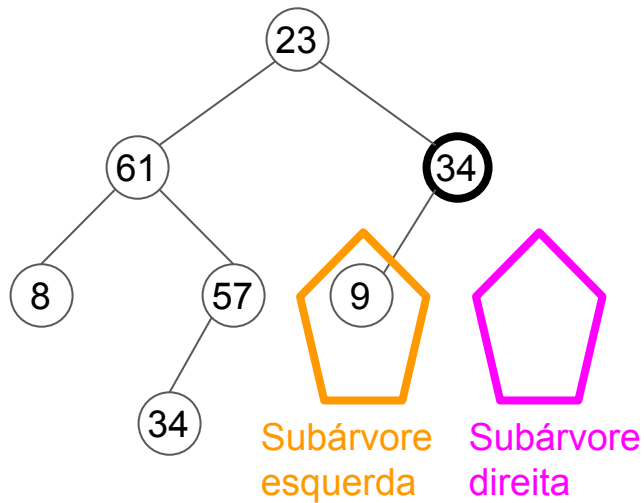
Árvore binária

Estas árvores binárias são iguais?



Árvore binária

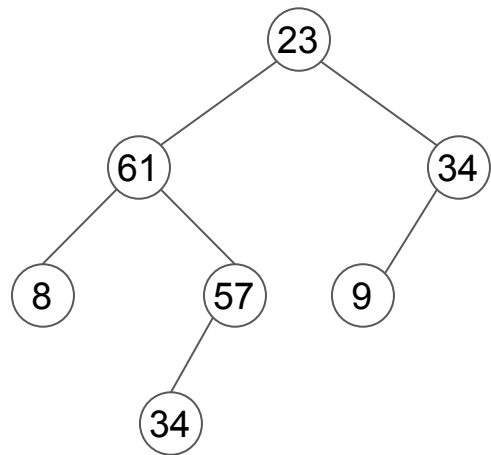
Estas árvores binárias são iguais? **NÃO**



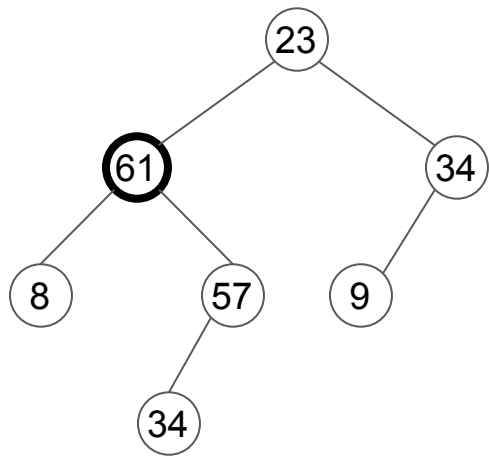
Árvore binária

- (Definição recursiva.) Uma **árvore binária**
 - ou não contém nenhum nó
 - ou é composta de um nó conectado
 - a uma árvore binária chamada de **subárvore esquerda** do nó e
 - a uma árvore binária chamada de **subárvore direita** do nó
- Em uma árvore binária, um filho de um nó é chamado de
 - **filho esquerdo** se ele é a raiz da subárvore esquerda do nó e
 - **filho direito** se ele é a raiz da subárvore direita do nó

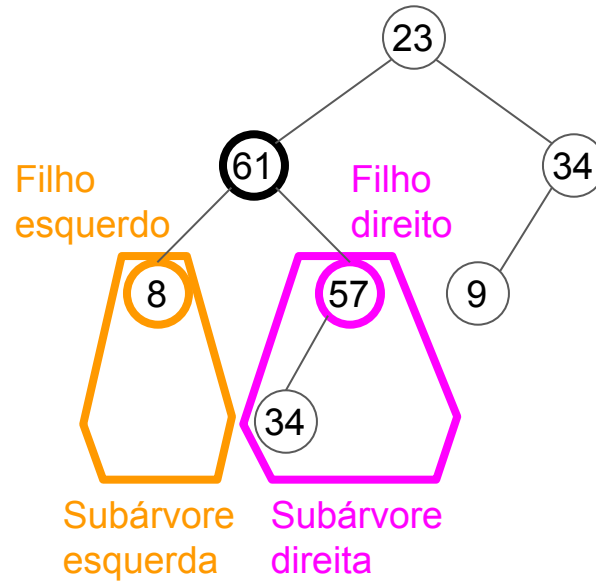
Árvore binária



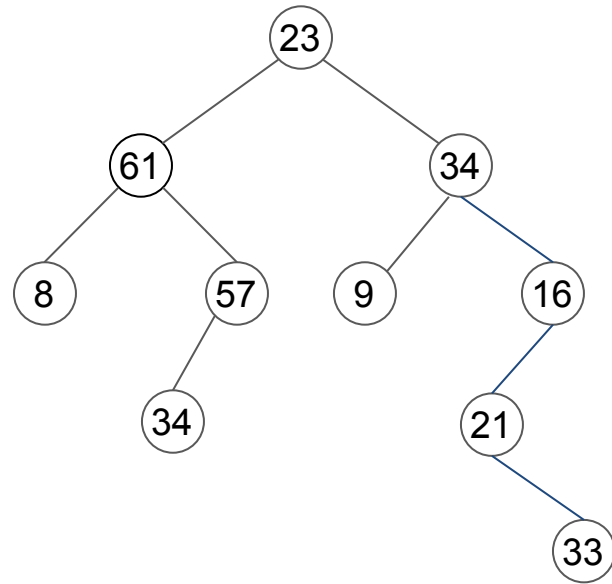
Árvore binária



Árvore binária

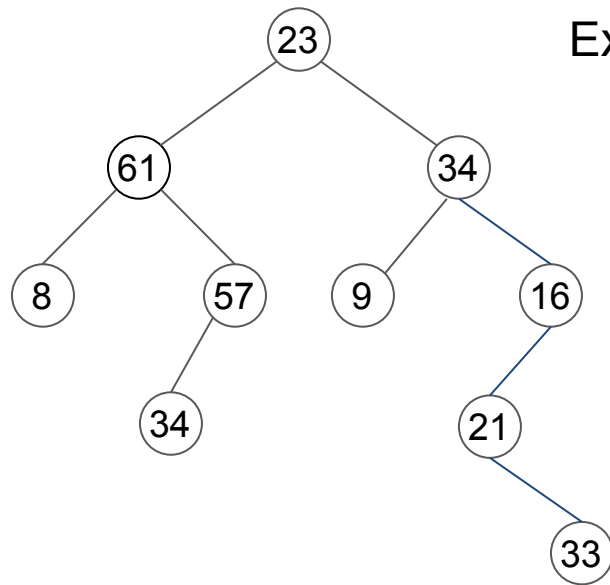


Árvore binária



Antecedentes de 16: (34,23)
Descendentes de 16: (21,33)

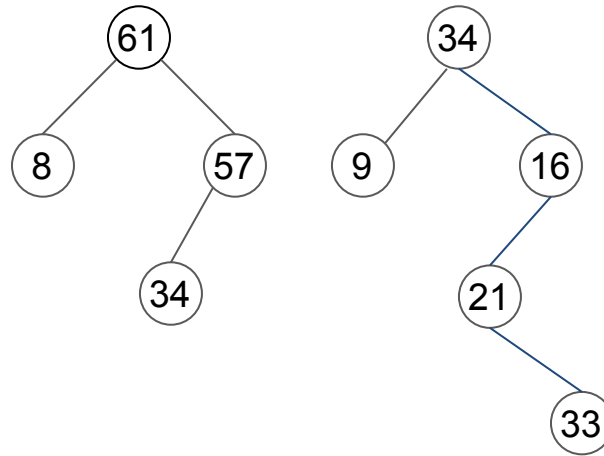
Árvore binária



Excluir a raiz:
Cria duas novas árvores

Árvore binária

Excluir a raiz:
Cria duas novas árvores



Referências

- Esta apresentação é baseada nas seguintes referências:
 - Capítulo 5 do livro Sedgewick, R., Algorithms in C, Addison-Wesley, 1998.
 - Apêndice B.5 do livro Cormen, T., Leiserson, C., Rivest, R., Stein, C., Introduction To Algorithms, MIT Press, 2001.

Árvore Binária de Busca (ABB)

Binary Search Tree (BST)

Definição

- As árvores binárias são árvores que os nós possuem no máximo 2 filhos
- Devem possuir os atributos que representam o conteúdo do nó + dois ponteiros para os filhos:



- O campo **int value**, apenas ilustra um tipo de dado a ser armazenado

Definição

- A inserção deve seguir a forma das listas:
 - Aloca memória
 - Atribui o valor para os campos de armazenamento
 - **NULL**ifica os ponteiros que apontam para esquerda e direita
 - Encontra a melhor posição na árvore para o novo nó

```
newnode=(Node *) malloc(sizeof(Node));  
newnode->value=value;  
newnode->left=NULL;  
newnode->right=NULL;
```



Definição

- Uma árvore binária de busca é uma árvore binária que restringe a forma que os nós são inseridos:
 - Binary Search Tree (BST)
- Para todo nó de uma árvore de busca binária:
 - O elemento à esquerda deve ser menor ou igual ao pai
 - O elemento à direita deve ser maior ou igual ao pai
 - A igualdade deve ser **escolhida** ou para **a direita** ou para **a esquerda**

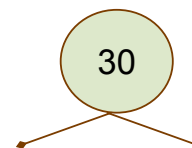
Exemplo

atual									
30	51	14	43	82	21	9	3	27	99
0	1	2	3	4	5	6	7	8	9

Exemplo

atual

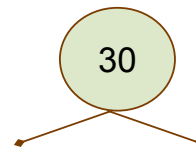
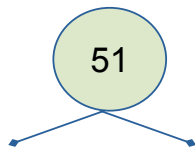
30	51	14	43	82	21	9	3	27	99
0	1	2	3	4	5	6	7	8	9



Raiz

Exemplo

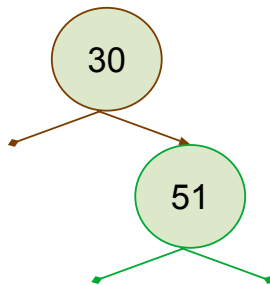
	atual								
30	51	14	43	82	21	9	3	27	99
0	1	2	3	4	5	6	7	8	9



Raiz

Exemplo

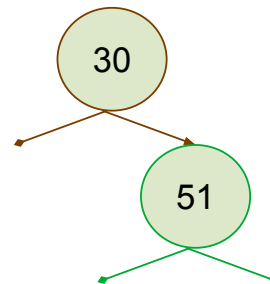
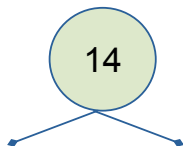
	atual								
30	51	14	43	82	21	9	3	27	99
0	1	2	3	4	5	6	7	8	9



Raiz

Exemplo

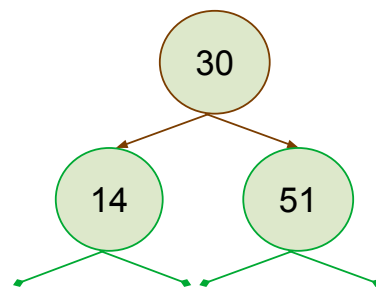
atual									
30	51	14	43	82	21	9	3	27	99
0	1	2	3	4	5	6	7	8	9



Raiz

Exemplo

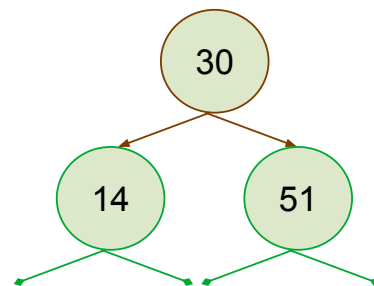
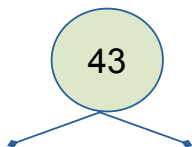
atual									
30	51	14	43	82	21	9	3	27	99
0	1	2	3	4	5	6	7	8	9



Raiz

Exemplo

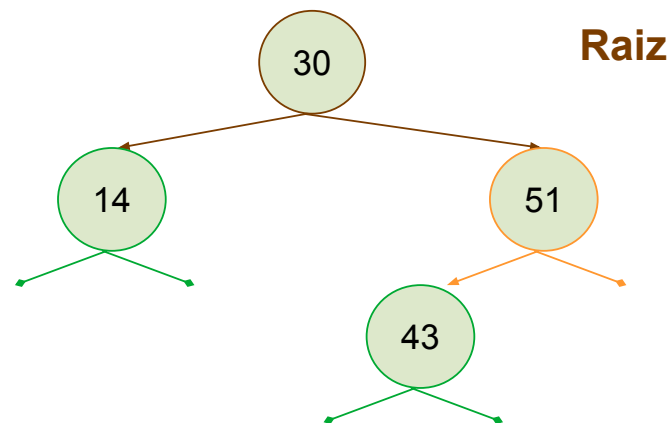
atual									
30	51	14	43	82	21	9	3	27	99
0	1	2	3	4	5	6	7	8	9



Raiz

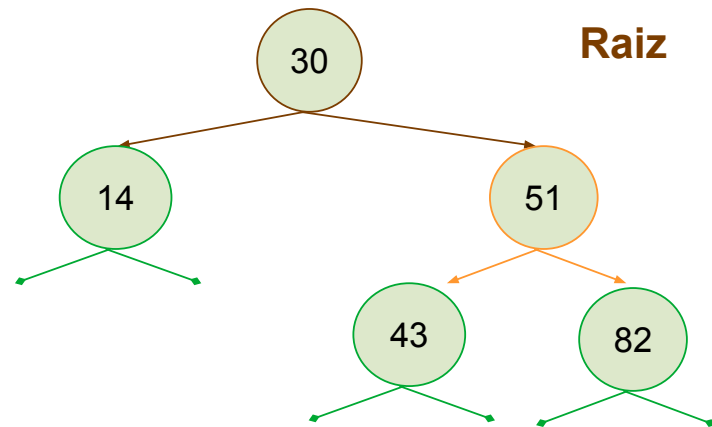
Exemplo

atual									
30	51	14	43	82	21	9	3	27	99
0	1	2	3	4	5	6	7	8	9



Exemplo

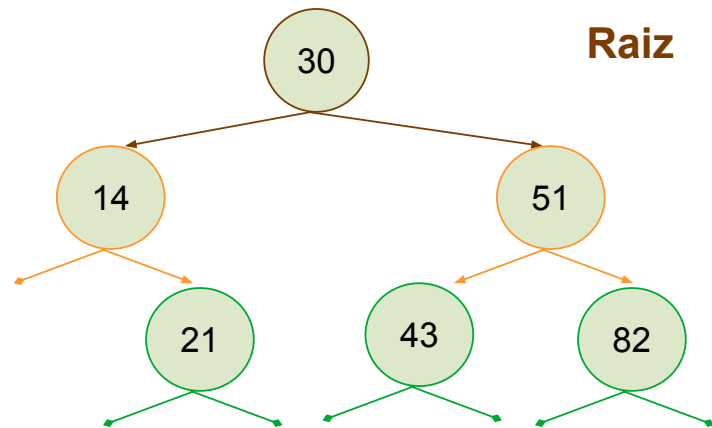
atual									
30	51	14	43	82	21	9	3	27	99
0	1	2	3	4	5	6	7	8	9



Exemplo

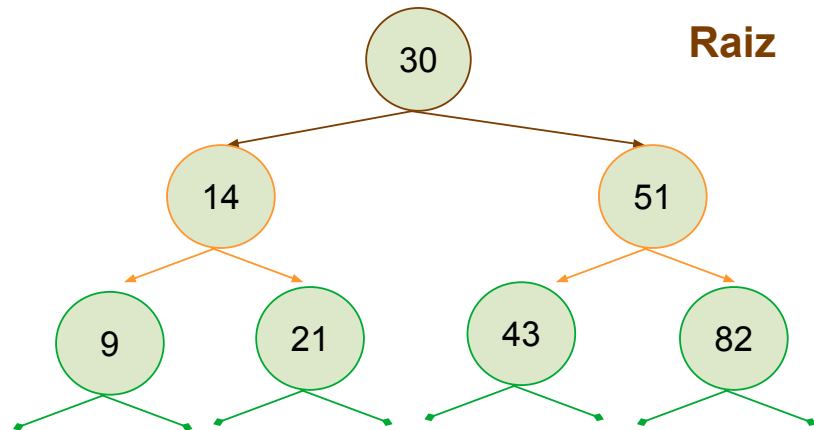
30	51	14	43	82	21	9	3	27	99
0	1	2	3	4	5	6	7	8	9

atual



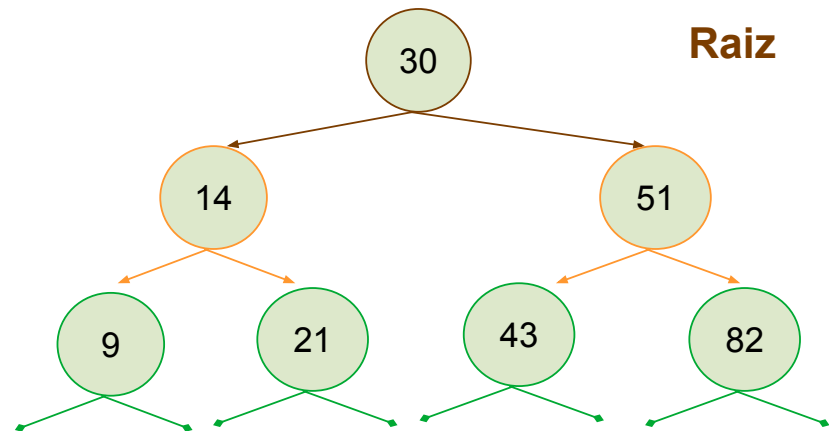
Exemplo

30	51	14	43	82	21	9	3	27	99
0	1	2	3	4	5	6	7	8	9



Exemplo

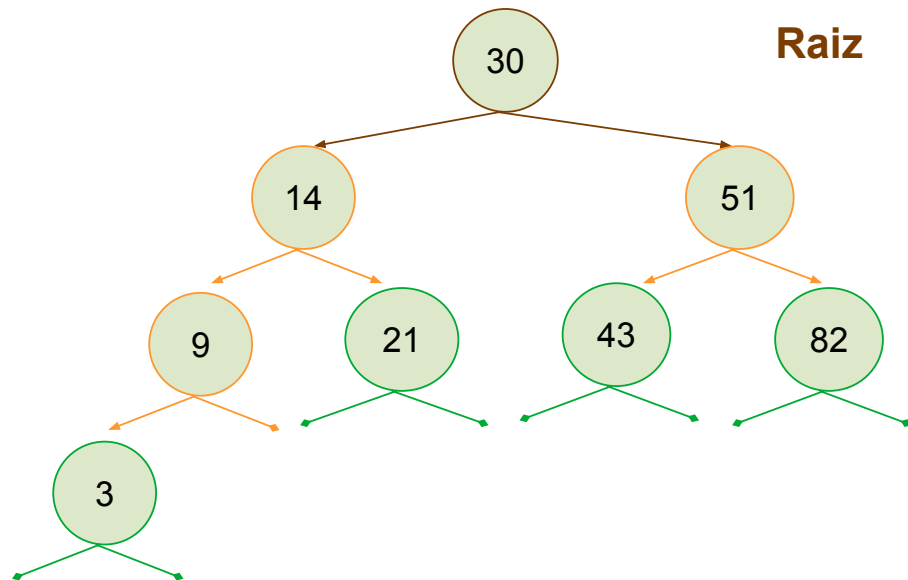
30	51	14	43	82	21	9	3	27	99
0	1	2	3	4	5	6	7	8	9



Exemplo

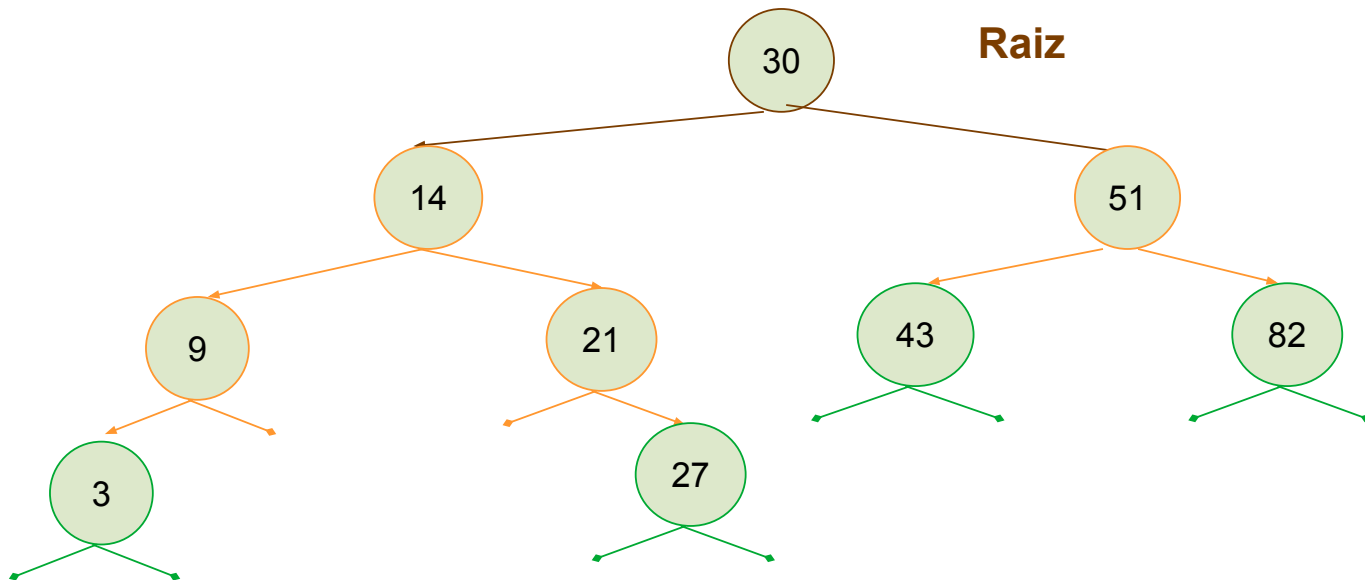
30	51	14	43	82	21	9	3	27	99
0	1	2	3	4	5	6	7	8	9

atual



Exemplo

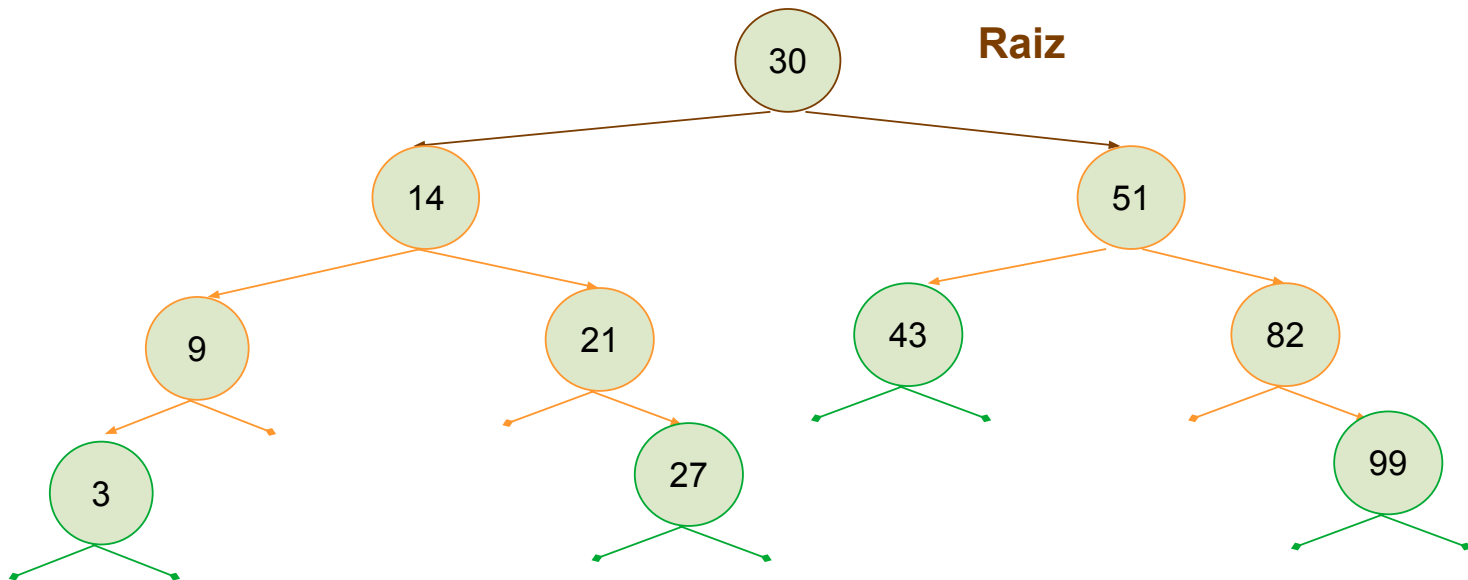
30	51	14	43	82	21	9	3	^{atual} 27	99
0	1	2	3	4	5	6	7	8	9



Exemplo

30	51	14	43	82	21	9	3	27	99
0	1	2	3	4	5	6	7	8	9

atual



Algoritmo

```
vnode *insertNodeR (vnode *root, vnode *n) //
recursive
{
    if (root==NULL)
        return n;
    if (n->info.age < root->info.age) // go to left
        root->left=insertNodeR(root->left,n);
    else {
        // no duplicates
        if (root->info.age==n->info.age) return root;
        root->right=insertNodeR(root->right,n);
    }
    return root;
}
```

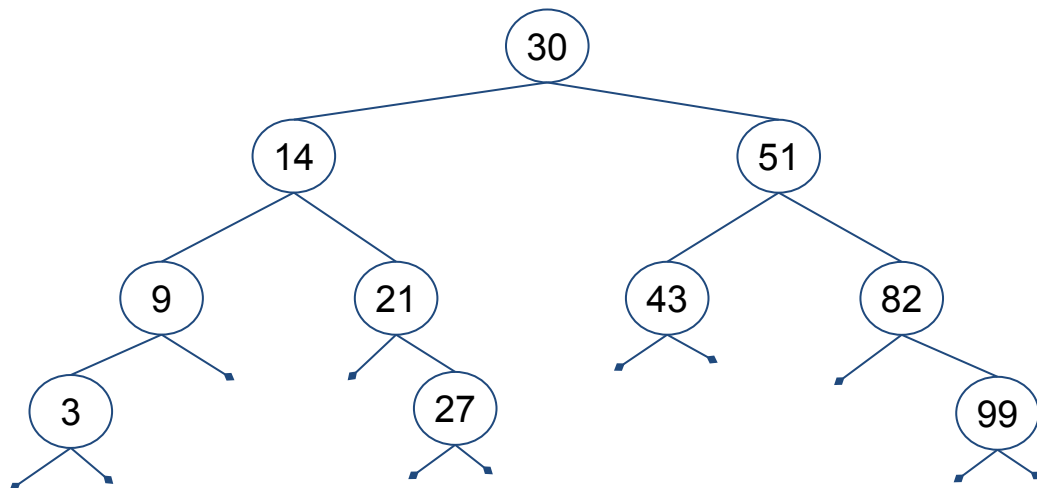
```
struct tinfo {
    int age;
};
struct tnode {
    struct tinfo info;
    struct tnode *left, *right;
};
typedef struct tnode vnode;
```

Caminhamento na BST

- Existem diversas aplicações onde devemos percorrer uma árvore de maneira sistemática, visitando todos os nós da árvore, um a um
- Existem três formas de percorrer uma árvore binária:
 - PreOrder (Pré-Ordem)
 - InOrder (Em-Ordem)
 - PostOrder (Pos-Ordem)

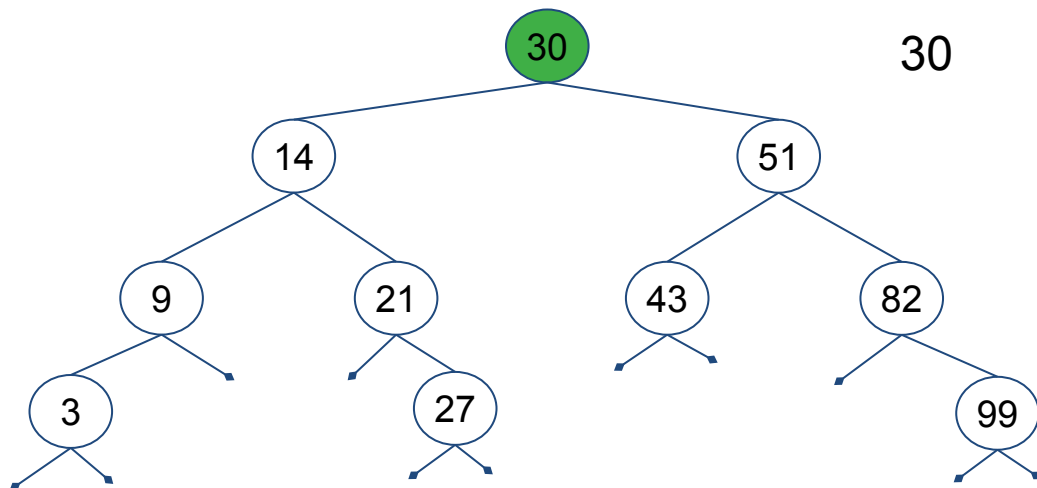
PreOrder

- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da esquerda**
- Percorre a **subárvore da direita**



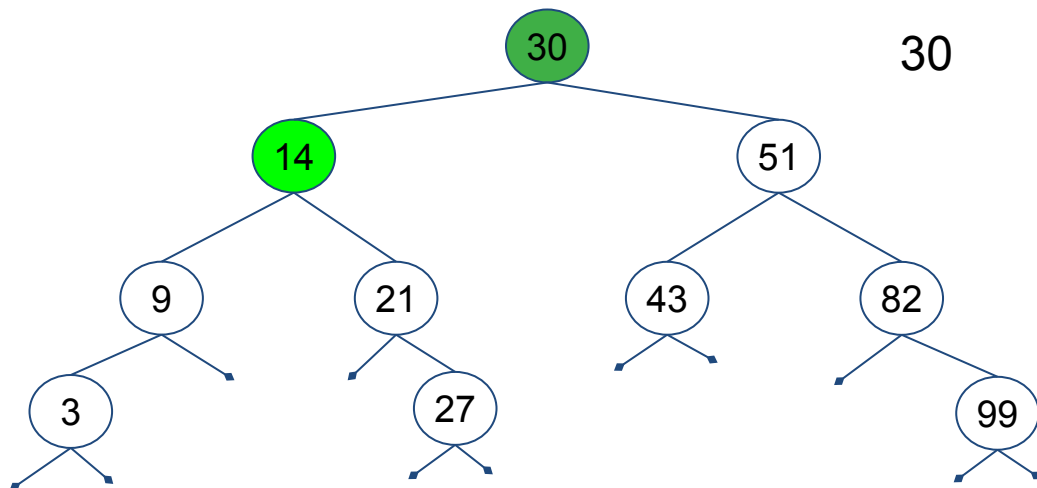
PreOrder

- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da esquerda**
- Percorre a **subárvore da direita**



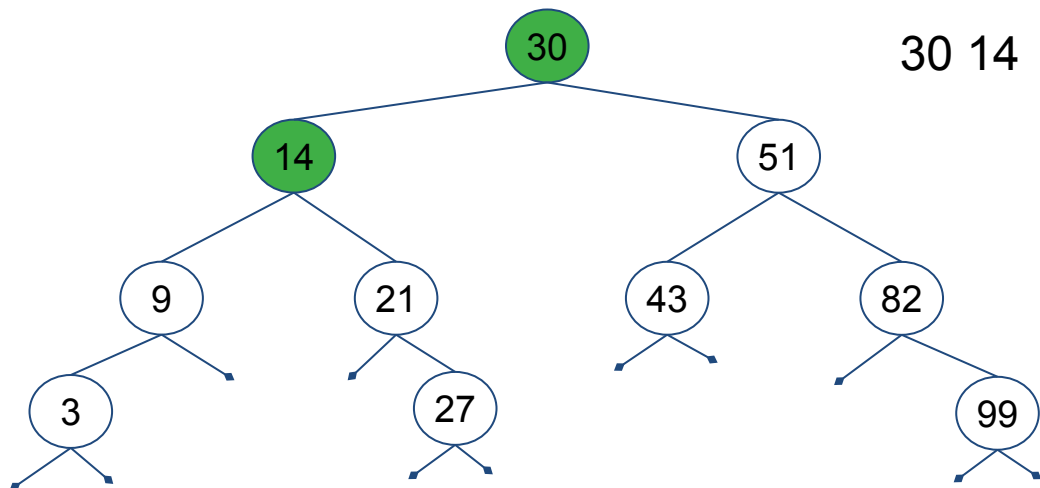
PreOrder

- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da esquerda**
- Percorre a **subárvore da direita**



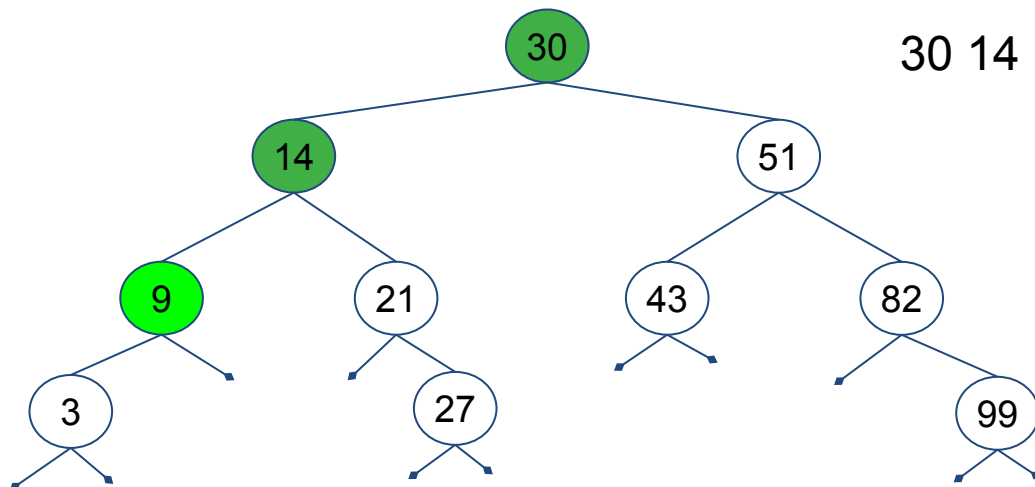
PreOrder

- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da esquerda**
- Percorre a **subárvore da direita**



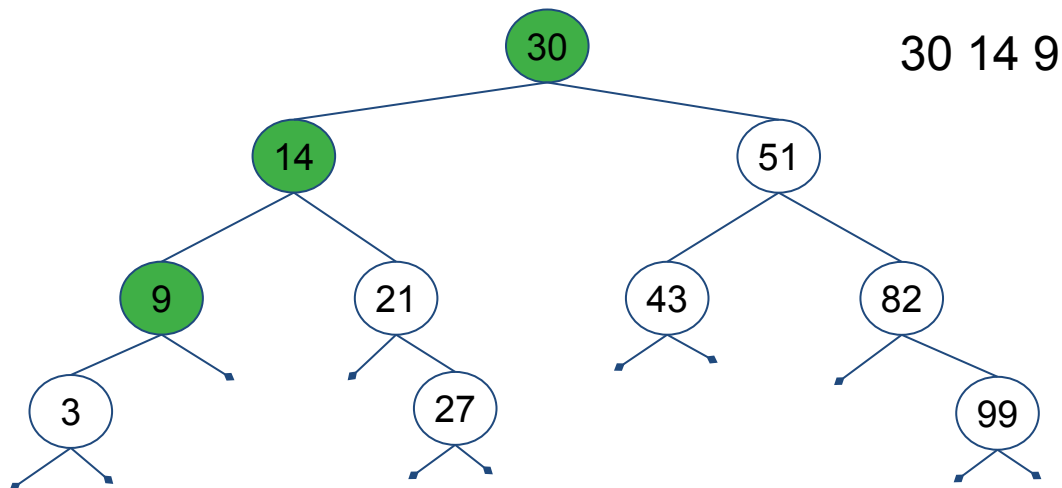
PreOrder

- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da esquerda**
- Percorre a **subárvore da direita**



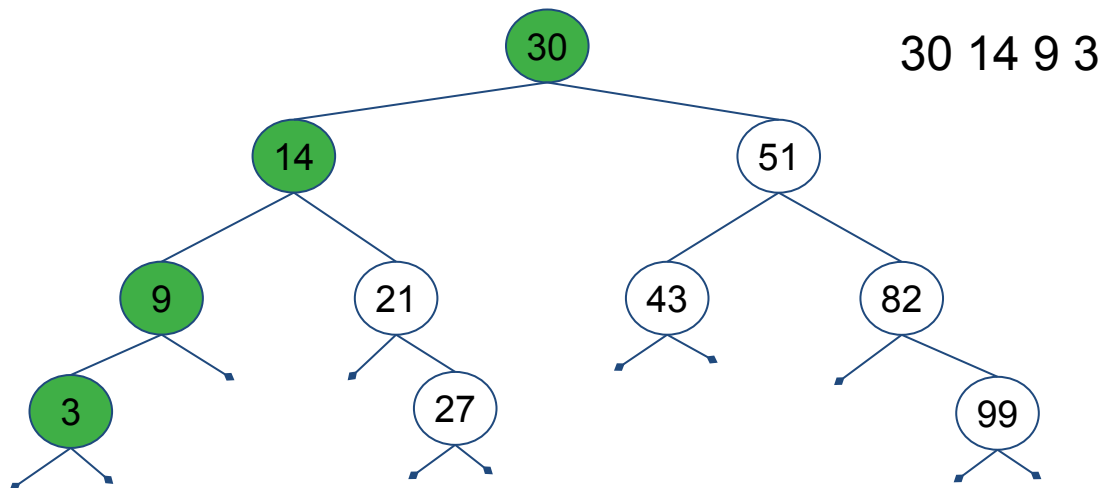
PreOrder

- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da esquerda**
- Percorre a **subárvore da direita**



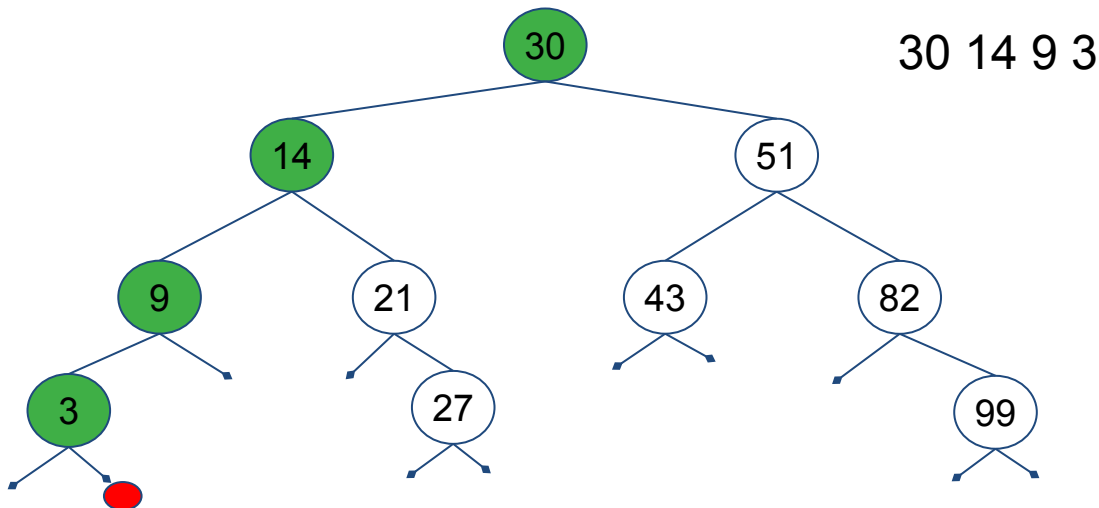
PreOrder

- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da esquerda**
- Percorre a **subárvore da direita**



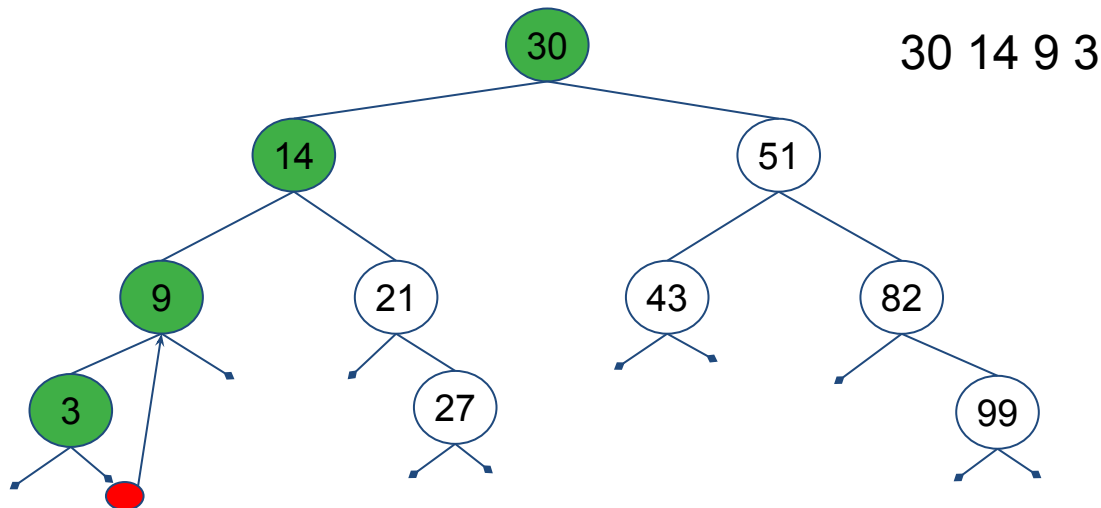
PreOrder

- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da esquerda**
- Percorre a **subárvore da direita**



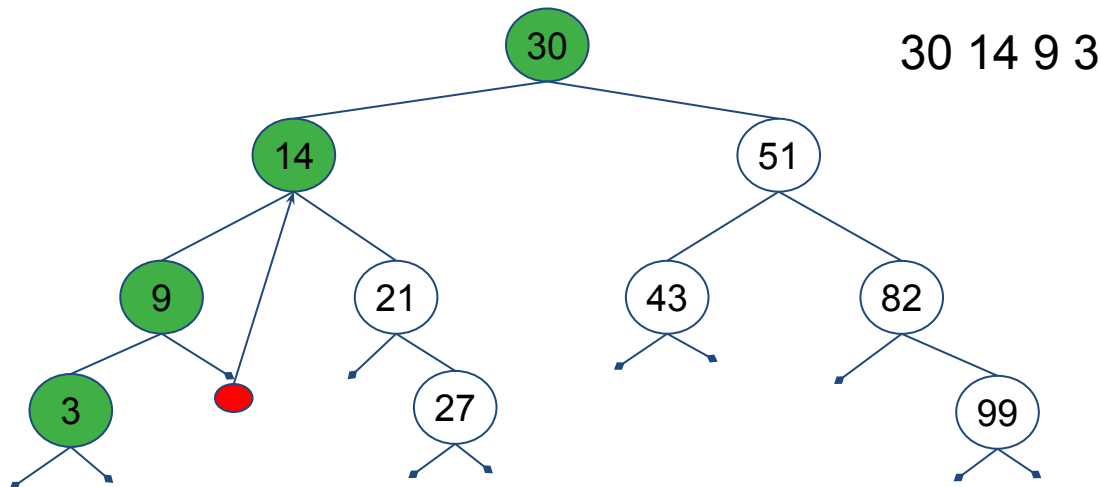
PreOrder

- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da esquerda**
- Percorre a **subárvore da direita**



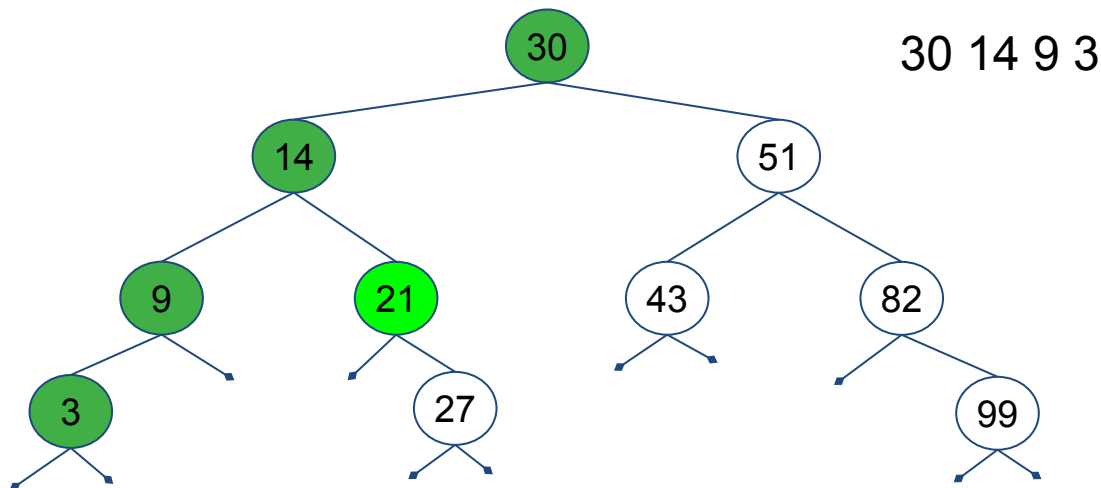
PreOrder

- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da esquerda**
- Percorre a **subárvore da direita**



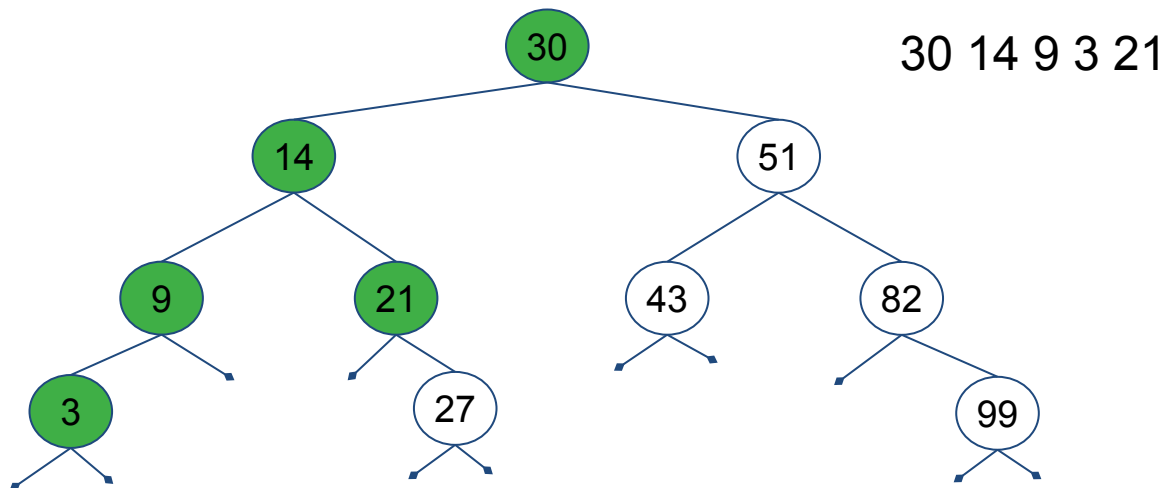
PreOrder

- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da esquerda**
- Percorre a **subárvore da direita**



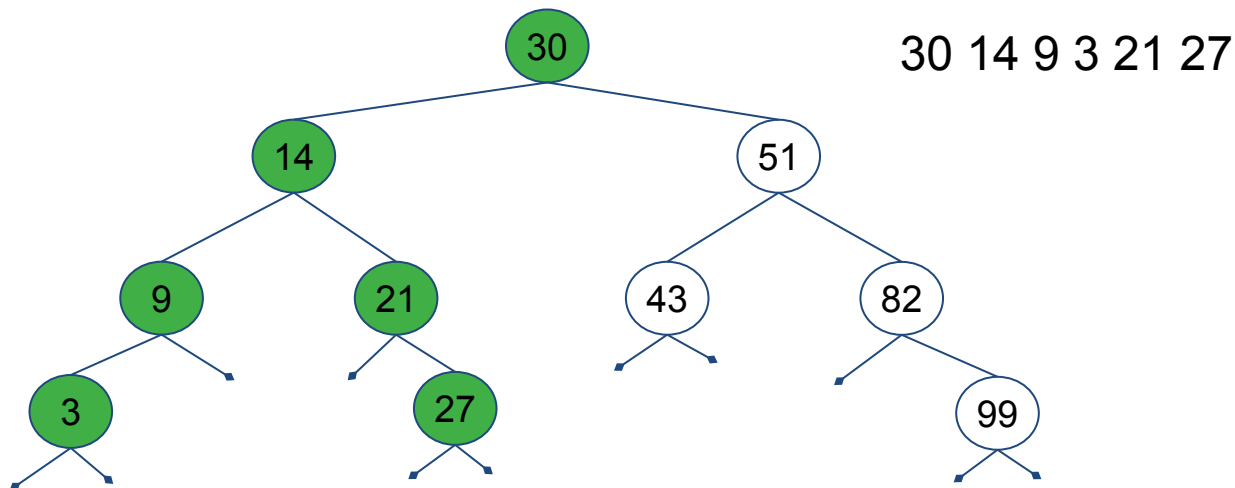
PreOrder

- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da esquerda**
- Percorre a **subárvore da direita**



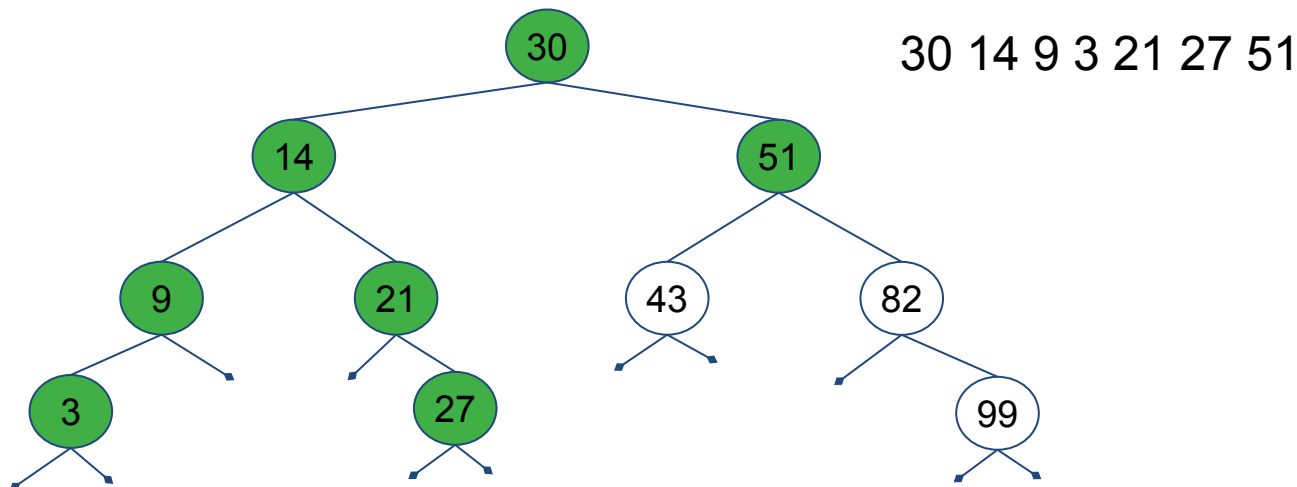
PreOrder

- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da esquerda**
- Percorre a **subárvore da direita**



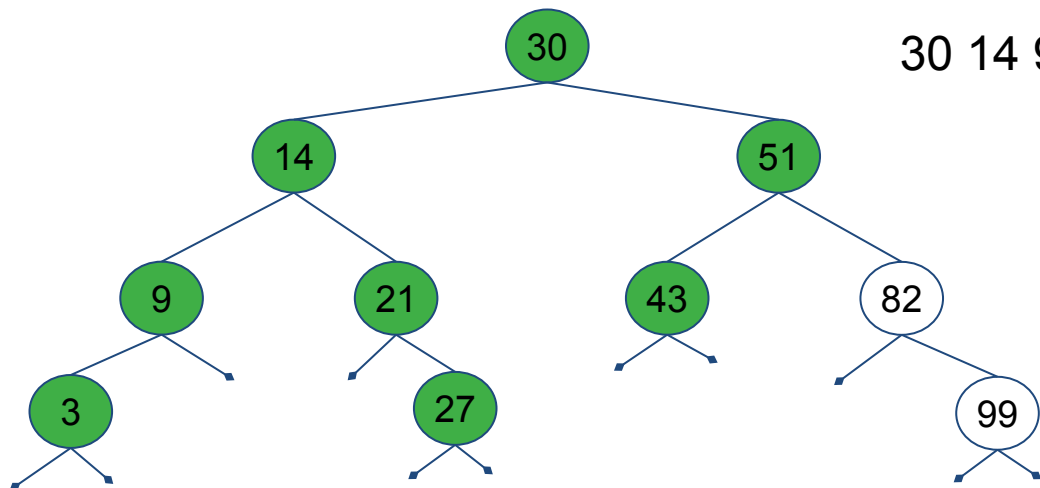
PreOrder

- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da esquerda**
- Percorre a **subárvore da direita**



PreOrder

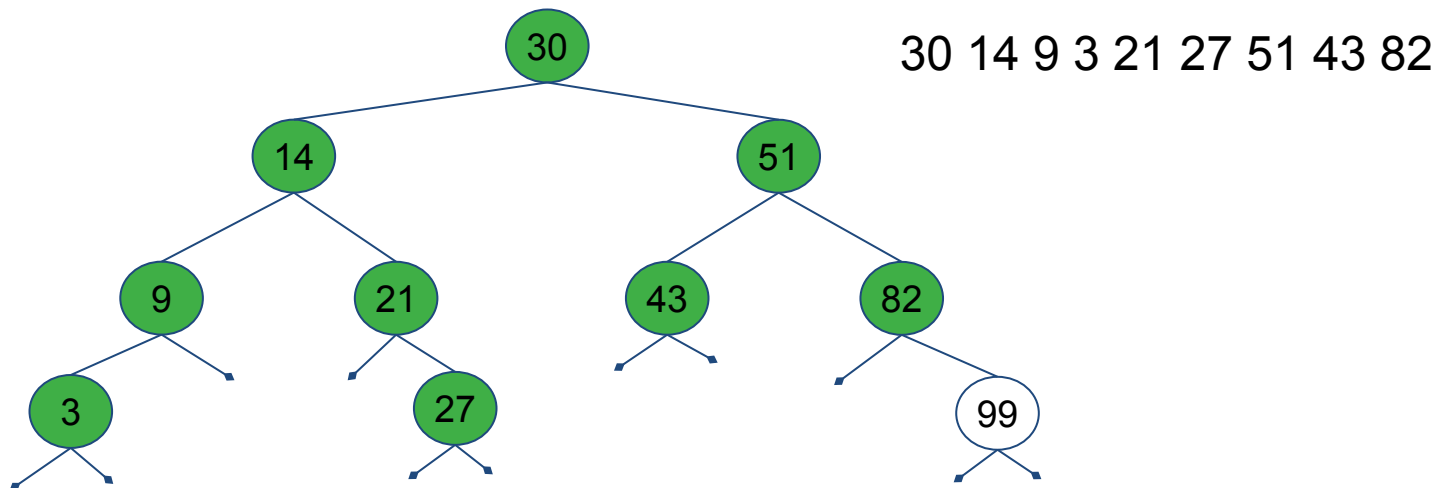
- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da esquerda**
- Percorre a **subárvore da direita**



30 14 9 3 21 27 51 43

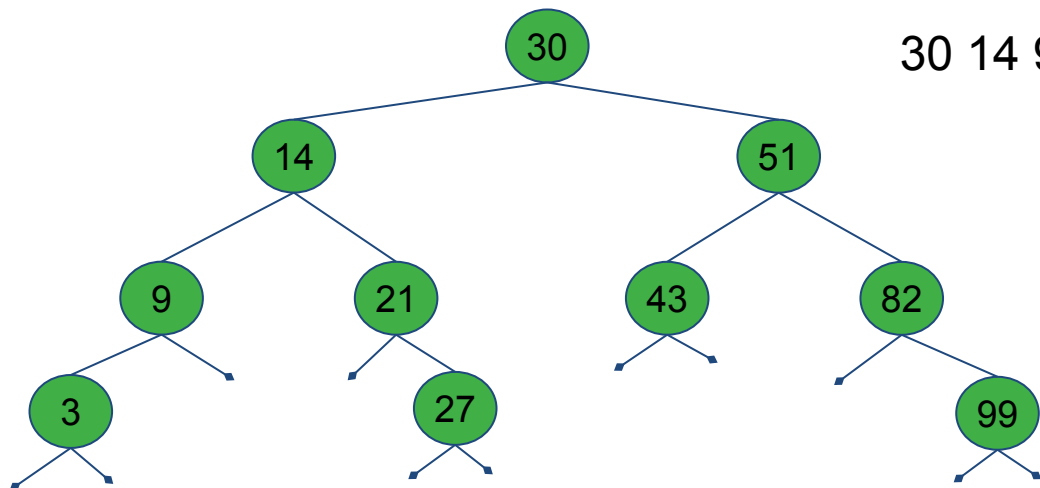
PreOrder

- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da esquerda**
- Percorre a **subárvore da direita**



PreOrder

- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da esquerda**
- Percorre a **subárvore da direita**



30 14 9 3 21 27 51 43 82 99

PreOrder

- Visita a raiz (acessa o conteúdo do nó)
- Percorre a subárvore da esquerda
- Percorre a subárvore da direita

```
void showPreOrder (vnode *root)
{
    if (root==NULL) return;
    printf("%d ",root->info.age);
    showPreOrder(root->left);
    showPreOrder(root->right);
}
```

PreOrder

- Visita a **raiz** (acessa o conteúdo do nó)
- **Percorre a subárvore da esquerda**
- Percorre a **subárvore da direita**

```
void showPreOrder (vnode *root)
{
    if (root==NULL) return;
    printf ("%d ", root->info.age);
    showPreOrder (root->left);
    showPreOrder (root->right);
}
```

PreOrder

- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da esquerda**
- **Percorre a subárvore da direita**

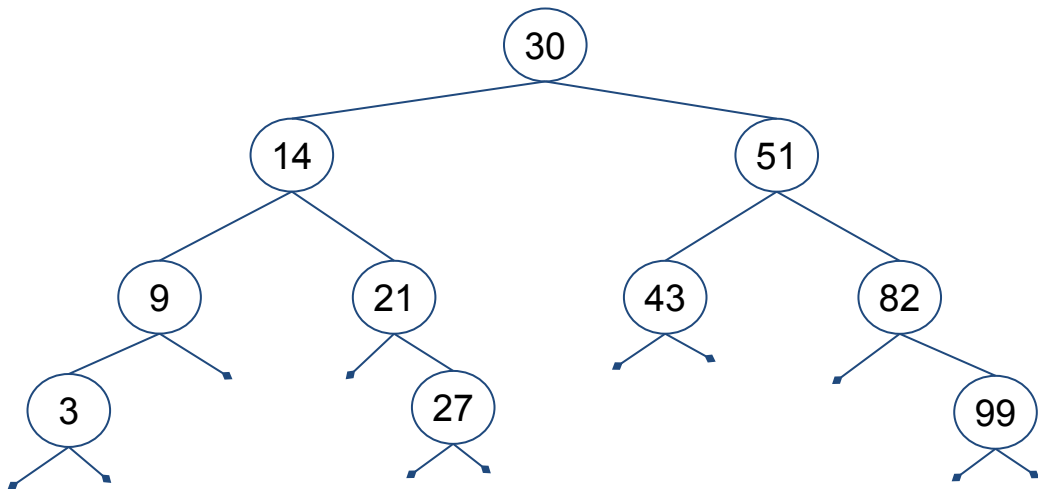
```
void showPreOrder (vnode *root)
{
    if (root==NULL) return;
    printf ("%d ", root->info.age);
    showPreOrder (root->left);
    showPreOrder (root->right);
}
```


PreOrder

- Clonar a árvore original
- Serializar a árvore
- Gera notação préfixa (notação polonesa)
 - $a + b \rightarrow + a b$
 - $(a+b) \times c \rightarrow +a b \times c$
- Navegação em folders (sistema operacional)

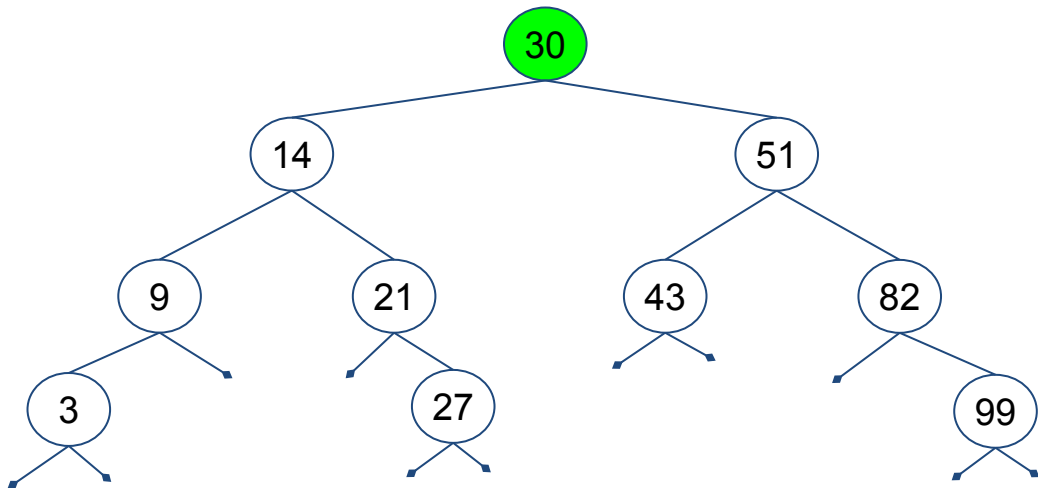
InOrder

- Percorre a **subárvore da esquerda**
- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da direita**



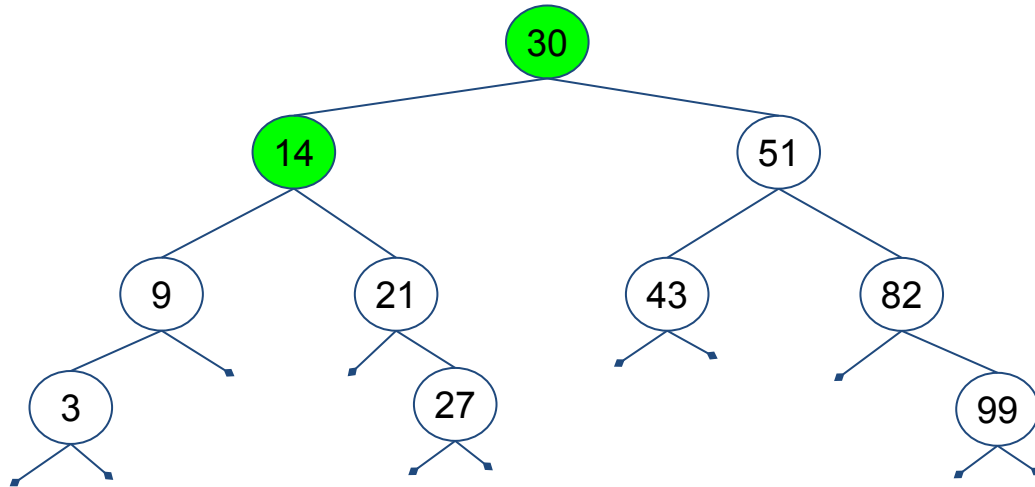
InOrder

- Percorre a **subárvore da esquerda**
- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da direita**



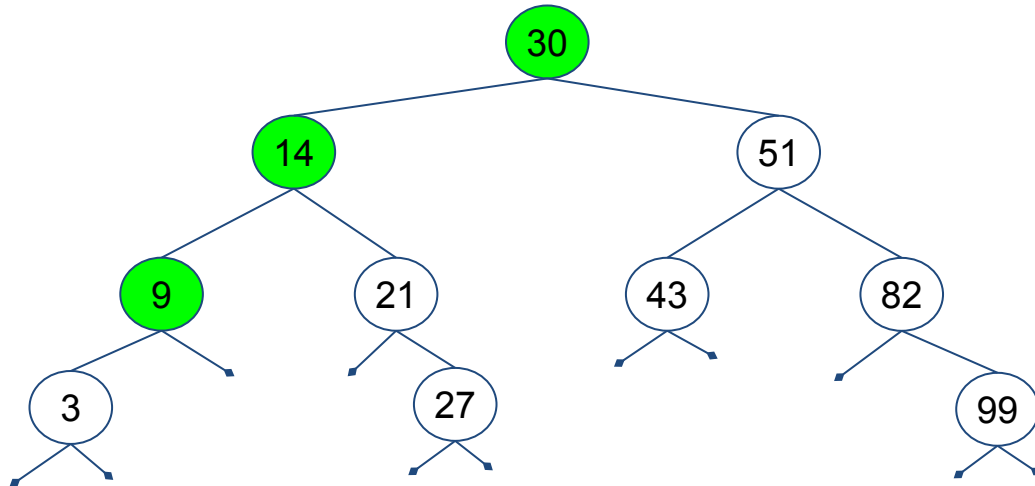
InOrder

- Percorre a **subárvore da esquerda**
- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da direita**



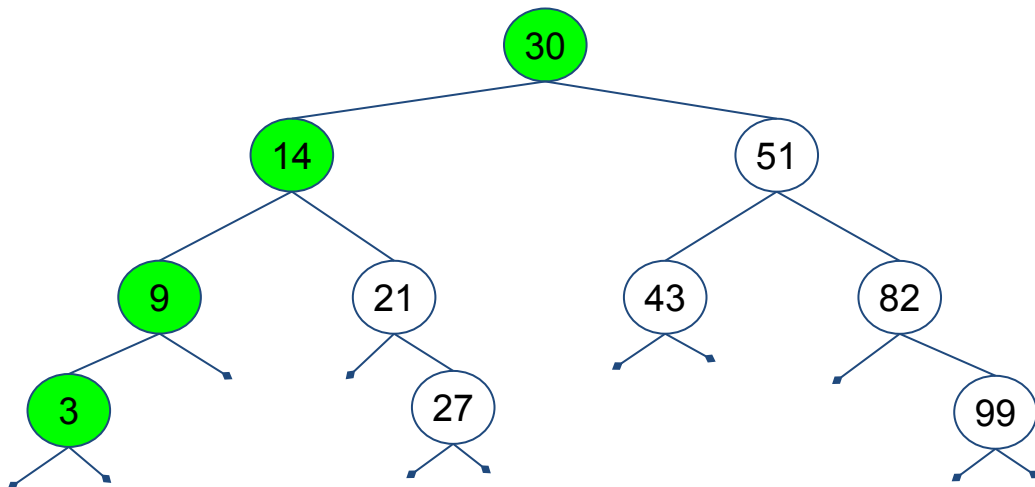
InOrder

- Percorre a **subárvore da esquerda**
- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da direita**



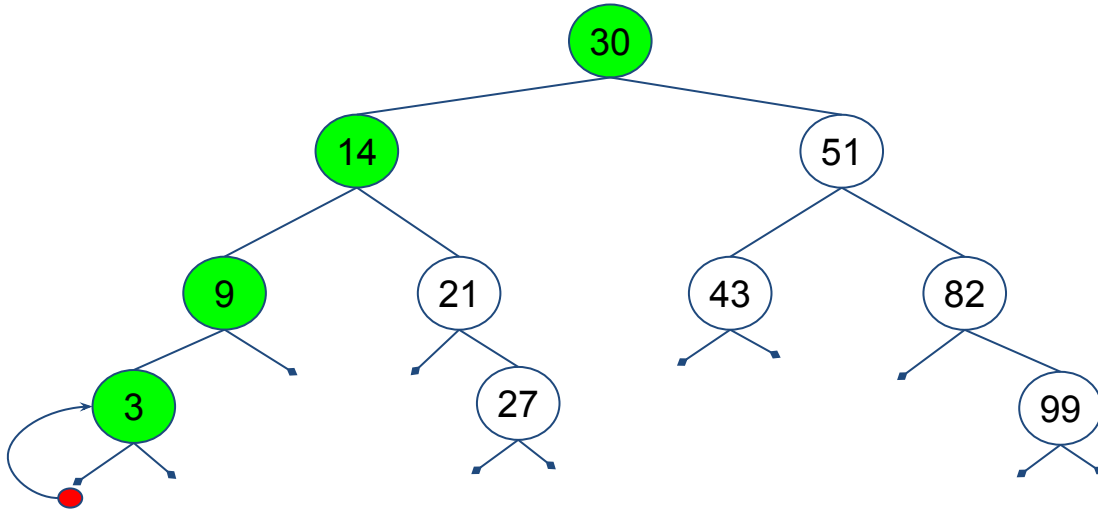
InOrder

- Percorre a **subárvore da esquerda**
- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da direita**



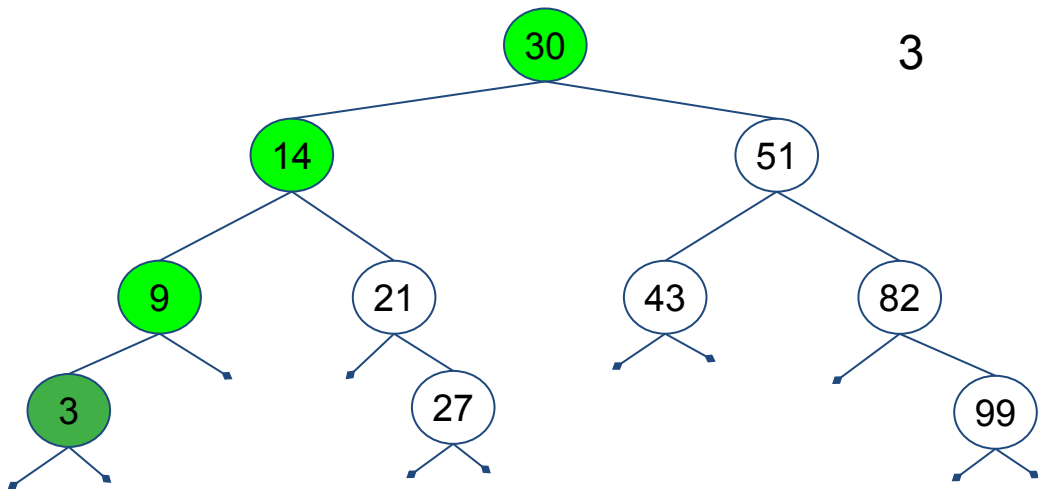
InOrder

- Percorre a **subárvore da esquerda**
- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da direita**



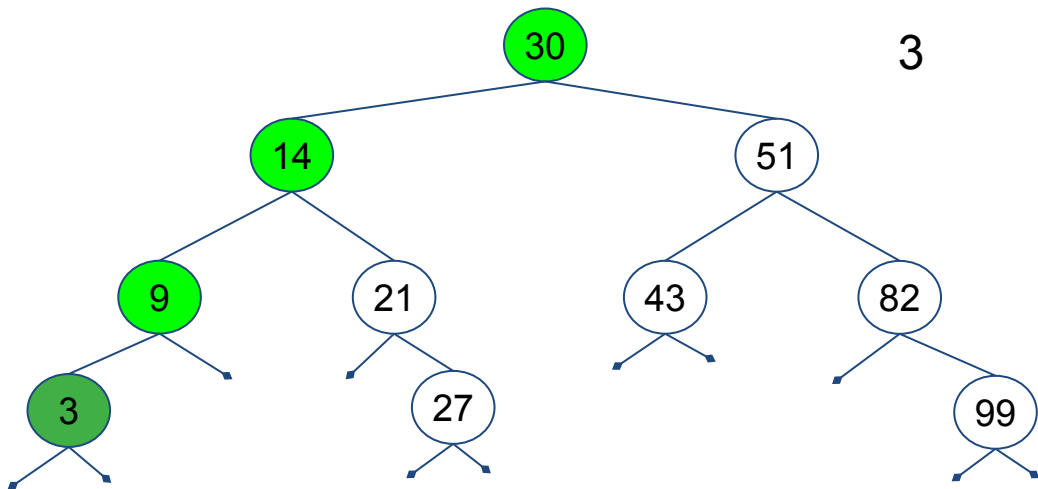
InOrder

- Percorre a subárvore da esquerda
- Visita a raiz (acessa o conteúdo do nó)
- Percorre a subárvore da direita



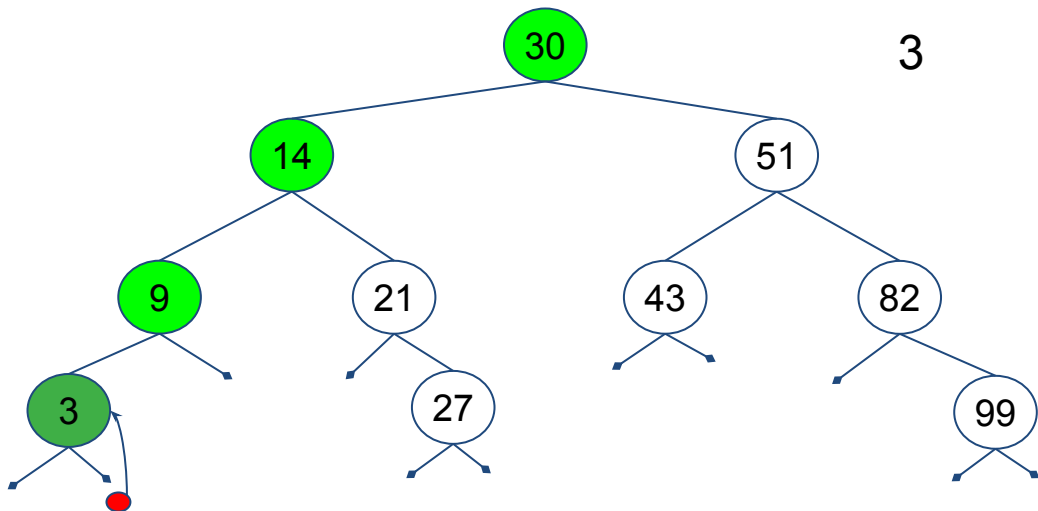
InOrder

- Percorre a subárvore da esquerda
- Visita a raiz (acessa o conteúdo do nó)
- Percorre a subárvore da direita



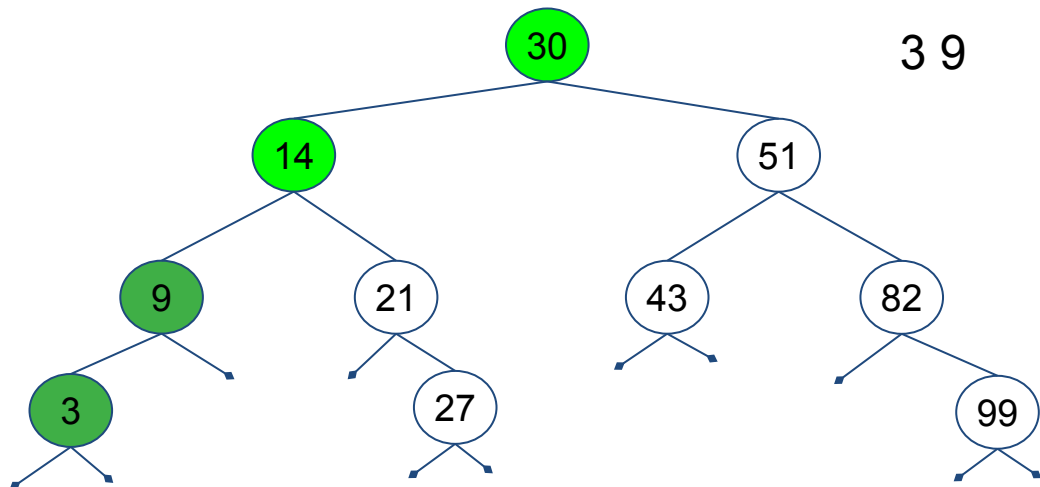
InOrder

- Percorre a **subárvore da esquerda**
- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da direita**



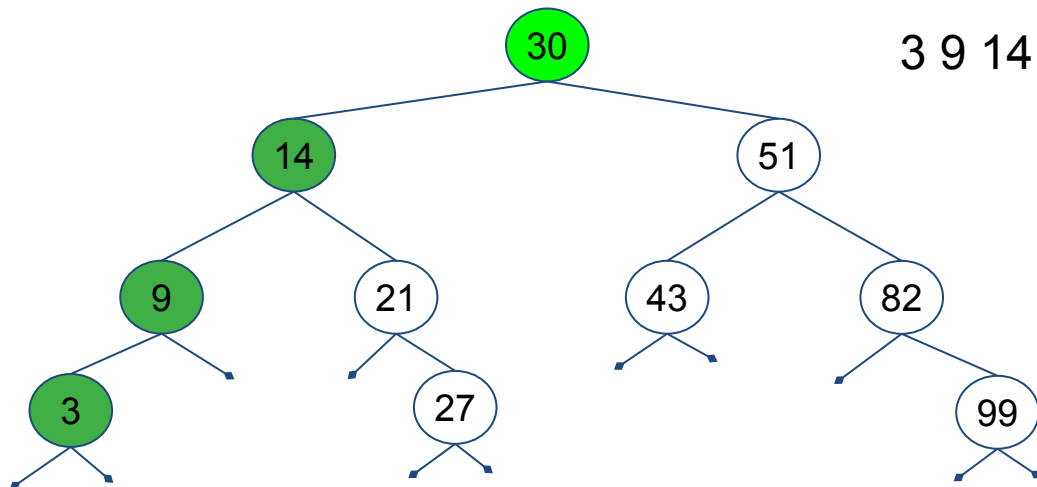
InOrder

- Percorre a **subárvore da esquerda**
- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da direita**



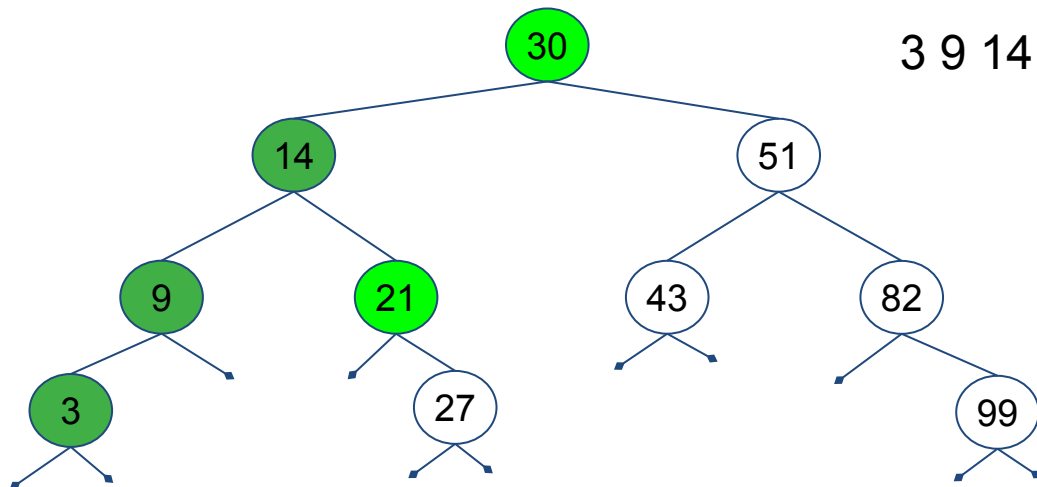
InOrder

- Percorre a **subárvore da esquerda**
- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da direita**



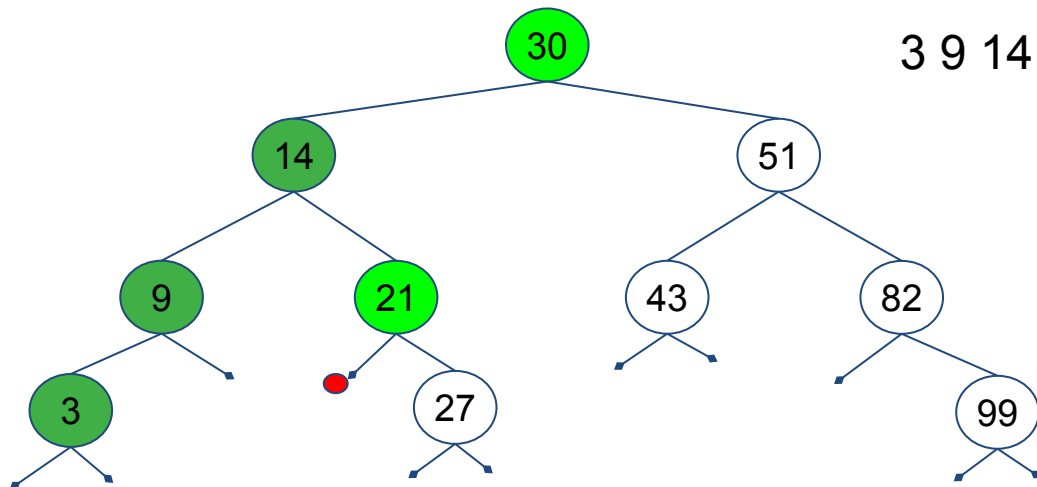
InOrder

- Percorre a **subárvore da esquerda**
- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da direita**



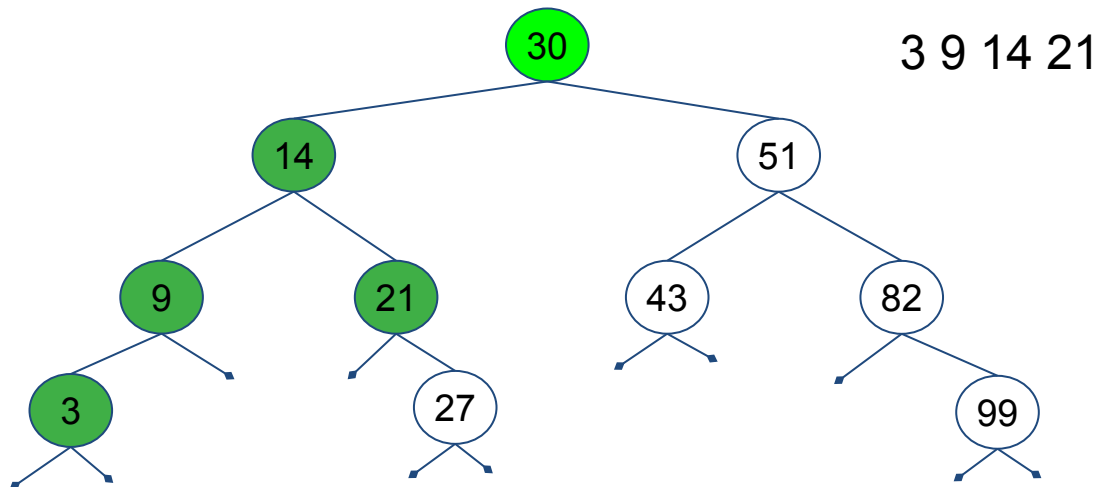
InOrder

- Percorre a **subárvore da esquerda**
- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da direita**



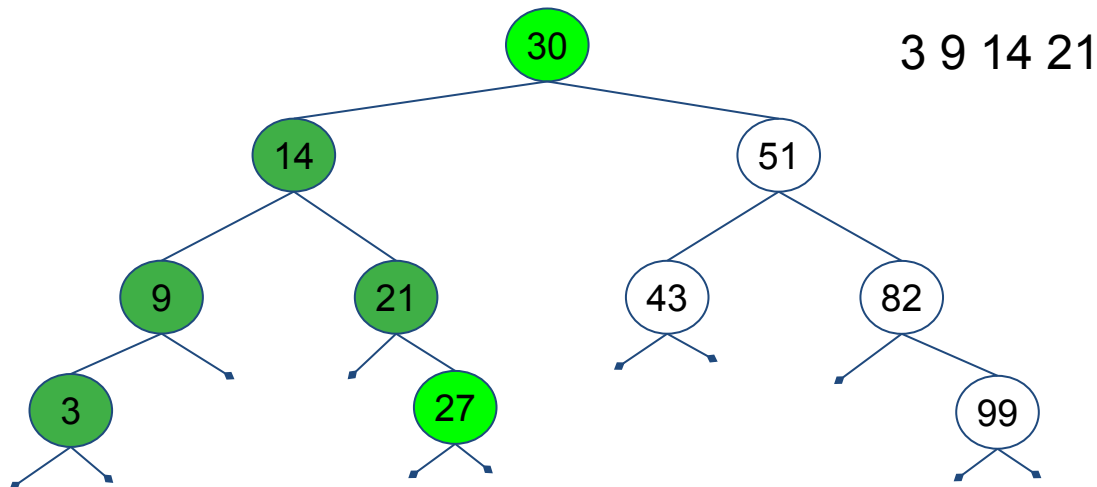
InOrder

- Percorre a **subárvore da esquerda**
- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da direita**



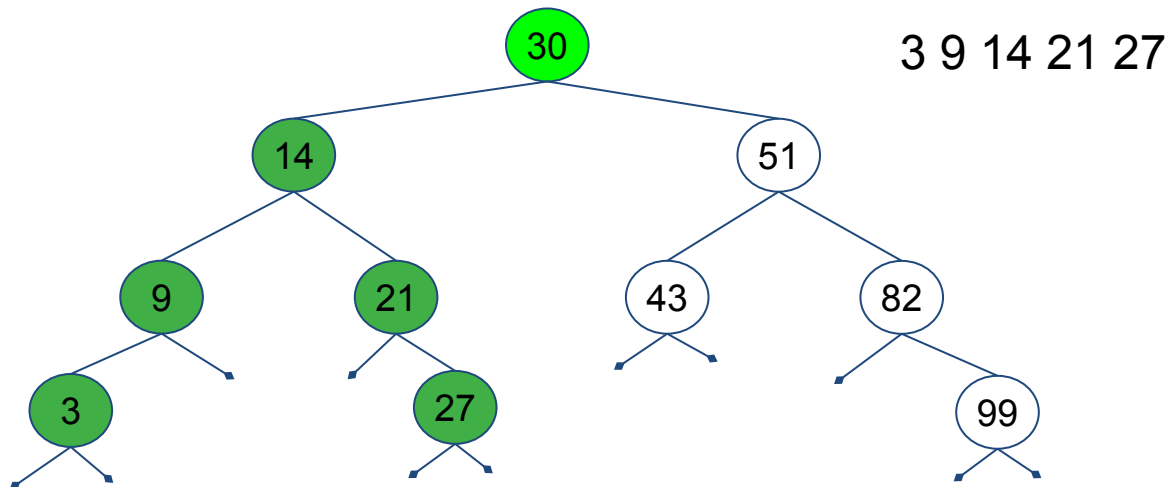
InOrder

- Percorre a **subárvore da esquerda**
- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da direita**



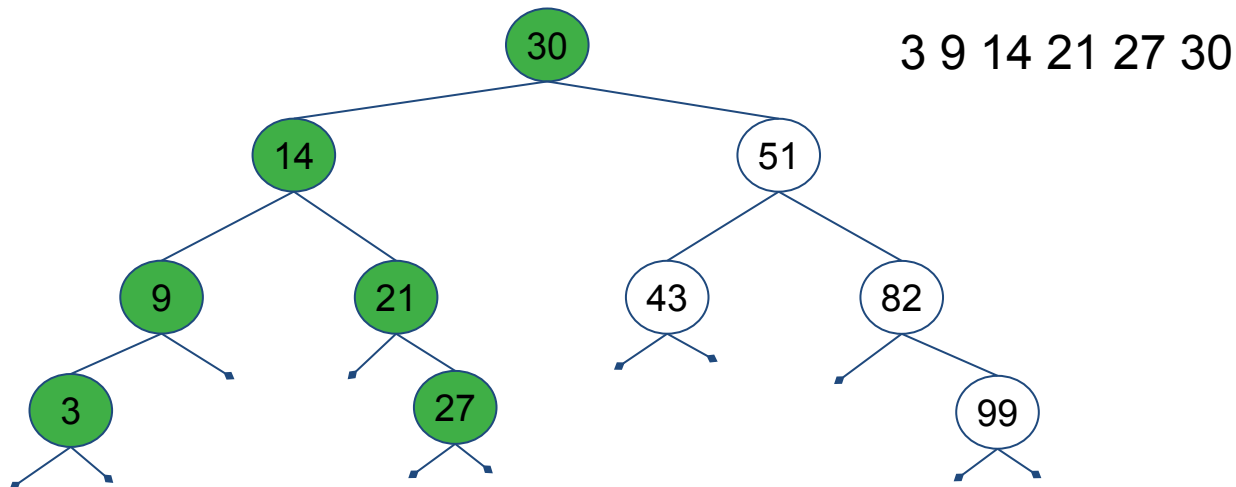
InOrder

- Percorre a **subárvore da esquerda**
- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da direita**



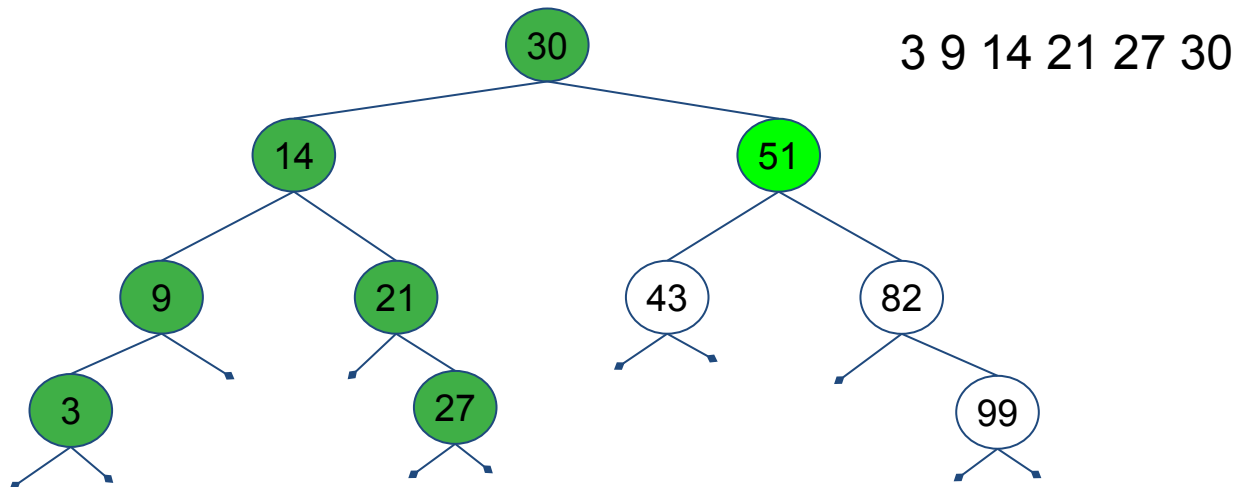
InOrder

- Percorre a **subárvore da esquerda**
- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da direita**



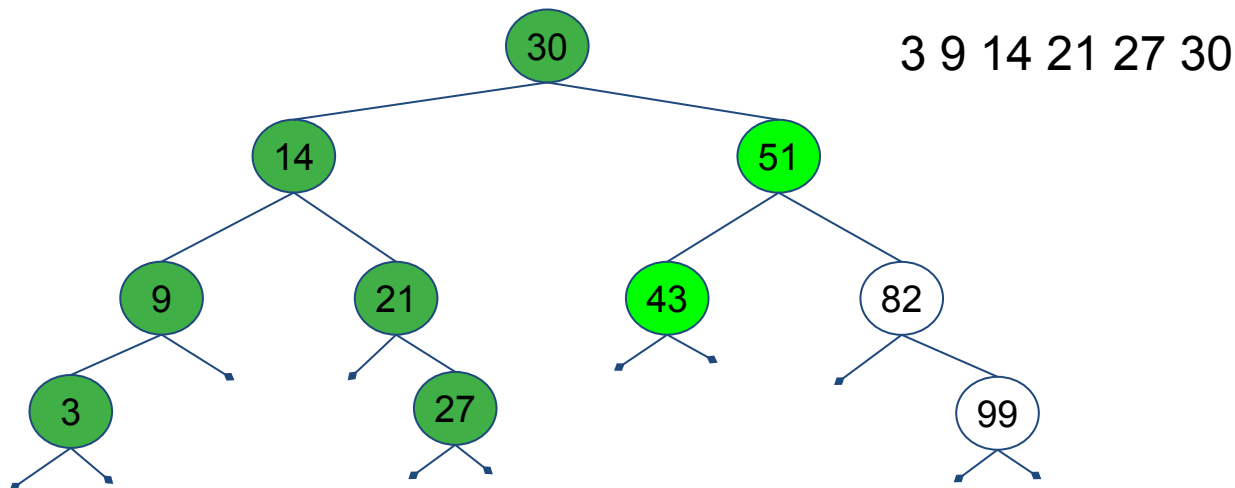
InOrder

- Percorre a **subárvore da esquerda**
- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da direita**



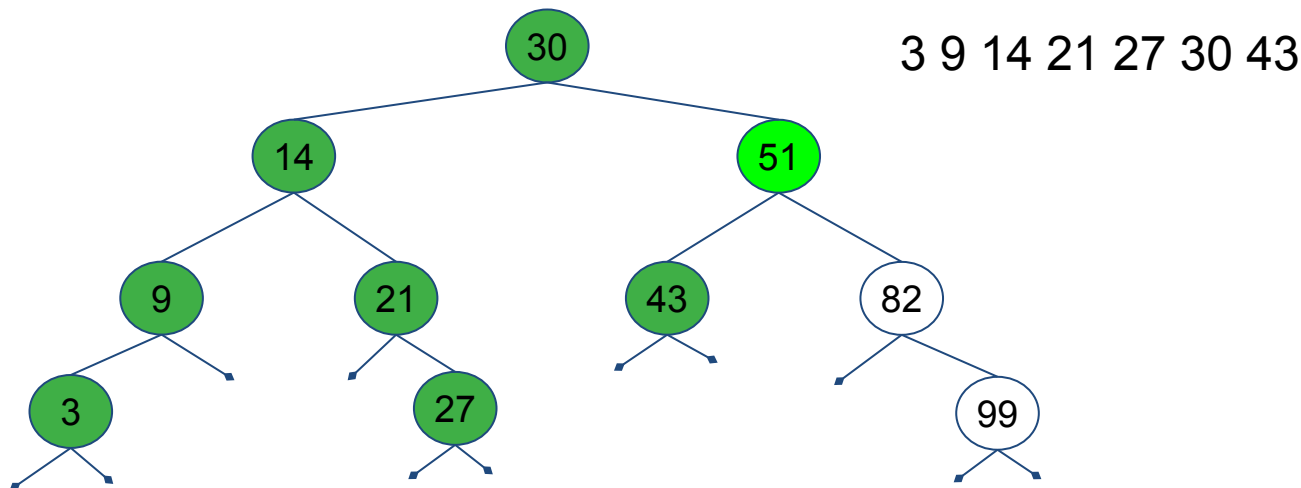
InOrder

- Percorre a **subárvore da esquerda**
- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da direita**



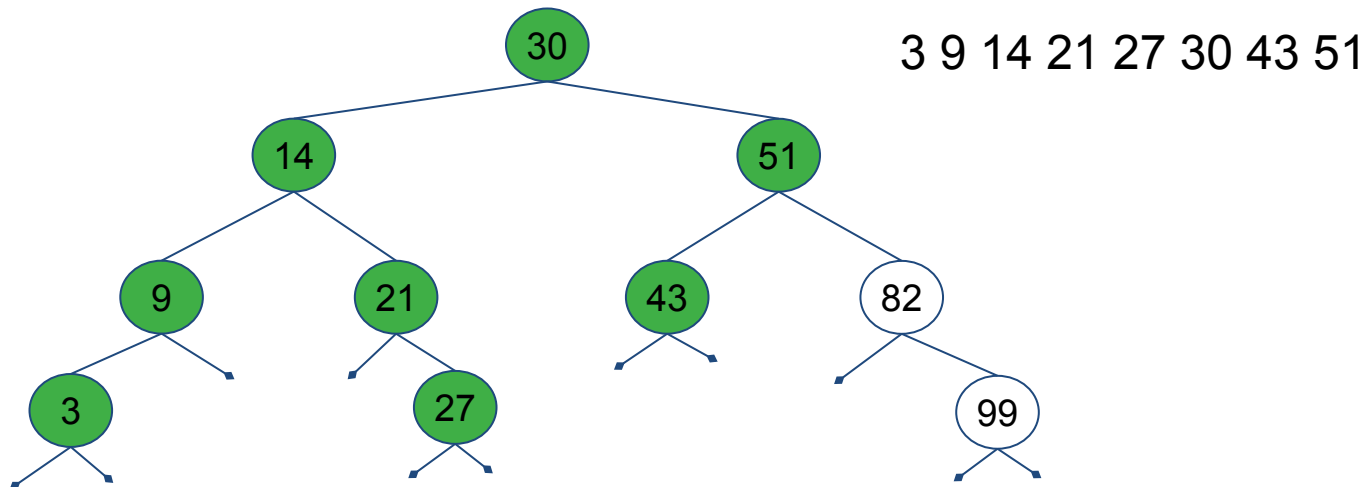
InOrder

- Percorre a **subárvore da esquerda**
- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da direita**



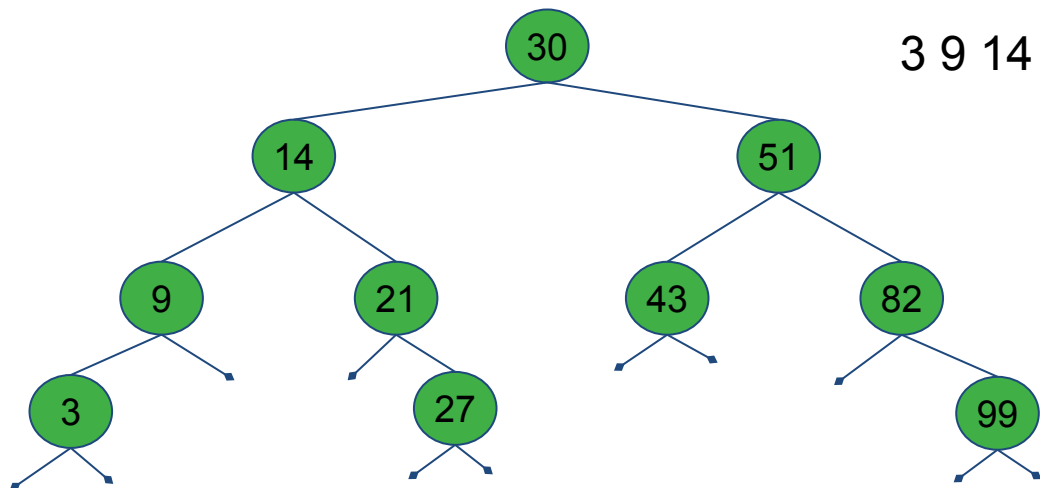
InOrder

- Percorre a **subárvore da esquerda**
- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da direita**



InOrder

- Percorre a **subárvore da esquerda**
- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da direita**



3 9 14 21 27 30 43 51 82 99

InOrder

- Percorre a **subárvore da esquerda**
- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da direita**

```
void showInOrder (vnode *root)
{
    /// Como seria??
}
```


InOrder

- Percorre a **subárvore da esquerda**
- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da direita**

```
void showInOrder (vnode *root)
{
    if (root==NULL) return;
    showInOrder(root->left);
    printf ("%d ", root->info.age);
    showInOrder(root->right);
}
```

InOrder

- Percorre a **subárvore da esquerda**
- Visita a **raiz** (acessa o conteúdo do nó)
- Percorre a **subárvore da direita**

```
void showInOrder (vnode *root)
{
    if (root==NULL) return;
    showInOrder(root->left);
    printf("%d ",root->info.age);
    showInOrder(root->right);
}
```

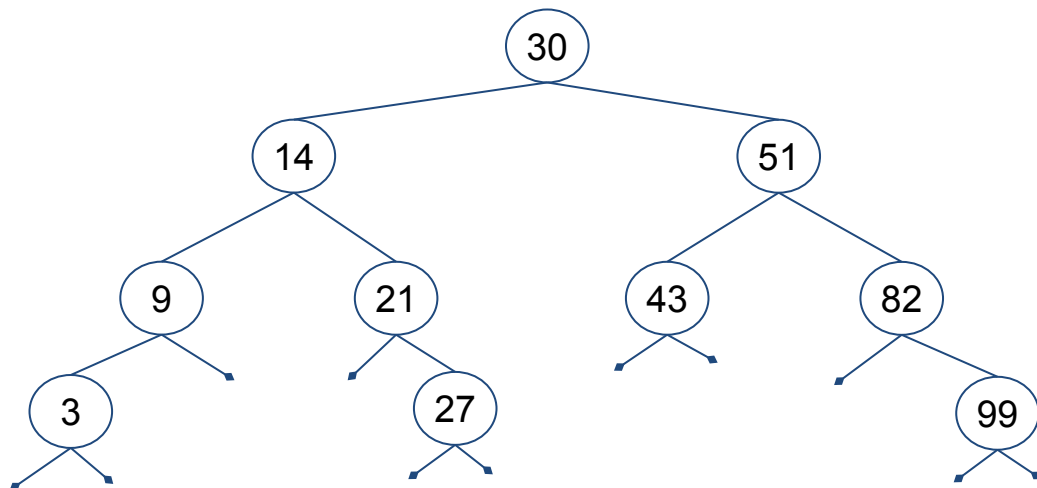
```
void showPreOrder (vnode *root)
{
    if (root==NULL) return;
    printf("%d ",root->info.age);
    showPreOrder(root->left);
    showPreOrder(root->right);
}
```

InOrder

- Retornar a lista ordenada
- Encontrar n-ésimo menor elemento
- Validar a árvore criada (resultado ordenado)

PostOrder

- Percorre a **subárvore da esquerda**
- Percorre a **subárvore da direita**
- Visita a **raiz** (acessa o conteúdo do nó)



PostOrder

- Excluir a árvore de forma segura
- Notação polonesa reversa
- Calcular tamanho da estrutura de diretório
 - Calcula dos filhos primeiro e soma os resultados com o pai

Árvore de Busca

- Árvores representam um ótima otimização de performance para o acesso a informações
 - A complexidade é da **ordem logarítmica** já que temos uma árvore
 - **$\log_2 n$** pois a árvore é binária (onde n é o número de nós)

Árvore de Busca

- Árvores representam um ótima otimização de performance para o acesso a informações
 - A complexidade é da **ordem logarítmica** já que temos uma árvore
 - **$\log_2 n$** pois a árvore é binária (onde n é o número de nós)

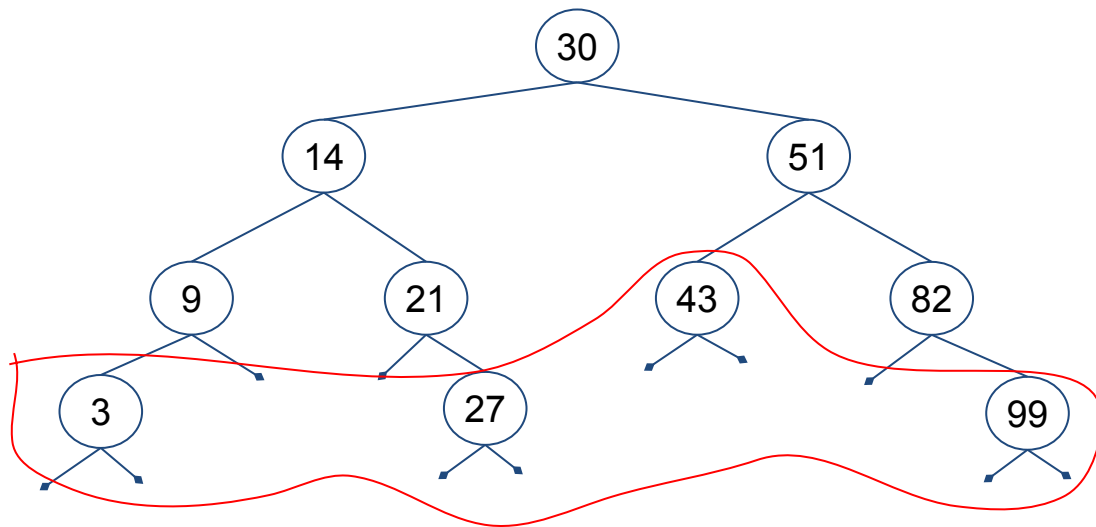


Árvore de Busca

- Árvores representam um ótima otimização de performance para o acesso a informações
 - A complexidade é da **ordem logarítmica** já que temos uma árvore
 - **$O(\log_2 n)$** pois a árvore é binária (onde n é o número de nós)
 - A construção da árvore pode gerar dois tipos
 - Balanceadas: as folhas estão quase na mesma altura
 - Desbalanceada (ou denegeradas): as alturas das folhas divergem muito. A busca pode se tornar **$O(n)$**

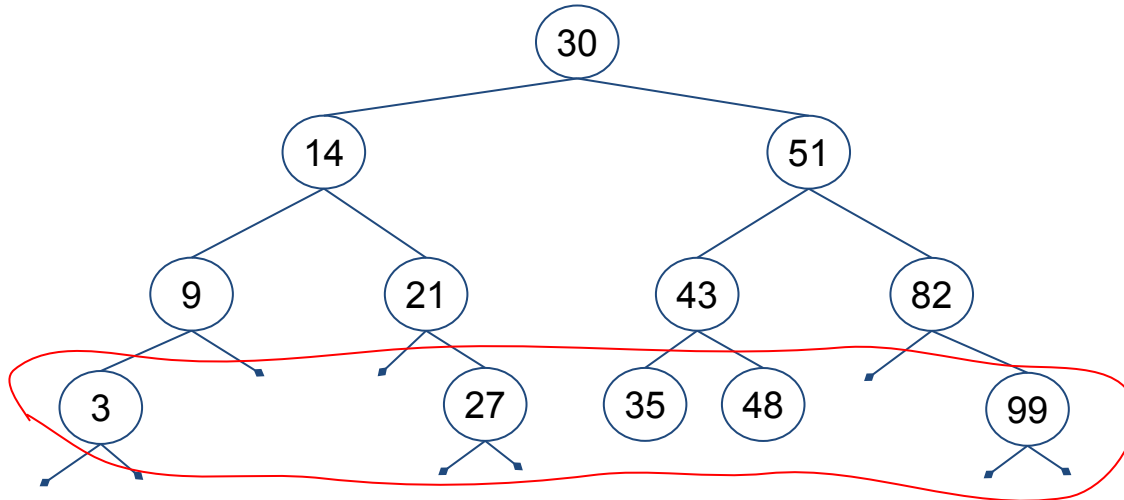
Árvore de Busca

- Balanceada



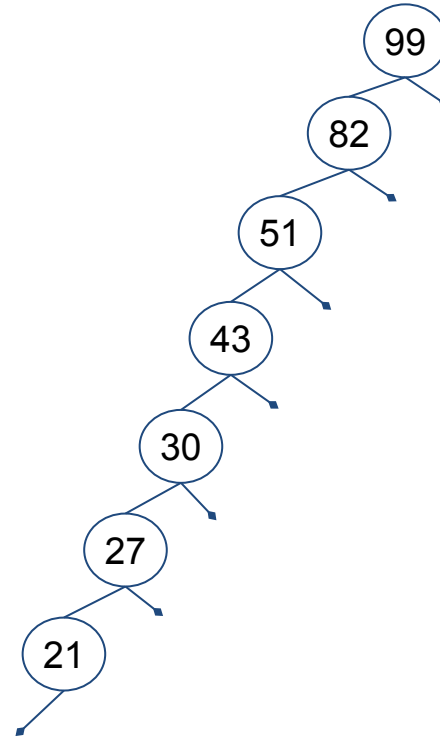
Árvore de Busca

- Balanceada (perfeita)



Árvore de Busca

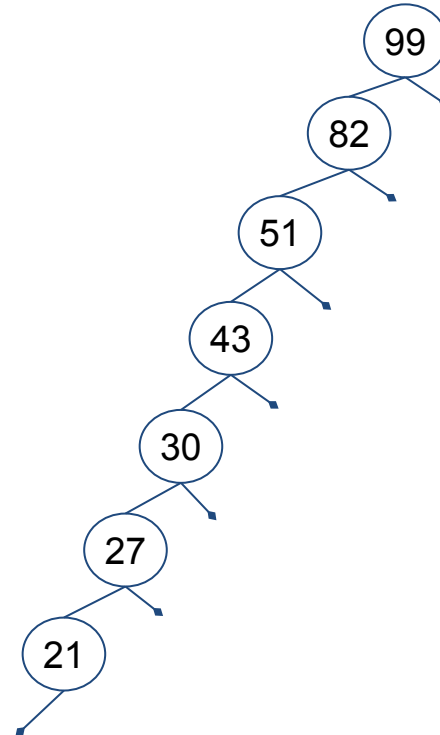
- Degenerada
 - 99 82 51 43 30 27 21



Árvore de Busca

- Degenerada
 - 99 82 51 43 30 27 21

Torna-se uma
lista simples



Árvore de Busca

- Como resolver?
 - Algoritmos de balanceamento
 - AVL
 - Red-Black Tree

Árvore de Busca

- Como resolver?
 - Algoritmos de balanceamento
 - AVL
 - Red-Black Tree

Árvore AVL

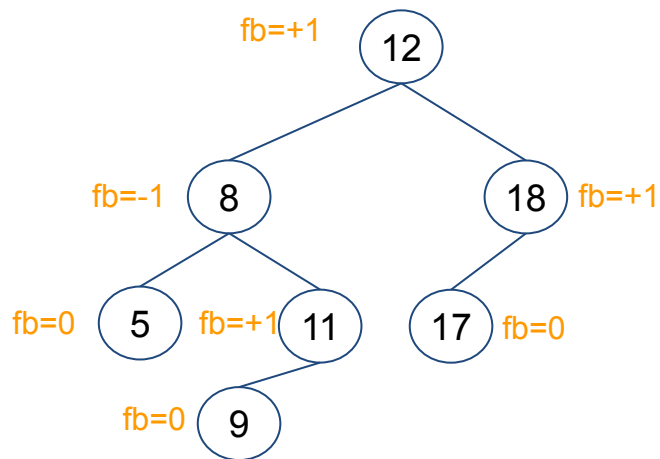
AVL Tree

AVL

- Uma árvore AVL é uma BST auto-balanceada
 - O nome vem de Adelson-Velsky and Landis (seus inventores)
 - Baseada em um fator de balanceamento
 - Balanced Factor (FB) = $Height(\text{Left Subtree}) - Height(\text{Right Subtree})$
 - A BST é considerada balanceada quando $FB = \{-1, 0, 1\}$
 - Viola quando $|FB| > 1$

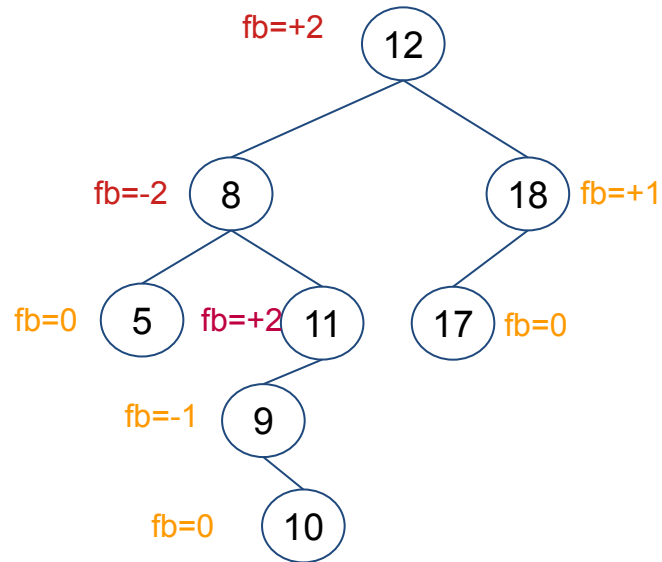
AVL

- Exemplo de AVL



AVL

- Exemplo de não AVL



AVL

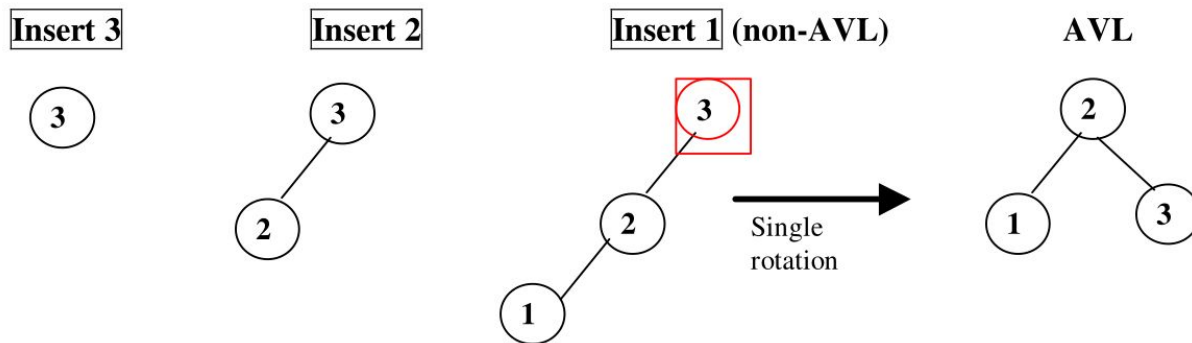
- O FB é calculado em tempo de inserção
- Quando viola a regra $|FB| > 1$
 - A árvore deve ser reestruturada para manter $|FB| \leq 1$

AVL

- Reestruturação da AVL
 - Rotação das sub-árvores que causaram $|FB| > 1$

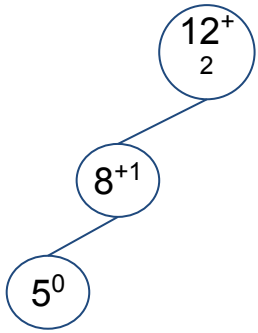
AVL

- Left-left case:
 - Ocorre quando $FB > 1$
 - A sub-árvore da esquerda está **mais pesada**
 - Deve ser executada uma rotação **simples à direita**



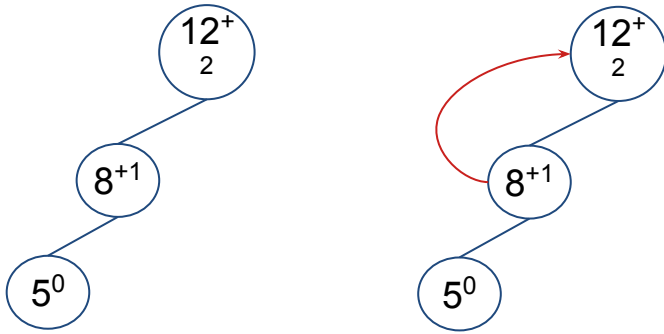
AVL

- Left-left case:



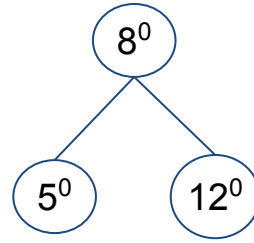
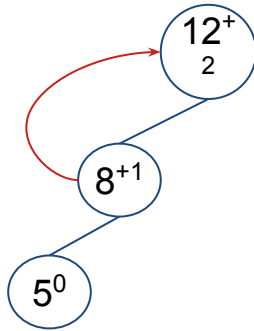
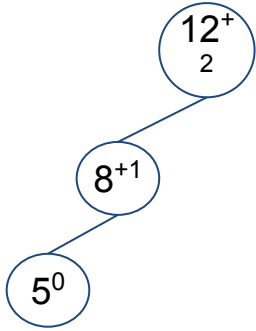
AVL

- Left-left case:



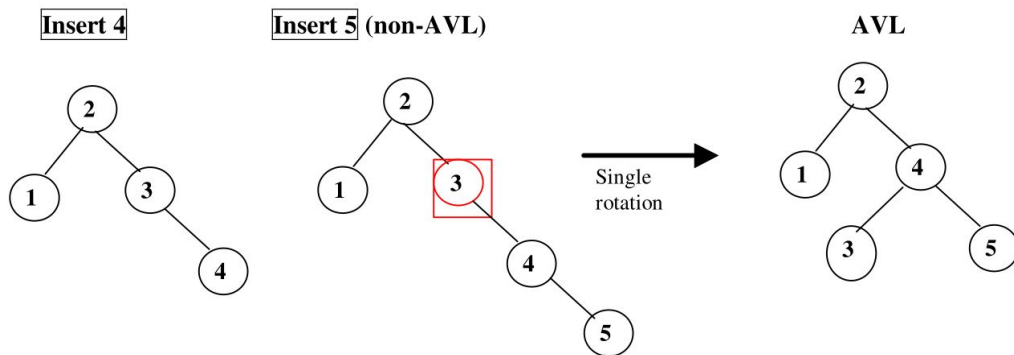
AVL

- Left-left case:



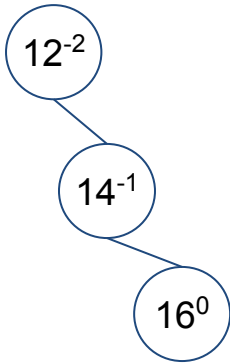
AVL

- Right-right case:
 - Ocorre quando $FB < -1$
 - A sub-árvore da direita está **mais pesada**
 - Deve ser executada uma rotação **simples à esquerda**



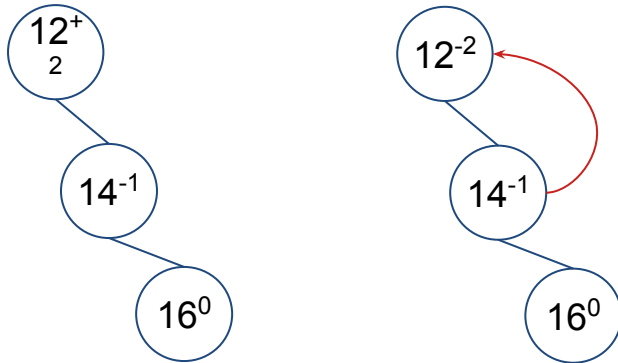
AVL

- Right-right case:



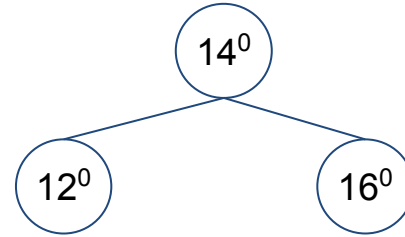
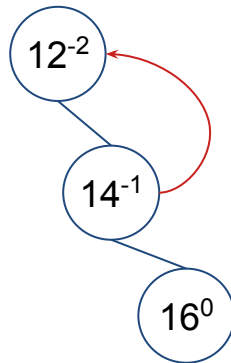
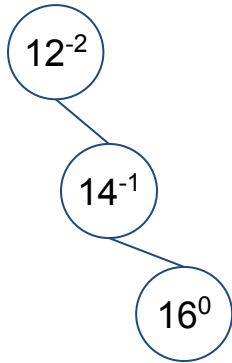
AVL

- Right-right case:



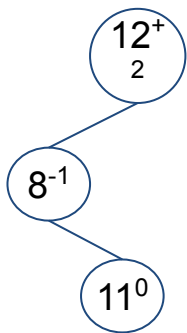
AVL

- Right-right case:



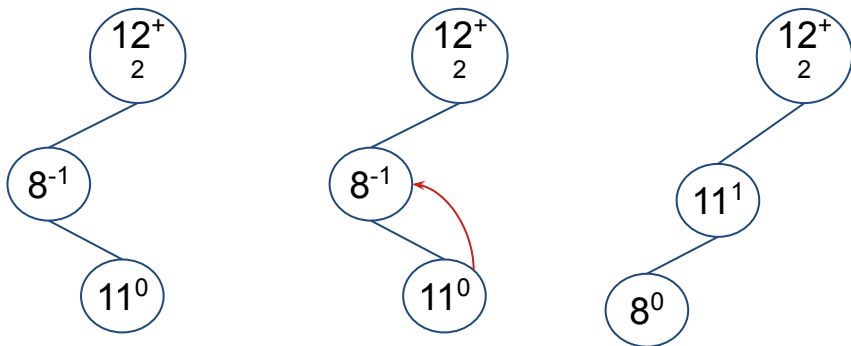
AVL

- Left-right case:
 - Ocorre quando $FB > 1$ e a sub-árvore da esquerda $FB < 0$
 - A sub-árvore da esquerda está **mais pesada**
 - Uma rotação **simples à direita** não resolve



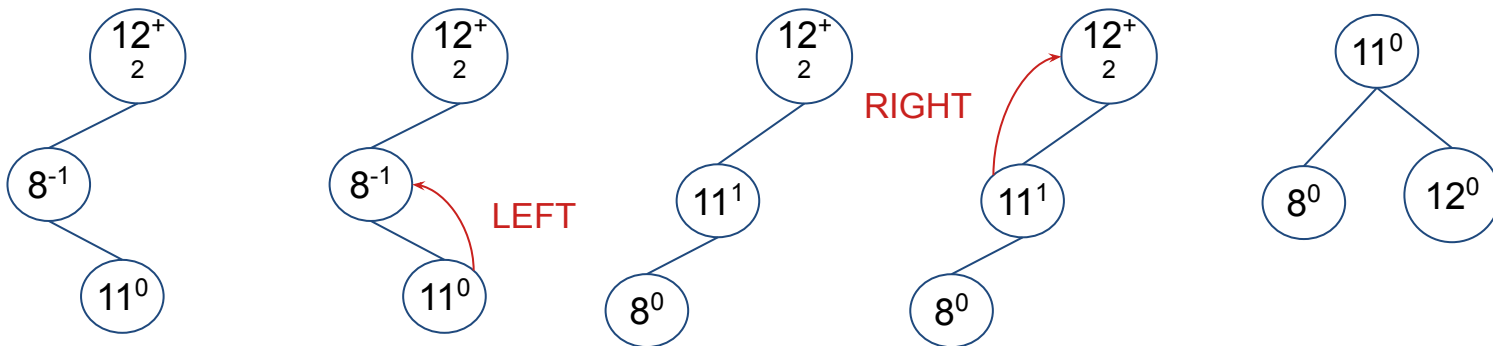
AVL

- Left-right case:
 - Ocorre quando $FB > 1$ e a sub-árvore da esquerda $FB < 0$
 - A sub-árvore da esquerda está **mais pesada**
 - Uma rotação **simples à esquerda** não resolve



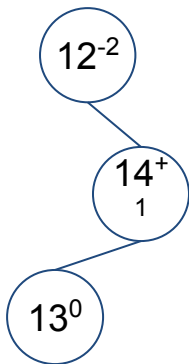
AVL

- Left-right case:
 - Ocorre quando $FB > 1$ e a sub-árvore da esquerda $FB < 0$
 - A sub-árvore da direita está **mais pesada**
 - Uma rotação **simples à esquerda** não resolve
 - Aplica-se mais uma **rotação à direita**



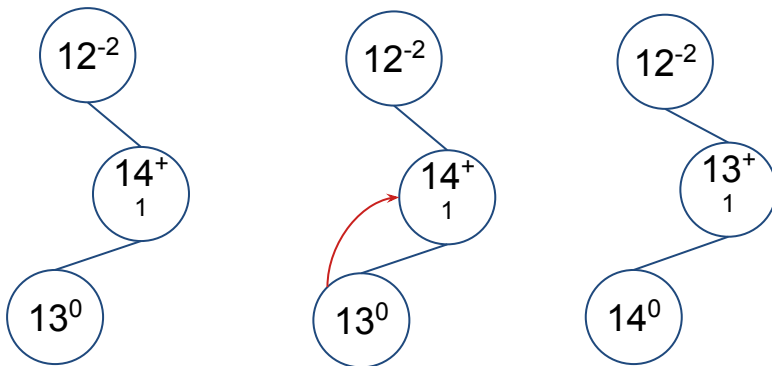
AVL

- Right-left case:
 - Ocorre quando $FB < -1$ e a sub-árvore da esquerda $FB > 0$
 - A sub-árvore da direita está **mais pesada**
 - Uma rotação **simples à direita** não resolve



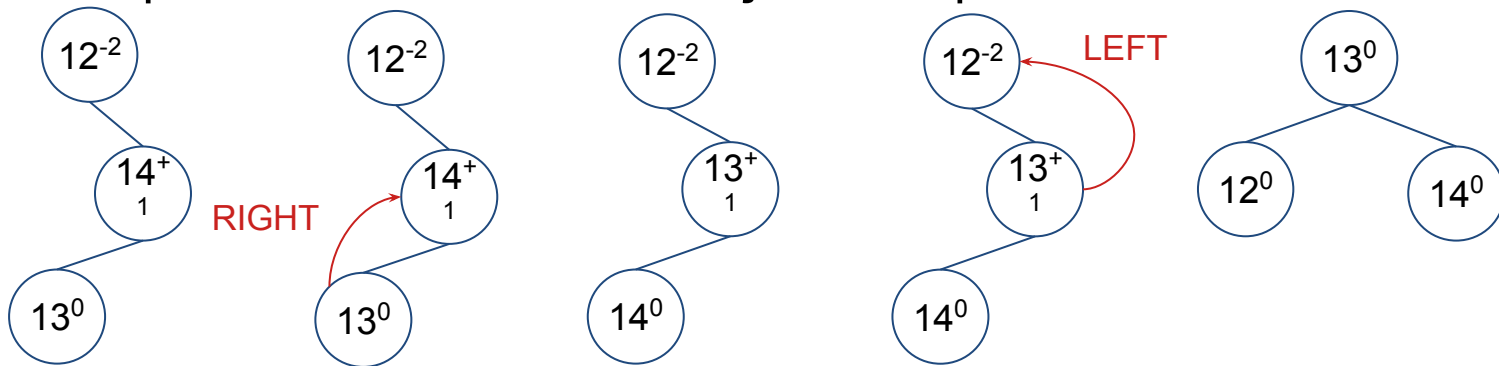
AVL

- Right-left case:
 - Ocorre quando $FB < -1$ e a sub-árvore da esquerda $FB > 0$
 - A sub-árvore da direita está **mais pesada**
 - Uma rotação **simples à direita** não resolve



AVL

- Right-left case:
 - Ocorre quando $FB < -1$ e a sub-árvore da esquerda $FB > 0$
 - A sub-árvore da direita está **mais pesada**
 - Uma rotação **simples à direita** não resolve
 - Aplicaca-se mais uma rotação à esquerda



AVL

- Let's see how it works

<https://www.cs.usfca.edu/~galles/visualization/AVLtree.html>

Exemplo

Árvore AVL (Ex)

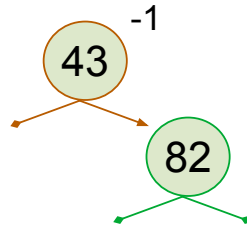
atual

43	82	99	14	21	30	3	51	9	27
0	1	2	3	4	5	6	7	8	9



Árvore AVL (Ex)

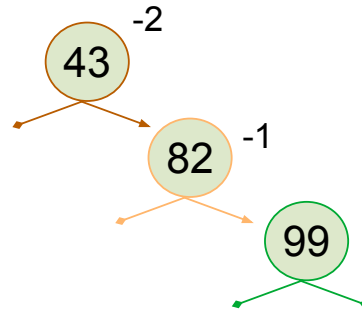
atual									
43	82	99	14	21	30	3	51	9	27
0	1	2	3	4	5	6	7	8	9



Árvore AVL (Ex)

atual

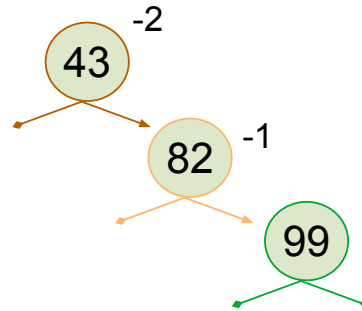
43	82	99	14	21	30	3	51	9	27
0	1	2	3	4	5	6	7	8	9



Árvore AVL (Ex)

atual									
43	82	99	14	21	30	3	51	9	27
0	1	2	3	4	5	6	7	8	9

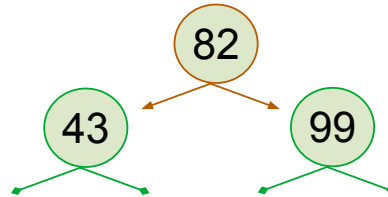
Rotação RR



Árvore AVL (Ex)

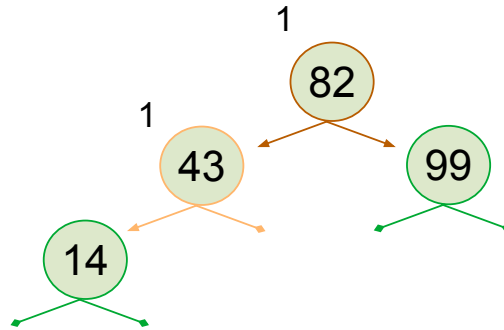
atual									
43	82	99	14	21	30	3	51	9	27
0	1	2	3	4	5	6	7	8	9

Rotação RR



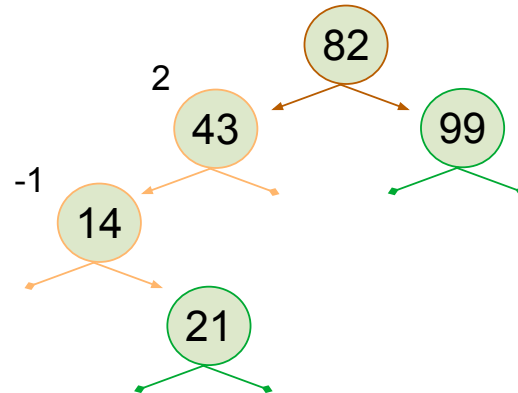
Árvore AVL (Ex)

atual									
43	82	99	14	21	30	3	51	9	27
0	1	2	3	4	5	6	7	8	9



Árvore AVL (Ex)

atual									
43	82	99	14	21	30	3	51	9	27
0	1	2	3	4	5	6	7	8	9

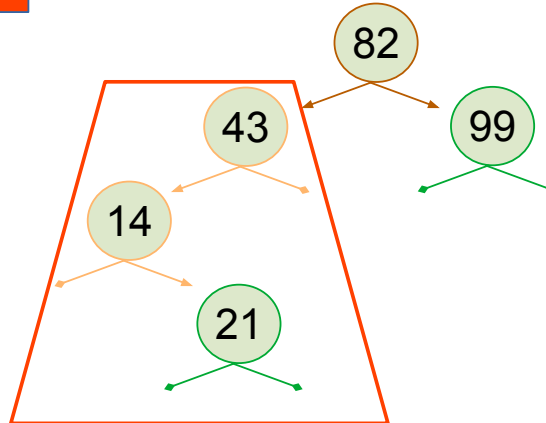


Árvore AVL (Ex)

				atual					
43	82	99	14	21	30	3	51	9	27
0	1	2	3	4	5	6	7	8	9

Rotação Left-Right
(LR)

- Chama a rotação RR para o filho da esquerda;
- Chama a rotação LL para a raiz;

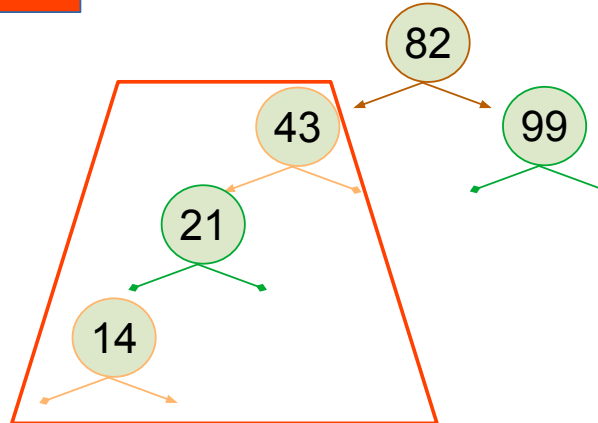


Árvore AVL (Ex)

				atual					
43	82	99	14	21	30	3	51	9	27
0	1	2	3	4	5	6	7	8	9

Rotação Left-Right
(LR)

- Chama a rotação RR para o filho da esquerda;
- Chama a rotação LL para a raiz;



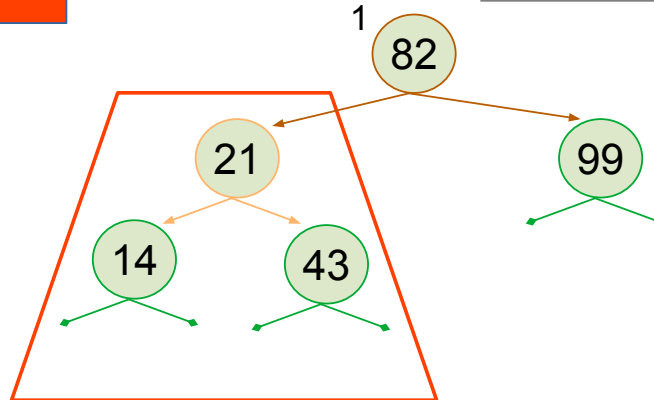
Árvore AVL (Ex)

43	82	99	14	21	30	3	51	9	27
0	1	2	3	4	5	6	7	8	9

atual

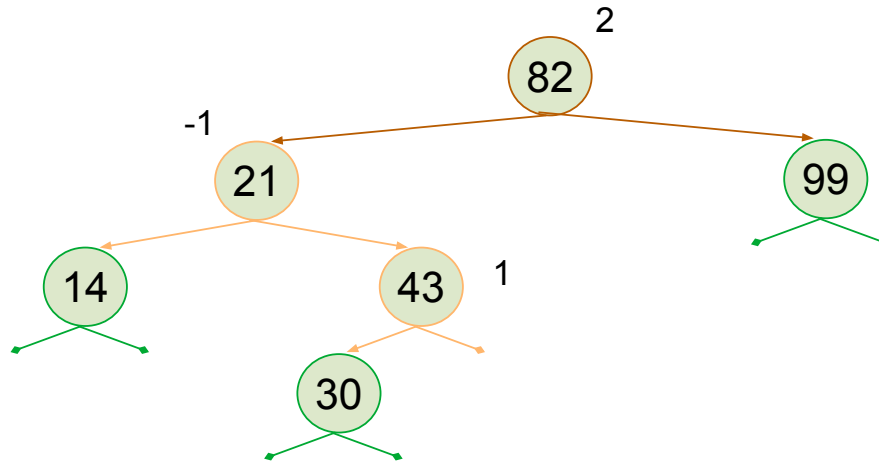
Rotação Left-Right
(LR)

- Chama a rotação RR para o filho da esquerda;
- Chama a rotação LL para a raiz;



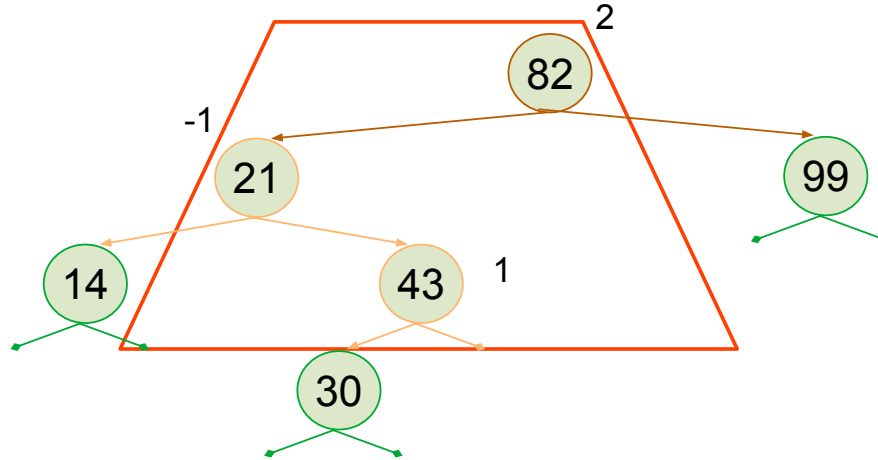
Árvore AVL (Ex)

43	82	99	14	21	30	3	51	9	27
0	1	2	3	4	5	6	7	8	9



Árvore AVL (Ex)

atual									
43	82	99	14	21	30	3	51	9	27
0	1	2	3	4	5	6	7	8	9

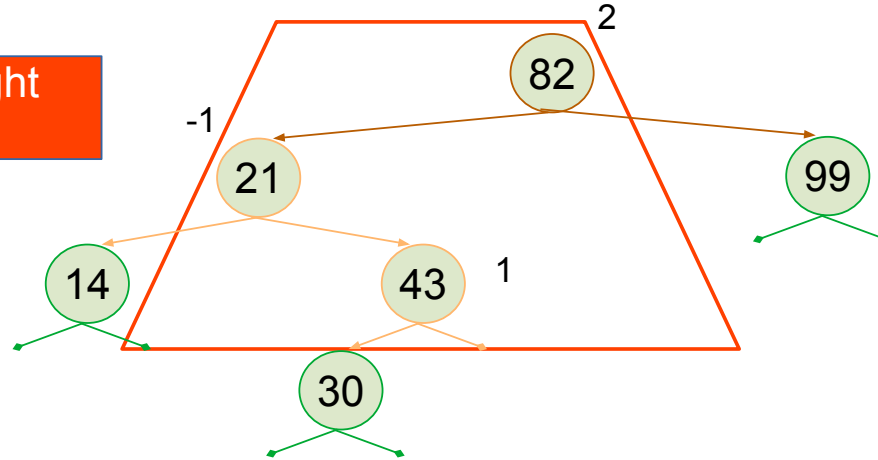


Árvore AVL (Ex)

43	82	99	14	21	30	3	51	9	27
0	1	2	3	4	5	6	7	8	9

atual

Rotação Left-Right
(LR)

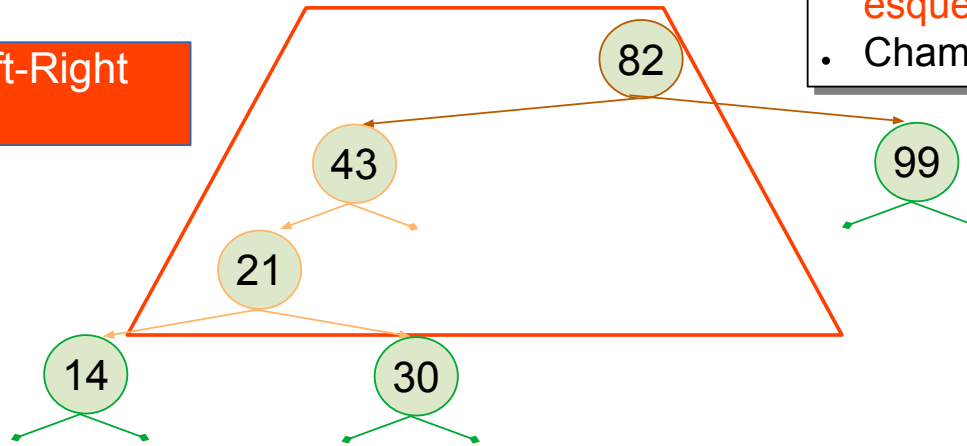


Árvore AVL (Ex)

					atual				
43	82	99	14	21	30	3	51	9	27
0	1	2	3	4	5	6	7	8	9

- Chama a rotação RR para o filho da esquerda;
- Chama a rotação LL para a raiz;

Rotação Left-Right
(LR)



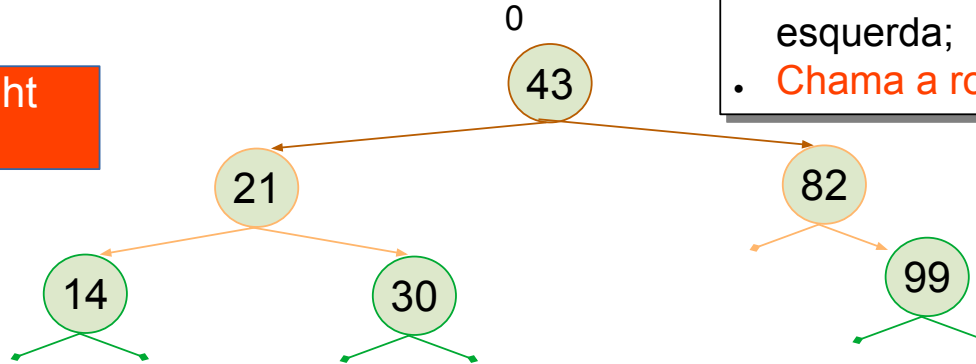
Árvore AVL (Ex)

43	82	99	14	21	30	3	51	9	27
0	1	2	3	4	5	6	7	8	9

atual

- Chama a rotação RR para o filho da esquerda;
- Chama a rotação LL para a raiz;

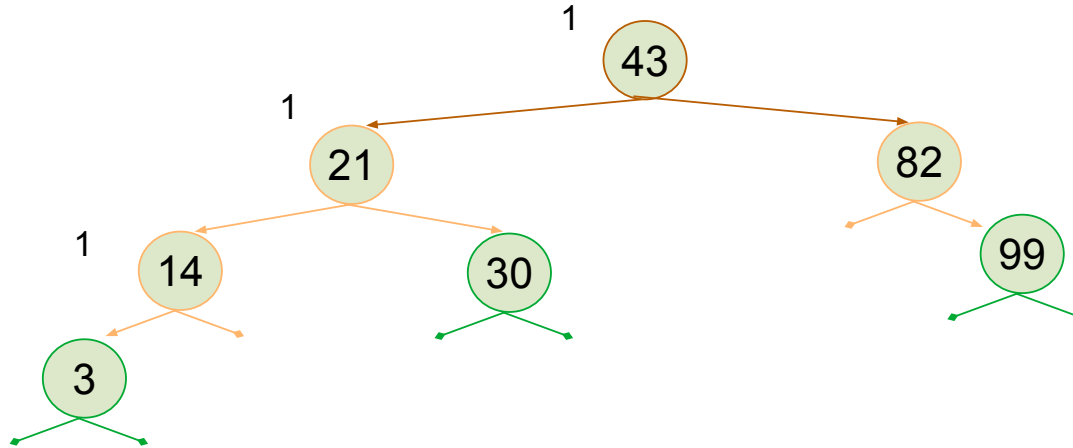
Rotação Left-Right
(LR)



Árvore AVL (Ex)

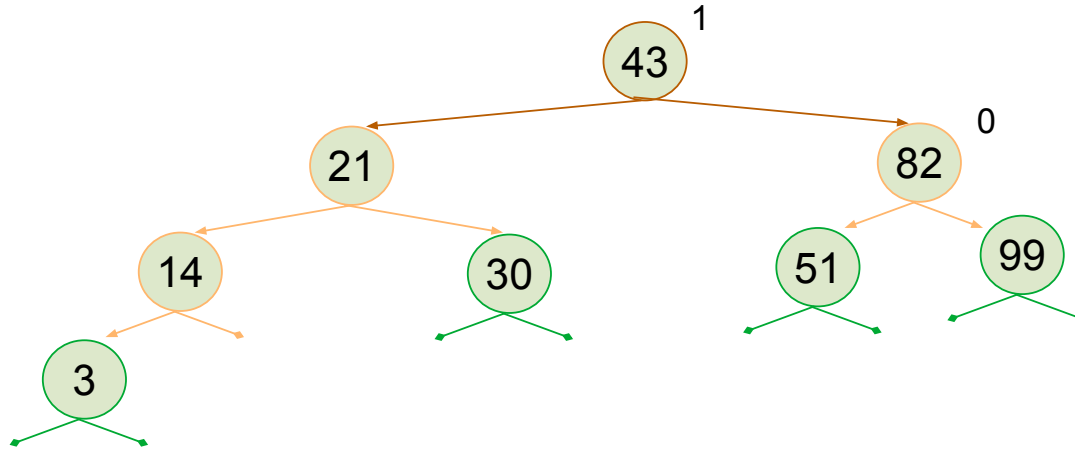
43	82	99	14	21	30	3	51	9	27
0	1	2	3	4	5	6	7	8	9

atual



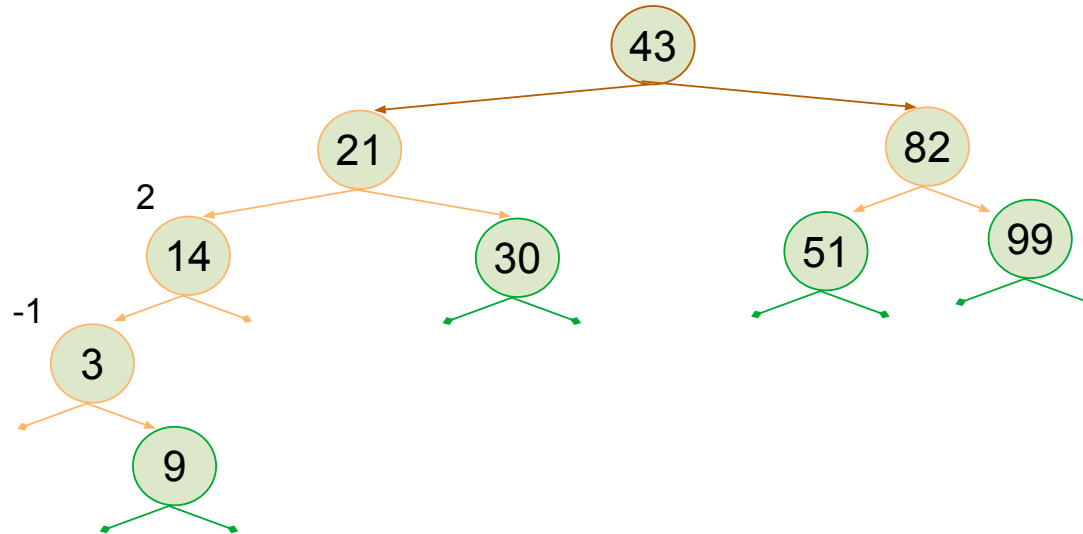
Árvore AVL (Ex)

43	82	99	14	21	30	3	51	9	27
0	1	2	3	4	5	6	7	8	9



Árvore AVL (Ex)

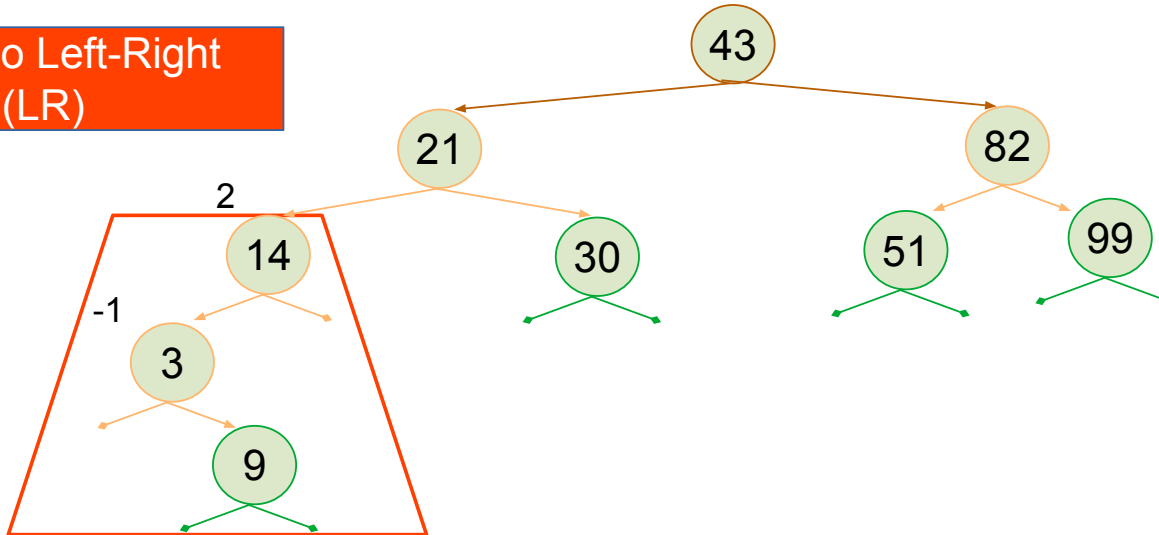
43	82	99	14	21	30	3	51	9	27
0	1	2	3	4	5	6	7	8	9



Árvore AVL (Ex)

43	82	99	14	21	30	3	51	9	27
0	1	2	3	4	5	6	7	8	9

Rotação Left-Right
(LR)

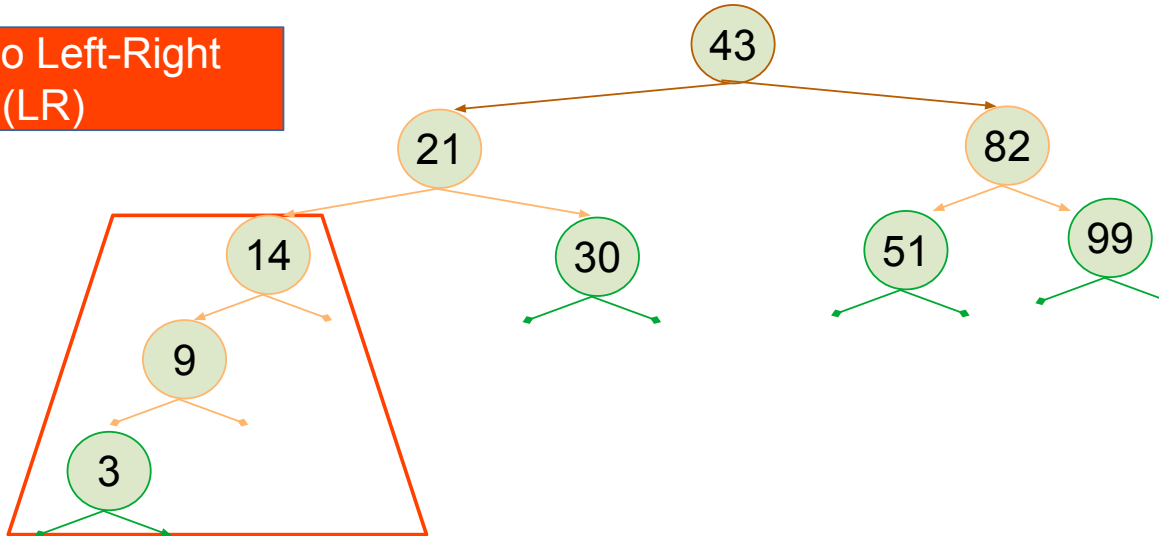


Árvore AVL (Ex)

43	82	99	14	21	30	3	51	9	27
0	1	2	3	4	5	6	7	8	9

atual

Rotação Left-Right
(LR)

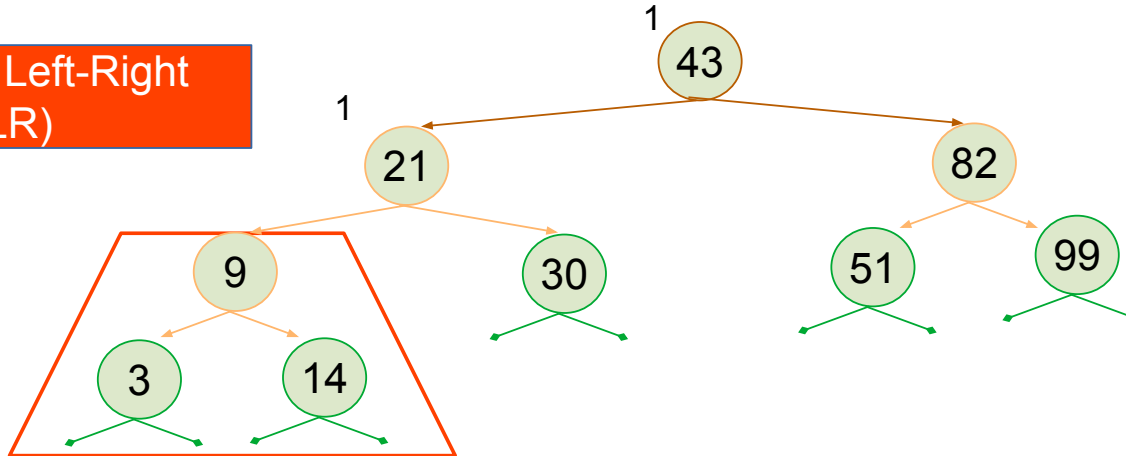


Árvore AVL (Ex)

43	82	99	14	21	30	3	51	9	27
0	1	2	3	4	5	6	7	8	9

atual

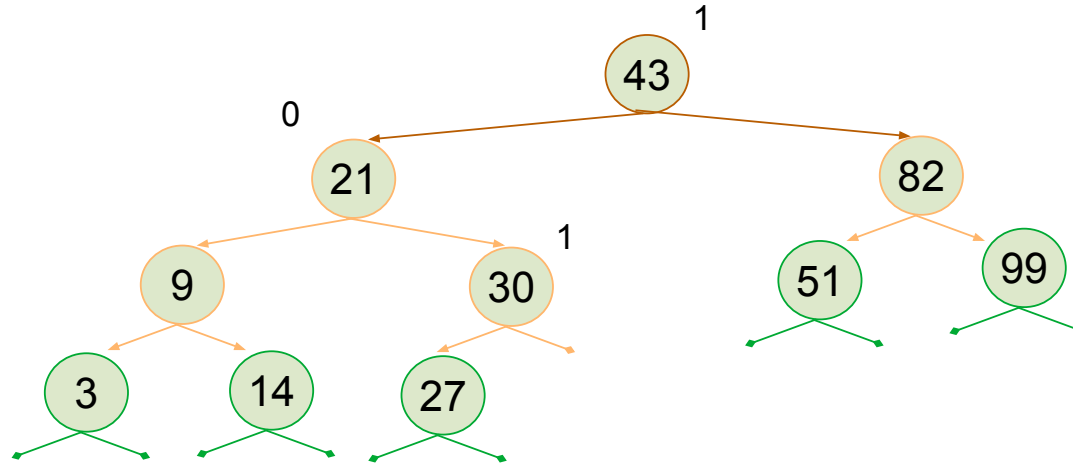
Rotação Left-Right
(LR)



Árvore AVL (Ex)

43	82	99	14	21	30	3	51	9	27
0	1	2	3	4	5	6	7	8	9

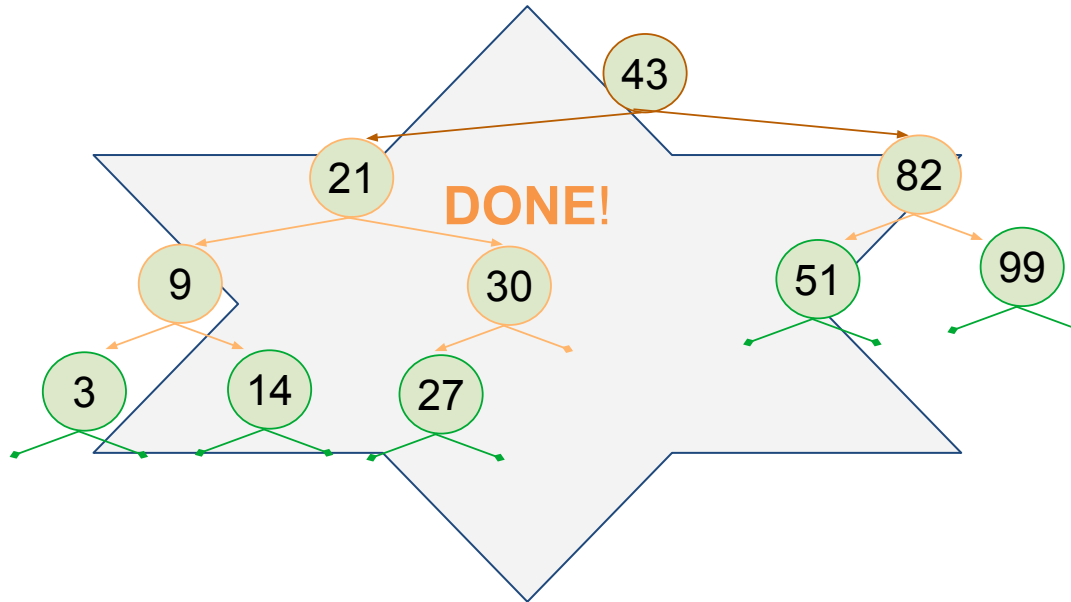
atual



Árvore AVL (Ex)

43	82	99	14	21	30	3	51	9	27
0	1	2	3	4	5	6	7	8	9

atual



AVL

- Pseudo-código (inserção)
 - Percorre a árvore em **pré-ordem** para adicionar o elemento
 - Calcula o **FB** do pai do nó inserido
 - $fb(no) = height(no \rightarrow left) - height(no \rightarrow right)$
 - Se $|FB| > 1$, árvore não respeita AVL
 - Aplica-se uma das **regras de balanceamento**
 - Repete-se a verificação dos nós antecessores até chegar a raiz

AVL

- Pseudo-código (inserção)
 - Regras:
 - Se **fb(no)** é **positivo**: calcula o **FB** do pai do nó inserido
 - Balanceamento do **no->left**
 - Negativo: **LR**
 - Positivo: **LL**
 - Se **fb(no)** é **negativo**: calcula o **FB** do pai do nó inserido
 - Balanceamento do **no->right**
 - Negativo: **RR**
 - Positivo: **RL**